Group Encryption with Oblivious Traceability

Khoa Nguyen[1⋆], Yanhong Xu[2,3⋆] ✉, Nam Tran[1], Willy Susilo[1], and Huaxiong Wang[4]

[1] School of Computing and Information Technology, University of Wollongong, Northfields Avenue, Wollongong, NSW 2522, Australia
[2] School of Computer Science, Shanghai Jiao Tong University, 800 Dongchuan Road, Shanghai 200240, China
[3] Key Laboratory of Intelligent Sensing System and Security (Ministry of Education), Hubei University, Wuhan, Hubei, China
✉ yanhong.xu@sjtu.edu.cn
[4] School of Physical and Mathematical Sciences, Nanyang Technological University, 21 Nanyang Link, Singapore 637371

Abstract. We revisit Group Encryption (GE)—an encryption analogue of group signatures introduced by Kiayias et al. (Asiacrypt 2007). A GE system simultaneously provides anonymity and traceability for receivers who are certified group members, enabling a range of privacy-preserving applications. While prior work has extensively addressed *how* to trace receivers in GE, the question of *why* a ciphertext should be traceable remains unexplored. Unlike group signatures, where opening can be justified by the signed content, tracing in GE poses a dilemma because the underlying plaintext is confidential.

To address this gap, we introduce *Group Encryption with Oblivious Traceability* (GEOT), an enhanced form of GE in which the traceability of a ciphertext ψ intended for receiver id and containing message $\mathbf{w}$ is governed by a public tracing policy $P(\mathsf{id}, \mathbf{w}) \in 0, 1$. Here, $P(\mathsf{id}, \mathbf{w}) = 0$ denotes traceability, whereas $P(\mathsf{id}, \mathbf{w}) = 1$ ensures non-traceability. The traceability status is known to the sender but remains hidden from all parties except the opening authority, which learns nothing about id in the non-traceable case. GEOT further supports message filtering and dynamic membership, following Nguyen et al. (PKC 2021). Filtering enforces that valid ciphertexts satisfy a public policy $F(\mathbf{w}) = 1$, while dynamicity enables users to join and leave the system over time.

We formalize GEOT with concise syntax and rigorous security notions, and present a modular construction based on standard cryptographic primitives: signatures, public-key encryption, and non-interactive zero-knowledge proofs. We also give a concrete instantiation from code-based assumptions supporting arbitrary tracing and filtering policies represented by polynomial-size Boolean circuits. In addition to expressive filtering and tracing functionalities, our scheme achieves significant efficiency improvements over existing post-quantum GE constructions.

⋆ This denotes equal contribution.

1 Introduction

Group encryption (GE), introduced by Kiayias, Tsiounis, and Yung (KTY) [42] as the encryption analogue of group signatures [23], is an appealing primitive that aims to simultaneously provide anonymity and accountability for message receivers who are registered members of a group. A GE scheme allows the sender of a message $\mathbf{w}$ to generate a verifiable ciphertext ψ such that (i) $\mathbf{w}$ can be recovered only by some anonymous receiver id who is a certified group member; (ii) an opening authority (OA) can trace ψ to id when necessary; and (iii) $\mathbf{w}$ satisfies certain message-filtering policy constraints. These properties make GE attractive from both theoretical and applied viewpoints. From a theoretical perspective, GE is considerably more intricate than group signatures because it requires proving the well-formedness of ciphertexts encrypted under hidden—but certified—public keys. Indeed, as observed by Kiayias et al. [42], GE implies hierarchical group signatures [70], a proper generalization of group signatures [12,13]. Practically, GE enables a range of privacy-preserving services [42,22,1,48,46,59,64]. Typical examples include encrypted email filtering, secure oblivious retrieval in corporate settings, and privacy-preserving financial transactions where encrypted transfers can still be traced for regulatory purposes.

Despite these advantages, GE inherits a structural weakness similar to that of group signatures: it cannot protect users' privacy against an all-powerful OA that can trace arbitrarily. Existing refinements, such as traceable GE [48], improve *how* tracing is performed but not *when* it should be invoked. In all existing schemes, the opener can freely deanonymize any ciphertext. This undermines the very privacy that GE seeks to achieve and is incompatible with modern expectations of accountability and due process.

The core conceptual gap lies in the absence of a clear rule specifying *why* a ciphertext should be traceable. In group signatures, opening a signer's identity is justified by the nature of an inappropriate or disputed message. In contrast, ciphertexts in GE conceal their contents, making it unclear under what circumstances the opening procedure ought to be triggered. For instance, if the OA suspects that some member is distributing illegal data and decides to open all ciphertexts to find the culprit, the privacy of every honest user is destroyed in the process. Such pervasive traceability is not only technically heavy-handed but also ethically questionable.

To capture a fairer balance between privacy and accountability, one may envision a tracing process that is *conditioned* on publicly verifiable criteria rather than left entirely to the discretion of the opening authority. Formally, such conditions could be represented by a *tracing policy* predicate $P(\mathsf{id}, \mathbf{w}) \in \{0, 1\}$, which specifies when tracing is justified based on the receiver's certified identity id and the message content $\mathbf{w}$. Similarly, it is often desirable to restrict the kinds of messages that can be validly encrypted, according to a *filtering policy* $F(\mathbf{w}) \in \{0, 1\}$ that captures format or compliance requirements. Together, these conceptual elements suggest a new direction: a group-encryption frame-

work where the ability to trace is no longer unconditional, but instead governed by explicit, policy-driven rules that are transparent to all participants.

To better understand the need for a more balanced approach, we illustrate three representative domains where receiver privacy and accountability are required simultaneously, and where a justified and *oblivious* tracing mechanism becomes indispensable.

Healthcare Data Collaboration. In large-scale medical research, hospitals and data-trust networks routinely share sensitive datasets with certified analysts. Receiver anonymity protects analysts from profiling and ensures that their data-access patterns remain confidential, while receiver accountability guarantees that if a dataset is mishandled or leaked, regulators can identify the certified recipient responsible. This dual requirement naturally justifies the use of GE, which offers both confidentiality and traceability.

However, in standard GE, the OA can open any ciphertext, enabling mass surveillance of compliant analysts and violating privacy regulations. A more reasonable system would allow tracing only when predefined privacy or clearance policies are violated, and would otherwise hide even the *possibility* of tracing. Such an "oblivious" form of traceability can be captured by a public predicate $P(\mathsf{id}, \mathbf{w})$ that encodes oversight conditions – for example, $P(\mathsf{id}, \mathbf{w}) = 0$ if the dataset $\mathbf{w}$ contains unredacted personal identifiers or if the analyst's clearance level is below the dataset's sensitivity label. Only when P evaluates to 0 may the OA learn the receiver's identity; otherwise, even the authority remains oblivious to whom the ciphertext was intended for. The filtering policy $F(\mathbf{w})$ can further enforce that only datasets containing mandatory consent tags or schema-conformant records are accepted, while full dynamicity accommodates continual membership changes in research consortia without reinitialization.

Financial-Compliance Communication. Financial institutions must communicate sensitive alerts and transaction reports among certified compliance officers. Receiver anonymity is essential for preserving investigative independence, whereas accountability is needed to establish who handled a particular case when regulatory review occurs. This again matches the conceptual goals of GE.

Yet conventional GE permits unfettered opening, making internal communications vulnerable to privacy abuse. A fairer design would tie traceability to explicit, public rules, preventing the OA from acting outside its mandate. In this setting, a predicate $P(\mathsf{id}, \mathbf{w})$ could encode statutory triggers such as "the alert $\mathbf{w}$ describes a transaction exceeding the reportable threshold without KYC verification," or "the receiver id lacks jurisdiction over the relevant account." Only if $P(\mathsf{id}, \mathbf{w}) = 0$ can the receiver be revealed; otherwise, tracing is infeasible and its status remains hidden. Here, $F(\mathbf{w})$ may guarantee that only authenticated, well-structured alerts are encrypted, and dynamicity supports officer rotations and role changes, ensuring long-term operability.

Whistleblowing and Internal Reporting. Whistleblowing systems epitomize the tension between privacy and accountability. Employees must be able to send encrypted reports to certified investigators anonymously, yet the organization must still be able to trace a report if it contains defamatory or classified

information. A GE-style design satisfies both objectives: confidentiality for reporters and conditional accountability for investigators.

However, in standard GE, pervasive traceability allows a malicious authority to open every report, compromising trust in the entire process. To prevent this, traceability should depend on an explicit policy predicate and remain hidden and unusable otherwise. For instance, $P(\mathsf{id}, \mathbf{w})$ may return 0 if the report $\mathbf{w}$ includes classified content or targets an incorrect investigative division. When P returns 0, the authority can reveal the investigator's identity; when P returns 1, even the authority learns nothing about who received the report. A filtering policy $F(\mathbf{w})$ can require that reports contain proper metadata (e.g., timestamps, case categories, or evidence references), and full dynamicity allows the investigative roster to evolve without disrupting ongoing processes.

Across these domains – healthcare collaboration, financial compliance, and whistleblowing – the need for receiver accountability justifies the use of group encryption, while the risk of privacy violations by an omnipotent opener calls for *oblivious traceability*. A tracing mechanism that is both conditional and hidden provides cryptographic fairness: tracing can occur only when justified by a public predicate, and otherwise even the authority remains ignorant of receiver identities. Message filtering and dynamic membership strengthen the model's deployability, but the essential innovation lies in ensuring that the *reason* for tracing is both explicit and privacy-preserving.

Our Contributions and Techniques. Building upon these motivations, we formalize and realize the notion of *Group Encryption with Oblivious Traceability* (GEOT), which revolutionizes the tracing function in GE by tying it to public, verifiable conditions. In GEOT, whether a ciphertext is traceable depends on a public predicate $P(\mathsf{id}, \mathbf{w})$, and the traceability status is oblivious to all parties except the OA, which learns nothing in the non-traceable case. Our formulation provides a cryptographic setting that is fair to both users and authorities: misbehaving users cannot evade tracing, and misbehaving authorities cannot compromise the privacy of honest users. GEOT further supports message filtering and full dynamicity as suggested in [59].

We equip GEOT with concise syntax – comprising only seven algorithms compared to thirteen in [59] – and rigorous security notions, including message secrecy, several forms of anonymity, and unforgeability. We next provide a generic construction of GEOT satisfying the proposed model and relying on commonly used cryptographic building blocks. Somewhat surprisingly, we show that a secure GEOT system can be generically constructed from the same building blocks that are used for building ordinary GE systems. This generic construction is meaningful in that it isolates the essential components of GE and demonstrates that the new functionality of *oblivious traceability* can be incorporated together with previously studied features such as message filtering and full dynamicity [59], all within a single unified framework. In particular, it shows that these enhancements can be achieved without introducing additional cryptographic assumptions or complex new machinery, and can be seamlessly integrated into existing GE schemes in a modular manner. Our framework can naturally lead to

concrete instantiations using existing cryptographic tools and techniques from pairings and lattices. For the sake of diversity of assumptions, we next pay attention to realizing GEOT from codes. A noticeable obstacle in the code-based setting is the current lack of a signature scheme[1] with efficient protocols [20] for proving in zero-knowledge the possession of a valid message–signature pair, which is utilized in our generic construction and which has been constructed from pairings [21,10] and lattices [45,40,17]. We nevertheless overcome this obstacle by using a code-based Merkle-tree accumulator [61], instead of a full-fledged standard-model signature. This approach is inspired by the construction of fully dynamic group encryption (FDGE) in [59]. Our code-based GEOT scheme operates smoothly with general policies and is sufficiently expressive to support arbitrary tracing and filtering policies represented by polynomial-size Boolean circuits.

Further Applications of GEOT. Beyond the classical privacy–accountability balance, GEOT also enables a new class of applications in which the traceability of ciphertexts is not merely a defensive mechanism against abuse, but a *functional feature* that parties may wish to invoke intentionally. In particular, GEOT provides a controlled environment where senders can deliberately generate ciphertexts that are traceable under specific, publicly known predicates, allowing traceability itself to become a mechanism for enforcing responsibility, eligibility, or reward.

For instance, in privacy-preserving financial systems, GEOT can protect the anonymity of ordinary low-value transfers while permitting the tracing of transactions that violate anti-money-laundering policies or exceed taxable thresholds. In other domains, receivers may have incentives or obligations to accept traceable ciphertexts. A patient may encrypt her medical records to her doctor, who is accountable for handling them responsibly; the traceability predicate ensures that, in case of dispute, the doctor's role can be verified. An elderly benefactor may send the details of his will to his lawyer, authorizing disclosure only under predefined legal conditions; here, traceability captures both timing and custodial responsibility. Similarly, a whistleblower might send evidence to a journalist who is authorized to reveal the information only if the content meets certain public-interest criteria. In all these cases, the ability to encode, in the policy predicate $P(\mathsf{id}, \mathbf{w})$, *when and why* a receiver may be traced transforms traceability from a passive safeguard into an active part of the communication semantics.

From a different angle, GEOT also supports voluntary or incentive-driven use cases. Users may intentionally produce traceable ciphertexts – e.g., when a system grants proofs, rewards, or credentials to those who send or receive verifiably traceable messages. Such flexibility is enable by the traceability condition in GEOT, which is explicit, policy-based, and cryptographically verifiable. Altogether, these examples illustrate that GEOT does not merely restrict the authority's tracing power, but generalizes it into a programmable mechanism

[1] Existing code-based signatures schemes such as Stern's [69], CFS [26], WAVE [29] are not compatible with zero-knowledge proofs due to their reliance on random oracles.

for modeling accountability, compliance, and incentive alignment across diverse privacy-sensitive ecosystems.

Defining GEOT. The definitions of GEOT are inspired by those of FDGE [59] and bifurcated anonymous signatures (BiAS) [47], and – to some extent – attribute-based signatures (ABS) [52]. Concretely, GEOT encompasses the full dynamicity feature of FDGE, the oblivious and predicate-based tracing feature of BiAS and the policy-based filtering feature of ABS. It is worth noting that, although GEOT is a richer and more advanced primitive than FDGE, we manage to provide a syntax that is much simpler, comprising 7 algorithms, compared to 13 algorithms of FDGE. This simplification is the result of three major changes in the definitions: (i) the key generations of the group manager (GM) and the OA are unified into the system setup algorithm; (ii) the filtering mechanism is simply defined in terms of predicates (while in GE [42], traceable GE [48] and FDGE [59], messages are associated with heavy machinery of NP-relations); (iii) the proving algorithm (used in previous systems to show the well-formedness of ciphertexts) is incorporated into the encryption algorithm.

The lifetime of a GEOT system is divided into time epochs belonging to an ordered time space $\mathcal{T} = \{\tau_1, \tau_2, \ldots\}$ of polynomial cardinality. Any GEOT system is further associated with a space $\mathcal{ID}$ of receiver identifiers, a message space $\mathcal{W}$, a collection $\mathcal{F} = \{F_\tau : \mathcal{W} \to \{0,1\}\}$ of filtering policies and a collection $\mathcal{P} = \{P_\tau : \mathcal{ID} \times \mathcal{W} \to \{0,1\}\}$ of tracing policies. Any ciphertext ψ in the system is bound to a time period $\tau \in \mathcal{T}$. If ψ contains message $\mathbf{w} \in \mathcal{W}$ and is intended for receiver $\mathsf{id} \in \mathcal{ID}$, then ψ is deemed valid when $F_\tau(\mathbf{w}) = 1$, traceable when $P_\tau(\mathsf{id}, \mathbf{w}) = 0$ and non-traceable when $P_\tau(\mathsf{id}, \mathbf{w}) = 1$.

A GEOT system must satisfy the following properties: correctness, privacy and unforgeability. While `correctness` can be defined naturally, formalizing security notions for GEOT is quite non-trivial, mainly due to the introduction of tracing (and filtering) policies. In particular, there are three types of `privacy` that we should capture.

- **Message secrecy** protects the receiver from a CCA2 adversary who tries to learn information about the encrypted message and who can corrupt the whole system except for the receiver of the challenge ciphertext. Here, compared to the corresponding notion in FDGE, we have to make a restriction on the challenge tuple returned by the adversary so that to prevent it from trivially winning by outputting $(\mathsf{id}, \mathbf{w}_0, \mathbf{w}_1)$ such that $P(\mathsf{id}, \mathbf{w}_0) \neq P(\mathsf{id}, \mathbf{w}_1)$.
- **Type-1-anonymity** protects the identity of the receiver as well as whether a given ciphertext is traceable or not against a CCA2 adversary. We allow the adversary to adaptively query the opening oracle and even allow it to output a challenge tuple $(\mathsf{id}_0, \mathsf{id}_1, \mathbf{w})$ such that $P(\mathsf{id}_0, \mathbf{w}) \neq P(\mathsf{id}_1, \mathbf{w})$.
- **Type-2-anonymity** further ensures that in the non-traceable case, nothing about the receiver's identity can be learned by an adversary who can corrupt the whole system except the two challenge users.

Defining `unforgeability` for GEOT requires additional care. Intuitively, we would like to ensure that one can generate a valid ciphertext ψ if: (i) the intended

receiver id is active in the system (i.e., id has joined and has not been revoked); (ii) the plaintext $\mathbf{w}$ passes the filtering policy, i.e., $F(\mathbf{w}) = 1$; (iii) id is traceable if and only if $P(\mathsf{id}, \mathbf{w}) = 0$. However, due to privacy properties, there is no efficient mechanism to determine whether a given ciphertext ψ does comply with these conditions. Hence, similar to the model of BiAS, here we also need to introduce an extractable mode for the system to determine if an adversary has produced a falsely accepted ciphertext.

Generically Constructing GEOT. Once the definitional framework has been well established, we turn our attention to designing systems satisfying the proposed security requirements, based on well-studied assumptions. To that end, we provide a generic construction of GEOT, which is essentially the first generic construction of fully dynamic group encryption, which was not given in previous works such as [59] and [64].

We first identify the cryptographic ingredients necessary for a modular construction of GEOT. Since a legitimate ciphertext must exhibit a certified group membership of the intended receiver, we would need an ordinary digital signature as a building block for GEOT. Now, the fact that only the intended receiver and the OA can potentially recover some information from a ciphertext suggests the necessity of public-key encryption (PKE) systems (with an extra requirement of key privacy for the system owned by the receiver). Finally, as a valid ciphertext must be honestly computed and must reveal no private information to the public, we would also need to employ some non-interactive zero-knowledge (NIZK) proof/argument system.

We next show that the commonly used ingredients identified above, i.e., ordinary signature, PKEs and NIZK, are indeed sufficient for designing GEOT for arbitrary tracing and filtering policies. The high-level ideas underlying the construction are as follows. Each potential user of the system is equipped with a key pair for a key-private PKE. The GM possesses a signing-verification key pair for a signature scheme, which will be used for certifying users' public keys. The OA, on the other hand, owns a key pair for another (not necessarily key-private) PKE. To send a message $\mathbf{w}$ to user id at time τ, the sender first makes sure that: (i) $\mathbf{w}$ is allowed under the filtering policy F_τ; (ii) id has been certified by the GM, and is not in the updated revocation list. Then, the sender generates two sub-ciphertexts: (i) $\mathbf{c}_{\mathsf{R}}$ that encrypts $\mathbf{w}$ under the receiver's public key; and (ii) $\mathbf{c}_{\mathsf{OA}}$ that encrypts the value $(1 - P_\tau(\mathsf{id}, \mathbf{w})) \cdot \mathsf{id}$ under the OA's public key, where $P_\tau(\cdot, \cdot)$ is the tracing predicate at time τ. Then the sender generates an NIZK argument to prove in zero-knowledge that all the steps have been done correctly. The final ciphertext ψ then is set as the triple $(\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$. Verification of ψ consists of verifying π. The receiver id can recover $\mathbf{w}$ by decrypting $\mathbf{c}_{\mathsf{R}}$. Meanwhile, depending on the predicate value $P_\tau(\mathsf{id}, \mathbf{w})$, the OA can learn from $\mathbf{c}_{\mathsf{OA}}$ either id (traceable case) or $\mathbf{0}$ (non-traceable case). The correctness and security properties of the resulting GEOT system follow quite naturally from those of the employed building blocks.

Concretely Instantiating GEOT. Since the building blocks used in the above generic construction have been realized based on pairings (e.g., in the stan-

dard model via [44,43,37]) and lattices (e.g., via [40] and [65] (standard model) and [73,17] (random oracle model)), we know that the respective pairing-based and lattice-based instantiations can, in principle, be achieved. However, when it comes to designing GEOT from code-based assumptions, it is less clear how to obtain a working solution, due to the current lack of a standard-model full-fledged signature in the code-based setting – as discussed above. Nevertheless, we observe that the problem can still be overcome by slightly departing from the generic construction approach and using a Merkle tree – which can be seen as a weak form of signature – to manage the enrollments and revocations of users.

At a high level, our instantiation of GEOT is inspired by the code-based FDGE scheme of [59], and employs an updatable Merkle tree accumulator [61] for the "signature layer", anonymous CCA2 secure versions [62,57] of McEliece scheme [53] for the "encryption layers". A remarkable feature of our scheme is that it allows *arbitrary choices* of filtering and tracing policies, represented by polynomial-size Boolean circuits. To develop the supporting "zero-knowledge layer", instead of using Stern-like [69] zero-knowledge protocols as in [30,61] for proving ciphertext validity, we employ the techniques in [63] which are built upon the VOLE-in-the-Head (VOLEitH) paradigm [9,7]. By adapting these techniques to handle the algebraic relations (i.e., relations among code-based components of Merkle hash tree and McEliece encryption) and non-algebraic relations (i.e., satisfiability of Boolean circuits representing our filtering and tracing policies), we obtain significant efficiency improvements over existing GE schemes from post-quantum assumptions: the lattice-based schemes from [46,64] and the code-based scheme from [59].

Table 1: Comparison of key sizes, ciphertext size and proof size between existing post-quantum GE schemes and our code-based GEOT scheme. The data achieving 80-bit security for [46,64] are taken from [64, Table 1]. The data achieving 128-bit security for [59,60] are taken from [60, Table 1]. The data for our scheme are calculated based on parameters specified in Table 2. For filtering and tracing policies, we address relatively large Boolean circuits of size 10^5. The group size $N = 2^{10}$ is chosen for all four schemes.

	λ	GM		OA		User		Ciph.	Proof
		pk	sk	pk	sk	pk	sk		
[46]	80	68.60 GB	482.55 GB	2.37 GB	38.86 GB	2.37 GB	38.86 GB	2.36 TB	3728 TB
[64]	80	0.54 KB	1.08 KB	10.85 KB	129.50 KB	9.40 MB	112.30 MB	0.13 MB	10.32 GB
[59]	128	0.43 KB	0.85 KB	2.26 MB	6.90 MB	2.26 MB	6.90 MB	1.70 KB	46.26 GB
Ours	128	8 KB	0.25 KB	3.40 MB	6.19 MB	3.40 MB	6.19 MB	2 KB	61.43 MB

In Table 1, we give a comparison between the key sizes, ciphertext size and proof size of our GEOT scheme and the GE schemes from [46,64,59]. Our scheme not only offers much more expressive tracing and filtering functionalities but also noteworthy gains in terms of efficiency. Nevertheless, we remark that the

scheme is still far from being practical, due to the need to encrypt McEliece public keys and to prove highly sophisticated relations capturing ciphertext well-formedness. We leave the question of achieving practically efficient (code-based) GEOT schemes as a fascinating open question for future investigations.

Related work. In their pioneering work that introduced GE, Kiayias et al. also provided a concrete construction based on the Decisional Composite Residuosity and the Decisional Diffie-Hellman assumptions. They used an interactive ZK proof of ciphertext well-formedness, which can be made non-interactive in the random oracle model using the Fiat-Shamir transformation [31]. Cathalo et al. [22] later suggested a non-interactive protocol based on pairings in the standard model. El Aimani and Joye [1] subsequently provided various efficiency improvements for pairing-based GE. The first post-quantum construction of GE was presented by Libert et al. [46] who put forward a lattice-based scheme.

Regarding enhanced variants of GE, Libert et al. [48] proposed a refined tracing mechanism inspired by that of traceable signatures [41]. In this setting, the OA can release a user-specific trapdoor which enables public tracing of ciphertexts sent to that user without violating the privacy of other users. However, a malicious OA can still violate the privacy of any user of its choice. Izabachène et al. [38] suggested mediated traceable anonymous encryption – a related primitive that addresses the problem of eliminating subliminal channels. Nguyen et al. [59] defined FDGE, which supports dynamic user enrollments and revocations, and described a code-based instantiation. Nguyen et al.'s construction also allows filtering of messages, based on either a "permissive policy" (that permits only messages containing certain keywords) or "prohibitive policy" (that prohibits all messages containing a substring similar to a given keyword). Subsequently, Pan et al. [64] presented a lattice-based FDGE construction that improves over [59] in several aspects.

Bifurcated anonymous signature, introduced in a recent work by Libert et al. [47], enables predicate-based oblivious tracing in the context of anonymity-oriented signatures. The novel idea underlying BiAS yields a good balance between anonymity and accountability. In [47], the authors also provided a generic construction of BiAS for arbitrary predicates in NC1, which allows standard-model instantiations from lattices and pairings. However, the technical ingredients required for their generic construction are not available in the code-based setting. In contrast, without much technical difficulty, one can design (the first) BiAS from codes based on a code-based GEOT (such as the construction discussed in the present work). Although both primitives improve group signature/encryption via the oblivious traceability feature, GEOT is more than just an encryption analogue of BiAS. As we argued earlier, oblivious traceability is not only an enhanced property but also seems to be the "right" notion for tracing in the GE context.

In a more recent work, Nguyen et al. [58] proposed the notion of Multimodal Private Signatures (MPS), which generalizes BiAS by relaxing the absolute accountability of users and categorizing users' traceable information into different

levels of anonymity. We leave the problem of translating the ideas of MPS into the GE setting as an interesting question for future investigations.

The major tools for building those privacy-preserving cryptographic constructions are ZK proof [35] and argument [33,19] systems that allow proving the truth of a statement while revealing no additional information. Until recently, many ZK proof/argument systems used in code-based privacy-preserving cryptography followed Stern's framework [69]. Variants of Stern's protocol have been employed to design privacy-preserving constructions, such as proofs of plaintext knowledge [56], accumulators and range proofs [61], ring signatures [54,28,55,18,61], group signatures [30,2,61], proofs of valid openings for commitments and proofs for general relations [39,49], GE [59], policy-based signatures [24,71] and ABS [49].

Upon the invention of the VOLEitH techniques by Baum et al. [9], VOLEitH proof systems have gained traction in the past two years. Cui et al. [27] first constructed a standard signature scheme via the VOLEitH techniques, based on the regular syndrome decoding (SD) problems. Baum et al. [7] proposed several optimizations to further reduce the proof sizes at the cost of slightly larger computation time. Bidoux et al. [15] then constructed standard signatures based on the rank SD and minRank problems. Ouyang et al. [63] adapted the VOLEitH proof systems to handle degree-d polynomial constraints, and built efficient ring signatures, group signatures, and fully dynamic ABS. Bettaieb et al. [14] introduced VOLE-friendly modelings for SD and permuted kernel problems, and constructed efficient standard signatures and ring signatures based on these problems. Quite recently, Chiang et al. [25] proposed threshold ring signatures from AES and VOLEitH proof systems.

Organization. The rest of the paper is organized as follows. In Section 2, we describe the definitions and security requirements of GEOT. In Section 3, we present a generic construction of GEOT from commonly used cryptographic building blocks. Our code-based instantiation of GEOT is described in Section 4. Due to space restrictions, several materials, including the reminders on generic cryptographic primitives, backgrounds on the necessary code-based building blocks, supporting zero-knowledge protocols and security proofs for the code-based scheme are deferred to the Appendix.

2 GEOT: Syntax and Security Definitions

In this section, we formalize the primitive of GEOT. A GEOT system is an enhanced group encryption system that supports conditional and oblivious tracing of ciphertexts, policy-based message filtering, as well as dynamic user enrollments and revocations.

2.1 Syntax

A GEOT system involves the following entities: a trusted authority who initializes the group parameters; senders, receivers and verifiers of ciphertexts; a group manager (GM) who is in charge of enrollments and revocations of receivers to

and from the group; and an opening authority (OA) who can identify receivers of ciphertexts that are traceable.

The lifetime of a GEOT system is divided into time epochs belonging to an ordered time space $\mathcal{T} = \{\tau_1, \tau_2, \ldots\}$ of polynomial cardinality. All entities of the system have access to a group database $\mathsf{info} = (\mathsf{info}_{\tau_1}, \mathsf{info}_{\tau_2}, \ldots)$, where each entry info_τ is announced and authenticated by the GM at the beginning of time epoch τ. This entry allows the public to determine which receivers are active (i.e., those who have joined the group but have not been revoked) at time epoch τ. The GM additionally maintains a table $\mathbf{reg}$ containing registration information of all registered receivers and a regularly updated set $\mathcal{S}$ of receivers to be revoked at the next time epoch. We remark that the well-formedness of $\mathbf{reg}$ can be publicly verified by checking if there is any inconsistency with info_τ.

A GEOT system is further associated with a space $\mathcal{ID}$ (where each $\mathsf{id} \in \mathcal{ID}$ contains identifying information of the respective receiver), a message space $\mathcal{W}$, a collection $\mathcal{F} = \{F_\tau : \mathcal{W} \to \{0,1\}\}$ of message filtering policies and a collection $\mathcal{P} = \{P_\tau : \mathcal{ID} \times \mathcal{W} \to \{0,1\}\}$ of tracing policies, each of which specifies whether an intended receiver id of a message w should be traceable or not. We assume that at epoch τ, the system information info_τ uniquely defines a filtering policy F_τ and a tracing policy P_τ.

Any ciphertext ψ in the system is bound to a time epoch $\tau \in \mathcal{T}$, and can only be verified, decrypted, traced relative to the policies specified by info_τ. If ψ contains message $w \in \mathcal{W}$ and is intended for receiver $\mathsf{id} \in \mathcal{ID}$, then ψ is said to pass the filter when $F_\tau(w) = 1$, to be traceable when $P_\tau(\mathsf{id}, w) = 0$ and to be non-traceable when $P_\tau(\mathsf{id}, w) = 1$.

$\mathsf{Setup}(1^\lambda)$ On input security parameter 1^λ, this probabilistic algorithm generates public parameters pp, a secret key $\mathsf{sk}_{\mathsf{GM}}$ for the GM and a secret key $\mathsf{sk}_{\mathsf{OA}}$ for the OA. It also initializes the group database $\mathsf{info} := \emptyset$ and a registration table $\mathbf{reg} := \emptyset$. We assume that pp contains the specifications of $(\mathcal{T}, \mathcal{ID}, \mathcal{W}, \mathcal{F}, \mathcal{P})$ and is an implicit input of all other algorithms.

$\langle \mathsf{Join}(\mathsf{info}), \mathsf{Issue}(\mathsf{sk}_{\mathsf{GM}}, \mathsf{info}, \mathbf{reg}) \rangle$ This is an interactive protocol between the GM and a potential group member who would like to join the group. If the protocol completes successfully, the user obtains a unique identifier $\mathsf{id} \in \mathcal{ID}$ and a membership certificate $\mathsf{cert}_{\mathsf{id}}$ that are publicly known, together with a secret key $\mathsf{sk}_{\mathsf{id}}$ known only to them. The GM, on the other hand, appends the registration information $(\mathsf{id}, \mathsf{cert}_{\mathsf{id}})$ to the table $\mathbf{reg}$.

$\mathsf{GUpdate}(\mathsf{sk}_{\mathsf{GM}}, \mathcal{S}, \mathsf{info}, \mathbf{reg})$ Executed by the GM, this algorithm periodically updates the group information and advances the time epoch. It takes as inputs $\mathsf{sk}_{\mathsf{GM}}$, a set of to-be-revoked users $\mathcal{S}$, the current group database info, the current registration table $\mathbf{reg}$, and then updates the information table with the new group information $\mathsf{info} := \mathsf{info} \cup \{\mathsf{info}_{\tau_{\mathrm{new}}}\}$, where τ_{new} is the next time epoch. We remark that $\mathsf{info}_{\tau_{\mathrm{new}}}$ should be authenticated by the GM and should contain information about users who are active during epoch τ_{new}.

$\mathsf{Enc}(\tau, w, \mathsf{id}, \mathsf{cert}_{\mathsf{id}})$ Given a time epoch τ, message w, a receiver's identifier id and its group membership certificate $\mathsf{cert}_{\mathsf{id}}$, this probabilistic algorithm outputs a ciphertext ψ.

$\mathsf{Verify}(\tau, \psi)$ This algorithm, run by any verifier, returns 1 or 0, indicating the validity or invalidity of the ciphertext ψ.

$\mathsf{Dec}(\mathsf{sk}_{\mathsf{id}}, \tau, \psi)$ This deterministic algorithm is run by receiver id, who uses its secret key $\mathsf{sk}_{\mathsf{id}}$ to decrypt ciphertext ψ and obtain $w' \in \mathcal{W}$.

$\mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$ This deterministic algorithm is run by the OA who possesses the key $\mathsf{sk}_{\mathsf{OA}}$. It returns a user identifier $\mathsf{id}' \in \mathcal{ID} \cup \{\perp\}$.

We also introduce the following algorithm, which will be used only in the security experiments.

$\mathsf{IsActive}(\tau, \mathsf{id})$ This algorithm returns 1 if user id is active at time epoch τ, i.e., id has been enrolled and has not been revoked, and 0 otherwise.

Remark 1. To simplify the presentation, we assume that, given pp and a time epoch τ, the descriptions of the corresponding group information info_τ, the corresponding filtering policy F_τ and the corresponding tracing policy P_τ are completely determined.

We require that a GEOT scheme must satisfy correctness, privacy (which subsumes message secrecy and receiver anonymity) and unforgeability.

Correctness. Roughly speaking, correctness requires that at any time epoch, an honestly-generated ciphertext from an active user should be successfully verified, decrypted, and opened relatively to the system policies at that epoch. More formally, for any ciphertext $\psi \leftarrow \mathsf{Enc}(\tau, w, \mathsf{id}, \mathsf{cert}_{\mathsf{id}})$, where id is active at epoch τ and $F_\tau(w) = 1$, the following conditions must hold: (i) ψ is deemed valid by the verification algorithm; (ii) decrypting ψ using secret key $\mathsf{sk}_{\mathsf{id}}$ yields w; (iii) the opening algorithm returns $\perp$ if $P_\tau(\mathsf{id}, w) = 1$, i.e., ψ is non-traceable; and (iv) the opening algorithm returns id if $P_\tau(\mathsf{id}, w) = 0$, i.e., ψ is traceable. These requirements are formally modeled in experiment $\mathbf{Exp}^{\mathsf{correct}}_{\mathcal{A}}(1^\lambda)$, which involves the following oracles.

$\mathsf{AddU}(\mathsf{sk}_{\mathsf{GM}})$ This oracle allows the adversary to add an honest user to the group at the current epoch. Upon request, it simulates an honest execution of the interactive protocol $\langle \mathsf{Join}(\mathsf{info}), \mathsf{Issue}(\mathsf{sk}_{\mathsf{GM}}, \mathsf{info}, \mathbf{reg}) \rangle$ (without the adversary's involvement). It maintains an honest-user list HUL and adds the obtained user identifier id to HUL.

$\mathsf{GUp}(\cdot)$ This oracle allows the adversary to update the group. When invoked with a set $\mathcal{S}$, it executes algorithm $\mathsf{GUpdate}(\mathsf{sk}_{\mathsf{GM}}, \mathcal{S}, \mathsf{info}, \mathbf{reg})$.

Definition 1. *Let* $\mathbf{Adv}^{\mathsf{correct}}_{\mathcal{A}}(1^\lambda) = \mathsf{Pr}[\mathbf{Exp}^{\mathsf{correct}}_{\mathcal{A}}(1^\lambda) = 1]$ *be the advantage of adversary* $\mathcal{A}$ *in experiment* $\mathbf{Exp}^{\mathsf{correct}}_{\mathcal{A}}(1^\lambda)$. *A* GEOT *scheme is said to be correct if the advantage of any PPT adversary* $\mathcal{A}$ *is negligible in* λ.

Experiment $\mathbf{Exp}^{\mathsf{correct}}_{\mathcal{A}}(1^\lambda)$
$(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}}) \leftarrow \mathsf{Setup}(1^\lambda)$; $\mathsf{HUL} \leftarrow \emptyset$.
$(\tau, w, \mathsf{id}) \leftarrow \mathcal{A}^{\mathsf{AddU}, \mathsf{GUp}}(\mathsf{pp})$.
If $\mathsf{id} \notin \mathsf{HUL}$ or $\mathsf{IsActive}(\tau, \mathsf{id}) = 0$ or $F_\tau(w) = 0$, return 0.

$\psi \leftarrow \mathsf{Enc}(\tau, w, \mathsf{id}, \mathsf{cert}_{\mathsf{id}})$. If $\mathsf{Verify}(\tau, \psi) = 0$, return 1.
$w' \leftarrow \mathsf{Dec}(\mathsf{sk}_{\mathsf{id}}, \tau, \psi)$. If $w' \neq w$, return 1.
$\mathsf{id}' \leftarrow \mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$.
If $(P_\tau(\mathsf{id}, w) = 1$ and $\mathsf{id}' \neq \perp)$ or $(P_\tau(\mathsf{id}, w) = 0$ and $\mathsf{id}' \neq \mathsf{id})$, return 1.
Return 0.

We remark that the adversary $\mathcal{A}$ wins (i.e., the experiment returns 1) whenever it finds an epoch τ, a message w, an active, honest identifier id so that a ciphertext encrypting w at epoch τ and from user id either: (i) fails to verify, (ii) is decrypted to some other $w' \neq w$, or (iii) has an opening inconsistent with what P_τ indicates.

2.2 Privacy of GEOT

A private GEOT scheme should satisfy message secrecy, type-1-anonymity and type-2-anonymity. These notions are formalized via the following oracles.

$\mathsf{USER}()$ This oracle enrolls an honest user into the group. It simulates the Join algorithm and interacts with an adversary playing the role of the GM. If the interaction ends successfully, the oracle adds id to the list HUL.

$\mathsf{RevealU}(\cdot)$ On input an identifier id, this oracle aborts if $\mathsf{id} \notin \mathsf{HUL}$. Otherwise, it returns the secret key $\mathsf{sk}_{\mathsf{id}}$ and adds id to a corrupted user list CUL.

$\mathsf{DEC}(\mathsf{sk}_{\mathsf{id}}, \cdot)$ This oracle permits the adversary to query the decryption of a ciphertext using the secret key of user id. When queried with a pair (τ, ψ), the oracle computes $w' \leftarrow \mathsf{Dec}(\mathsf{sk}_{\mathsf{id}}, \tau, \psi)$ and returns w' to the adversary. We write $\mathsf{DEC}^{\neg(\tau,\psi)}(\mathsf{sk}_{\mathsf{id}}, \cdot)$ if the pair (τ, ψ) is prohibited from this oracle query.

$\mathsf{OPEN}(\cdot)$ This oracle allows the adversary to learn the recipient of a ciphertext. When (τ, ψ) is queried, it computes $\mathsf{id}' \leftarrow \mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$ and returns id' to the adversary. We also write $\mathsf{OPEN}^{\neg(\tau,\psi)}(\cdot)$ if the pair (τ, ψ) is precluded.

Message Secrecy. This notion protects the receiver from an adversary who tries to learn information about the underlying message. It captures the inability of an adversary to tell whether a ciphertext encrypts message w_0 or w_1, in the IND-CCA2 sense. The adversary is allowed to corrupt the GM and the OA. It can also adaptively query the decryption oracle. The formal definition is given below.

Definition 2. *A* GEOT *scheme is said to satisfy message secrecy if for any PPT adversary $\mathcal{A}$, we have* $\mathbf{Adv}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda) = |\mathsf{Pr}[\mathbf{Exp}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2}| \leq \mathsf{negl}(\lambda)$.

Experiment $\mathbf{Exp}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda)$
$(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}}) \leftarrow \mathsf{Setup}(1^\lambda)$; $\mathsf{HUL} \leftarrow \emptyset, \mathsf{CUL} \leftarrow \emptyset$.
$(\tau, w_0, w_1, \mathsf{id}, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{USER},\mathsf{RevealU},\mathsf{DEC}}(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}})$.
If $(\mathsf{id} \notin \mathsf{HUL} \setminus \mathsf{CUL})$ or $(\mathsf{IsActive}(\tau, \mathsf{id}) = 0)$, return 0.
If $(F_\tau(w_0) = 0)$ or $(F_\tau(w_1) = 0)$ or $(P_\tau(\mathsf{id}, w_0) \neq P_\tau(\mathsf{id}, w_1))$, return 0.
$b \xleftarrow{\$} \{0, 1\}$.

$\psi \leftarrow \mathsf{Enc}(\tau, w_b, \mathsf{id}, \mathsf{cert}_{\mathsf{id}})$.
$b' \leftarrow \mathcal{A}^{\mathsf{USER},\mathsf{RevealU},\mathsf{DEC}^{\neg(\tau,\psi)}(\mathsf{sk}_{\mathsf{id}},\cdot)}(\mathsf{aux}, \psi)$. If $b' = b$, return 1. Else return 0.

In experiment $\mathbf{Exp}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda)$, the adversary $\mathcal{A}$ fully controls both the GM and the OA. The adversary interacts with honest users via the oracle USER and is allowed to learn secret keys of all but one honest user via the oracle $\mathsf{RevealU}$. It is also allowed to make decryption queries w.r.t. decryption key $\mathsf{sk}_{\mathsf{id}'}$, for any $\mathsf{id}' \in \mathsf{HUL} \backslash \mathsf{CUL}$. At the end of the first stage, $\mathcal{A}$ outputs a tuple $(\tau, w_0, w_1, \mathsf{id})$ satisfying the conditions specified in the experiment. Afterwards, a challenge ciphertext ψ is computed based on a random bit b and returned to $\mathcal{A}$. Upon receiving ψ, $\mathcal{A}$ continues to have access to the decryption oracle $\mathsf{DEC}(\mathsf{sk}_{\mathsf{id}}, \cdot)$, with the exception that it cannot trivially ask to decrypt ψ w.r.t. τ. Finally, $\mathcal{A}$ outputs a guess bit b' and the experiment returns 1 iff $b' = b$.

We note that the tuple $(\tau, w_0, w_1, \mathsf{id})$ outputted by $\mathcal{A}$ at the end of the first stage must satisfy $F_\tau(w_0) = F_\tau(w_1) = 1$, in order to ensure that the challenge ciphertext ψ passes the filter. We also stress that $P_\tau(\mathsf{id}, w_0) = P_\tau(\mathsf{id}, w_1)$ is a compulsory condition. Otherwise, $\mathcal{A}$ can open ψ using $\mathsf{sk}_{\mathsf{OA}}$ to tell whether ψ is traceable or non-traceable and thus can easily determine which message is encrypted. This is a notable difference compared to the formulation of message secrecy in FDGE [59], for which all the valid ciphertexts are traceable.

Type-1-Anonymity. This notion is orthogonal to message secrecy, and protects the identity of the intended receiver of a ciphertext from being extracted by a malicious adversary. It demands that the adversary $\mathcal{A}$ cannot distinguish whether a ciphertext is intended for user id_0 or user id_1 even if $\mathcal{A}$ can fully corrupt the GM. Although corruption of the OA is not permitted, $\mathcal{A}$ is allowed to adaptively query the OPEN oracle. We model this requirement in experiment $\mathbf{Exp}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda)$ and give a formal definition below.

Definition 3. *Let* $\mathbf{Adv}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda) = |\mathsf{Pr}[\mathbf{Exp}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2}|$. *A* GEOT *scheme is called type-*1*-anonymous if for any PPT* $\mathcal{A}$*, we have* $\mathbf{Adv}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda) \leq \mathsf{negl}(\lambda)$.

Experiment $\mathbf{Exp}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda)$
$(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}}) \leftarrow \mathsf{Setup}(1^\lambda)$; $\mathsf{HUL} \leftarrow \emptyset, \mathsf{CUL} \leftarrow \emptyset$.
$(\tau, w, \mathsf{id}_0, \mathsf{id}_1, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{USER},\mathsf{RevealU},\mathsf{OPEN}}(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}})$.
If $\mathsf{IsActive}(\tau, \mathsf{id}_0) = 0$ or $\mathsf{IsActive}(\tau, \mathsf{id}_1) = 0$, return 0.
If $\mathsf{id}_0 \notin \mathsf{HUL} \setminus \mathsf{CUL}$ or $\mathsf{id}_1 \notin \mathsf{HUL} \setminus \mathsf{CUL}$, return 0.
If $F_\tau(w) = 0$, return 0.
$b \xleftarrow{\$} \{0, 1\}$.
$\psi \leftarrow \mathsf{Enc}(\tau, w, \mathsf{id}_b, \mathsf{cert}_{\mathsf{id}_b})$.
$b' \leftarrow \mathcal{A}^{\mathsf{USER},\mathsf{RevealU},\mathsf{OPEN}^{\neg(\tau,\psi)},\mathsf{DEC}^{\neg(\tau,\psi)}(\mathsf{sk}_{\mathsf{id}_0},\cdot),\mathsf{DEC}^{\neg(\tau,\psi)}(\mathsf{sk}_{\mathsf{id}_1},\cdot)}(\mathsf{aux}, \psi)$.
If $b' = b$, return 1. Else return 0.

In experiment $\mathbf{Exp}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda)$, the adversary $\mathcal{A}$ fully corrupts the GM and interacts with honest users and learns their secret keys via the oracles USER and $\mathsf{RevealU}$, respectively. The OA remains honest, yet $\mathcal{A}$ is given CCA2 access to the opening oracle. Similar to the experiment $\mathbf{Exp}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda)$, $\mathcal{A}$ outputs a challenge tuple satisfying certain conditions and receives back a challenge ciphertext ψ

based on a random bit b. The adversary is asked to determine which one of the challenge users is the intended receiver of ψ.

We remark that the adversary is allowed to choose the challenge tuple such that $P_\tau(\mathsf{id}_0, w) \neq P_\tau(\mathsf{id}_1, w)$. Thus, the definition of type-1-anonymity implies the incapability of $\mathcal{A}$ to tell apart ciphertexts that are traceable and those that are non-traceable (similar to the branch-hiding property in BiAS [47]).

It is also worth mentioning that the two challenge users must not be corrupted by $\mathcal{A}$. In fact, by the correctness of GEOT, algorithm $\mathsf{Dec}(\mathsf{sk}_{\mathsf{id}_d}, \cdot)$ can recover the underlying message w if $d = b$. Therefore, corrupting any of the challenge users would enable $\mathcal{A}$ to trivially break anonymity.

Type-2-Anonymity. This is a strengthened anonymity notion that GEOT can achieve in the non-traceable case. Here, we further allow the adversary to fully corrupt the OA if $P(\mathsf{id}_0, w) = P(\mathsf{id}_1, w) = 1$, i.e., the to-be-computed challenge ciphertext is always non-traceable. This notion requires the infeasibility of $\mathcal{A}$ possessing the opening key $\mathsf{sk}_{\mathsf{OA}}$ to learn any information about the ciphertext receiver. Type-2-anonymity is formally defined as follows.

Definition 4. *Define* $\mathbf{Adv}^{\mathsf{anony-ntr}}_{\mathcal{A}}(1^\lambda) = |\mathsf{Pr}[\mathbf{Exp}^{\mathsf{anony-ntr}}_{\mathcal{A}}(1^\lambda) = 1] - \frac{1}{2}|$. *A* GEOT *scheme is called type-2-anonymous if* $\mathbf{Adv}^{\mathsf{anony-ntr}}_{\mathcal{A}}(1^\lambda)$ *is negligible in* λ *for any PPT adversary* $\mathcal{A}$.

Experiment $\mathbf{Exp}^{\mathsf{anony-ntr}}_{\mathcal{A}}(1^\lambda)$
$(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}}) \leftarrow \mathsf{Setup}(1^\lambda)$; $\mathsf{HUL} \leftarrow \emptyset, \mathsf{CUL} \leftarrow \emptyset$.
$(\tau, w, \mathsf{id}_0, \mathsf{id}_1, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{USER},\mathsf{RevealU}}(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}})$.
If $\mathsf{IsActive}(\tau, \mathsf{id}_0) = 0$ or $\mathsf{IsActive}(\tau, \mathsf{id}_1) = 0$, return 0.
If $\mathsf{id}_0 \notin \mathsf{HUL} \setminus \mathsf{CUL}$ or $\mathsf{id}_1 \notin \mathsf{HUL} \setminus \mathsf{CUL}$, return 0.
If $F_\tau(w) = 0$, return 0. If $\neg(P_\tau(\mathsf{id}_0, w) = P_\tau(\mathsf{id}_1, w) = 1)$, return 0.
$b \xleftarrow{\$} \{0, 1\}$.
$\psi \leftarrow \mathsf{Enc}(\tau, w, \mathsf{id}_b, \mathsf{cert}_{\mathsf{id}_b})$.
$b' \leftarrow \mathcal{A}^{\mathsf{USER},\mathsf{RevealU},\mathsf{DEC}^{\neg(\tau,\psi)}(\mathsf{sk}_{\mathsf{id}_0},\cdot),\mathsf{DEC}^{\neg(\tau,\psi)}(\mathsf{sk}_{\mathsf{id}_1},\cdot)}(\mathsf{aux}, \psi)$.
If $b' = b$, return 1. Else return 0.

2.3 Unforgeability of GEOT

Unforgeability of a GEOT system aims to prevent the system from falsely accepting inappropriate ciphertexts. It captures the infeasibility of an adversary to generate a verifiable pair (τ, ψ) such that either (i) the recipient id of ψ is not active at time epoch τ; or (ii) the encrypted message w does not satisfy the filtering policy F_τ; or (iii) the outcome of the opening algorithm is not consistent with the tracing policy P_τ, e.g., it points to a receiver id despite $P_\tau(\mathsf{id}, w) = 1$ (i.e., non-traceable case), or it declares failure despite $P_\tau(\mathsf{id}, w) = 0$ (i.e., traceable case). Formalizing this notion turns out to be a non-trivial task.

Recall that in the related contexts of traditional group signatures [12,13,16] and group encryption [42,59], a violation of traceability/non-frameability can easily be detected, because the result of the opening algorithm in those systems can help determine if an adversarially produced signature/ciphertext corresponds to an honest or corrupted user. Here, in contrast, the opening algorithm

of GEOT does not provide sufficient information for deciding if a ciphertext (τ, ψ) forms a forgery. If Open returns $\perp$, it is not guaranteed that the ciphertext in question is supposed to be non-traceable. Even when Open returns an identifier id, we also lack information about w (due to message secrecy) to be certain that $P_\tau(\mathsf{id}, w) = 0$ and that user id is not framed.

To overcome the aforementioned definitional challenge, we adapt the approach of [47], by employing two auxiliary algorithms SimSetup and Extract, which allow us to extract meaningful information (such as identify id' and message w') from an accepted ciphertext and to determine if a forgery has occurred.

$\mathsf{SimSetup}(1^\lambda)$ Given the security parameter 1^λ, this algorithm generates simulated $(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}})$ together with an extraction trapdoor τ_{ext}.

$\mathsf{Extract}\big(\tau_{\mathsf{ext}}, (\mathsf{pp}, \tau, \psi)\big)$ Given the extraction trapdoor τ_{ext}, a valid ciphertext ψ at time τ, this extraction algorithm returns a pair $(\mathsf{id}', w') \in \mathcal{ID} \times \mathcal{W}$.

It is natural to require the indistinguishability of the outputs of Setup and SimSetup. In addition, to capture the fact that Extract outputs meaningful information, we require that the extraction result does not contradict the opening result. Formally, we define extractability as follows.

Definition 5. *A* GEOT *scheme with algorithms* (SimSetup, Extract) *is said to be extractable if the following two conditions are conformed.*

- *To any PPT adversary $\mathcal{A}$, the outputs* $(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}})$ *from* SimSetup *and* Setup *are indistinguishable.*
- *For any PPT adversary $\mathcal{A}$ involved in experiment* $\mathbf{Exp}^{\mathsf{extract}}_{\mathcal{A}}(1^\lambda)$*, its advantage* $\mathbf{Adv}^{\mathsf{extract}}_{\mathcal{A}}(1^\lambda) = \mathsf{Pr}[\mathbf{Exp}^{\mathsf{extract}}_{\mathcal{A}}(1^\lambda) = 1]$ *is negligible in λ.*

Experiment $\mathbf{Exp}^{\mathsf{extract}}_{\mathcal{A}}(1^\lambda)$
$(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}}; \tau_{\mathsf{ext}}) \leftarrow \mathsf{SimSetup}(1^\lambda)$.
$(\tau, \psi) \leftarrow \mathcal{A}(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}})$.
If $\mathsf{info}_\tau = \perp$ or $\mathsf{Verify}(\tau, \psi) = 0$, return 0.
$(\mathsf{id}', w') \leftarrow \mathsf{Extract}(\tau_{\mathsf{ext}}, \mathsf{pp}, \tau, \psi)$.
$F_\tau(w') = 0$, return 1.
$\mathsf{id}^* \leftarrow \mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$.
If $(P_\tau(\mathsf{id}', w') = 1) \wedge (\mathsf{id}^* \neq \perp)$, return 1.
If $(P_\tau(\mathsf{id}', w') = 0) \wedge (\mathsf{id}^* \neq \mathsf{id}')$, return 1.
Else return 0.

Let us now define unforgeability of a GEOT system. We consider experiment $\mathbf{Exp}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda)$, which involves the following oracle.

$\mathsf{REG}()$ This oracle allows the adversary to introduce users to the group at the current epoch. On behalf of the GM, the oracle acts as Issue to interact with the adversary who may or may not follow the program of Join.

Definition 6. *Let* $\mathbf{Adv}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda) = \mathsf{Pr}[\mathbf{Exp}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda) = 1]$ *be the advantage of an adversary $\mathcal{A}$ against unforgeability. A* GEOT *scheme is unforgeable if it is extractable and the advantage of any PPT adversary $\mathcal{A}$ in experiment* $\mathbf{Exp}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda)$ *is negligible in λ.*

Experiment $\mathbf{Exp}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda)$
$(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}}, \tau_{\mathsf{ext}}) \leftarrow \mathsf{SimSetup}(1^\lambda)$.
$(\tau, \psi) \leftarrow \mathcal{A}^{\mathsf{REG},\mathsf{GUp}}(\mathsf{pp}, \mathsf{sk}_{\mathsf{OA}})$.
If $\mathsf{info}_\tau = \perp$ or $\mathsf{Verify}\big(\tau, \psi\big) = 0$, return 0.
$(\mathsf{id}', w') \leftarrow \mathsf{Extract}(\tau_{\mathsf{ext}}, \mathsf{pp}, \tau, \psi)$.
$\mathsf{id}^* \leftarrow \mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$.
If $(\mathsf{IsActive}(\tau, \mathsf{id}') = 0)$ or $F_\tau(w') = 0$, return 1.
If $\big(P_\tau(\mathsf{id}', w') = 1\big) \wedge (\mathsf{id}^* \neq \perp)$, return 1.
If $\big(P_\tau(\mathsf{id}', w') = 0\big) \wedge (\mathsf{id}^* \neq \mathsf{id}')$, return 1.
Return 0.

In experiment $\mathbf{Exp}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda)$, adversary $\mathcal{A}$ is given simulated pp and $\mathsf{sk}_{\mathsf{OA}}$, and cannot tell if it is in an extractable mode or a real mode due to extractability. Via the oracles REG and GUp, $\mathcal{A}$ can enroll malicious users into the group and remove some users from the group, respectively. At the end of the experiment, $\mathcal{A}$ outputs (τ, ψ). The experiment then extracts (id', w') from the ciphertext using τ_{ext}. If either one of the aforementioned three cases occurs, this experiment returns 1.

3 A Generic Construction of GEOT

This section presents a generic construction of GEOT for arbitrary filtering and tracing policies. The construction satisfies the rigorous requirements in Section 2. We first provide a high-level overview of the construction in Section 3.1. The detailed description is then presented in Section 3.2 and the scheme analyses are given in Section 3.3. Our construction can be viewed as a feasibility result for designing GEOT based on well-studied computational assumptions and in a modular manner. In particular, it can be instantiated in the standard model from pairings and lattices, via the techniques for obtaining NIZKs for NP by Groth-Ostrovsky-Sahai [37] and by Peikert-Shiehian [65], respectively. It can also be efficiently realized in the random oracle model based on the lattice-based techniques from [40,73,17]. The code-based instantiation we will present in Section 4 is inspired by but slightly departs from this generic construction.

3.1 Technical Overview

The construction employs the following cryptographic building blocks, some background reminders of which are given in Appendix A.

- A signature scheme $\mathcal{SIG} = (\mathsf{S.Kg}, \mathsf{S.Sign}, \mathsf{S.Ver})$ that is existentially unforgeable under chosen message attacks (EUF-CMA);
- Two public-key encryption schemes $\mathcal{E}_{\mathcal{R}} = (\mathsf{E_R.Kg}, \mathsf{E_R.Enc}, \mathsf{E_R.Dec})$ and $\mathcal{E}_{\mathcal{OA}} = (\mathsf{E_{OA}.Kg}, \mathsf{E_{OA}.Enc}, \mathsf{E_{OA}.Dec})$, that satisfy IND-CCA2 security. We additionally require IK-CCA2 security (i.e., key privacy) for $\mathcal{E}_{\mathcal{R}}$;
- A dual-mode $\mathcal{NIZK}$ argument system $\mathcal{NIZK} = (\mathsf{ZK.Setup}, \mathsf{ZK.ExtSetup}, \mathsf{ZK.Prove}, \mathsf{ZK.Ver}, \mathsf{ZK.Sim}, \mathsf{ZK.Extr})$ for the NP-relation $\mathcal{R}$ defined below.

The major ideas underlying the construction are as follows. Each potential user of the system is equipped with a decryption-encryption key pair $(\mathsf{sk}_{\mathsf{id}}, ek = \mathsf{id})$ for $\mathcal{E}_{\mathcal{R}}$. The GM possesses a signing-verification key pair $(\mathsf{sk}_{\mathsf{GM}}, \mathsf{vk}_{\mathsf{GM}})$ for $\mathcal{SIG}$, which will be used for certifying users' public keys $ek = \mathsf{id}$. The OA, on the other hand, owns a decryption-encryption key pair $(\mathsf{sk}_{\mathsf{OA}}, \mathsf{ek}_{\mathsf{OA}})$ for $\mathcal{E}_{\mathcal{OA}}$. To send a plaintext w to user id at time τ, the sender first makes sure that: (i) w is allowed under the filtering policy F_τ; (ii) id has been certified by the GM, and is not in the revocation list $\mathcal{S}$. Then, the sender generates two sub-ciphertexts:

- Ciphertext $\mathbf{c}_{\mathsf{R}}$ that encrypts w under encryption key id;
- Ciphertext $\mathbf{c}_{\mathsf{OA}}$ that encrypts the value $(1-P_\tau(\mathsf{id}, w))\cdot\mathsf{id}$ under the encryption key $\mathsf{ek}_{\mathsf{OA}}$, where $P_\tau(\cdot,\cdot)$ is the tracing predicate at time τ.

Then the sender generates a NIZK argument for the relation $\mathcal{R}$, as defined in (1), to prove in ZK that all the steps have been done correctly.

$$\begin{aligned}\mathcal{R} := \Big\{ \ \big(\ & x = (\mathsf{vk}_{\mathsf{GM}}, \mathsf{ek}_{\mathsf{OA}}, \mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \mathcal{S}, F_\tau, P_\tau), \xi = (w, \mathsf{id} = ek, \mathsf{cert}_{\mathsf{id}}, s_{\mathsf{R}}, s_{\mathsf{OA}})\big) : \\ & \big(\mathsf{S.Ver}(\mathsf{vk}_{\mathsf{GM}}, \mathsf{id}, \mathsf{cert}_{\mathsf{id}}) = 1\big) \wedge \big(\mathsf{id} \notin \mathcal{S}\big) \wedge \big(F_\tau(w) = 1\big) \\ & \wedge \ \Big(\mathbf{c}_{\mathsf{R}} = \mathsf{E}_{\mathsf{R}}.\mathsf{Enc}(ek, w, s_{\mathsf{R}})\Big) \\ & \wedge \ \Big(\mathbf{c}_{\mathsf{OA}} = \mathsf{E}_{\mathsf{OA}}.\mathsf{Enc}(\mathsf{ek}_{\mathsf{OA}}, (1 - P_\tau(\mathsf{id}, w)) \cdot \mathsf{id}, s_{\mathsf{OA}})\Big) \ \Big\}. \end{aligned} \tag{1}$$

The ciphertext ψ then is set as the triple $(\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$. Verification of ψ consists of verifying π. The receiver id can recover w by decrypting $\mathbf{c}_{\mathsf{R}}$. Meanwhile, depending on the predicate value $P_\tau(\mathsf{id}, w)$, the OA can learn from $\mathbf{c}_{\mathsf{OA}}$ either id (traceable case) or $\mathbf{0}$ (non-traceable case).

At a high level, the correctness of the obtained GEOT scheme is guaranteed by the correctness/completeness of the $\mathcal{SIG}$, $\mathcal{E}_{\mathcal{R}}$, $\mathcal{E}_{\mathcal{OA}}$ and $\mathcal{NIZK}$. The security of the scheme also relies on the security properties of these building blocks. In particular, we require key privacy for $\mathcal{E}_{\mathcal{R}}$ (but not for $\mathcal{E}_{\mathcal{OA}}$) to ensure that the ciphertext $\mathbf{c}_{\mathsf{R}}$ does not leak any information about the receiver's public key.

3.2 Description of our Generic Construction

Let $\lambda \in \mathbb{N}$ be a security parameter. Our generic construction of a GEOT system associated with $(\mathcal{T},\ \mathcal{ID},\ \mathcal{W},\ \mathcal{F},\ \mathcal{P})$ works as follows.

$\mathsf{Setup}(1^\lambda)$ This algorithm performs the following steps:

1. Run $\mathsf{S.Kg}(1^\lambda)$ to obtain a signing-verification key-pair $(\mathsf{sk}_{\mathsf{GM}}, \mathsf{vk}_{\mathsf{GM}})$.
2. Run $\mathsf{E}_{\mathsf{OA}}.\mathsf{Kg}(1^\lambda)$ to obtain a decryption-encryption key pair $(\mathsf{sk}_{\mathsf{OA}}, \mathsf{ek}_{\mathsf{OA}})$.
3. Run $\mathsf{ZK.Setup}(1^\lambda)$ to obtain a common reference string crs (and a simulation trapdoor τ_{sim} - which is discarded) for the NIZK system.

Return $\mathsf{pp} := (\mathsf{crs}, \mathsf{vk}_{\mathsf{GM}}, \mathsf{ek}_{\mathsf{OA}})$, as well as $\mathsf{sk}_{\mathsf{GM}}$ and $\mathsf{sk}_{\mathsf{OA}}$. The algorithm also initializes $\mathbf{reg} := \emptyset$ and $\mathsf{info} := \emptyset$.

$\langle \mathsf{Join}(\mathsf{info}), \mathsf{Issue}(\mathsf{sk}_{\mathsf{GM}}, \mathsf{info}, \mathbf{reg})\rangle$ A user who would like to join GEOT system interacts with the GM as follows.

1. The user runs $\mathsf{E_R.Kg}(1^\lambda)$ to obtain a decryption-encryption key pair (sk, ek). Define $\mathsf{id} := ek$ and send id to the GM.
2. The GM aborts if id has already been registered. Otherwise, it certifies id using $\mathsf{sk_{GM}}$ as $\mathsf{cert_{id}} \leftarrow \mathsf{S.Sign}(\mathsf{sk_{GM}}, \mathsf{id})$ and sends $\mathsf{cert_{id}}$ to the user.
3. Upon receiving $\mathsf{cert_{id}}$, the user aborts if $\mathsf{S.Ver}(\mathsf{vk_{GM}}, \mathsf{id}, \mathsf{cert_{id}}) = 0$. Otherwise, it stores $\mathsf{sk_{id}} := (sk, \mathsf{id}, \mathsf{cert_{id}})$.
4. The GM appends the registration information $(\mathsf{id}, \mathsf{cert_{id}})$ to the table $\mathbf{reg}$.

$\mathsf{GUpdate}(\mathsf{sk_{GM}}, \mathcal{S}, \mathsf{info}, \mathbf{reg})$ The GM updates the group information and advances the time period as follows.

1. Certify the set $\mathcal{S} = \{\mathsf{id}_i\}_i$ of to-be-revoked users, together with the current and new time periods as $cert \leftarrow \mathsf{S.Sign}(\mathsf{sk_{GM}}, (\mathcal{S}, \tau_{\text{current}}, \tau_{\text{new}}))$.
2. Let $\mathsf{info}_{\tau_{\text{new}}} = (\tau_{\text{new}}, \mathcal{S}, cert)$ and update the information table with $\mathsf{info} := \mathsf{info} \cup \{\mathsf{info}_{\tau_{\text{new}}}\}$.

$\mathsf{Enc}(\tau, w, \mathsf{id}, \mathsf{cert_{id}})$ This algorithm performs the following steps.

1. If $F_\tau(w) = 0$, i.e., message w does not satisfy the filtering policy F_τ, then return $\psi = \perp$.
2. If the receiver id is currently revoked, i.e., $\mathsf{id} \in \mathcal{S}$, where $\mathcal{S}$ is fetched from info_τ, then return $\psi = \perp$.
3. Encrypt the plaintext w under the receiver's public key $ek = \mathsf{id}$, as $\mathbf{c_R} = \mathsf{E_R.Enc}(ek, w, s_\mathsf{R})$, where s_R is the encryption randomness.
4. Encrypt $(1 - P_\tau(\mathsf{id}, w)) \cdot \mathsf{id}$ and under the OA's public key $\mathsf{ek_{OA}}$ as $\mathbf{c_{OA}} = \mathsf{E_{OA}.Enc}(\mathsf{ek_{OA}}, (1-P_\tau(\mathsf{id}, w)) \cdot \mathsf{id}, s_\mathsf{OA})$, where s_OA is the encryption randomness.
5. Generate a non-interactive zero-knowledge argument π to prove knowledge of witness $\xi = (w, \mathsf{id}, \mathsf{cert_{id}}, s_\mathsf{R}, s_\mathsf{OA})$ such that:
   - id was properly certified by the GM via $\mathsf{cert_{id}}$;
   - id is not currently revoked, i.e., $\mathsf{id} \notin \mathcal{S}$;
   - w satisfies the filtering policy, i.e., $F_\tau(w) = 1$;
   - $\mathbf{c_R}$ and $\mathbf{c_{OA}}$ are well-formed ciphertexts of w and $(1 - P_\tau(\mathsf{id}, w)) \cdot \mathsf{id}$, as described above.

   Let $x = (\mathsf{vk_{GM}}, \mathsf{ek_{OA}}, \mathbf{c_R}, \mathbf{c_{OA}}, \mathcal{S}, F_\tau, P_\tau)$, then we have $(x, \xi) \in \mathcal{R}$, where $\mathcal{R}$ is as defined in (1). Then, the desired NIZK argument π is generated as $\pi \leftarrow \mathsf{ZK.Prove}(\mathsf{crs}, x, \xi)$.
6. Let $\psi := (\mathbf{c_R}, \mathbf{c_{OA}}, \pi)$ and return ψ.

$\mathsf{Verify}(\tau, \psi)$ This algorithm essentially consists of verifying π. It parses $\psi = (\mathbf{c_R}, \mathbf{c_{OA}}, \pi)$, then return $b' \leftarrow \mathsf{ZK.Ver}(\mathsf{crs}, (\mathsf{vk_{GM}}, \mathsf{ek_{OA}}, \mathbf{c_R}, \mathbf{c_{OA}}, \mathcal{S}, F_\tau, P_\tau), \pi)$.

$\mathsf{Dec}(\mathsf{sk_{id}}, \tau, \psi)$ This algorithm essentially consists of decrypting the ciphertext component $\mathbf{c_R}$. It parses $\psi = (\mathbf{c_R}, \mathbf{c_{OA}}, \pi)$, then return $w' \leftarrow \mathsf{E_R.Dec}(\mathsf{sk_{id}}, \mathbf{c_R})$.

$\mathsf{Open}(\mathsf{sk_{OA}}, \tau, \psi)$ This algorithm basically consists of decrypting $\mathbf{c_{OA}}$. It parses $\psi = (\mathbf{c_R}, \mathbf{c_{OA}}, \pi)$, then compute $\mathbf{d} \leftarrow \mathsf{E_{OA}.Dec}(\mathsf{sk_{OA}}, \mathbf{c_{OA}})$. If $\mathbf{d} = \mathbf{0}$ or $\mathbf{d} \notin \mathcal{ID}$, then return $\perp$. Otherwise, return $\mathsf{id}' = \mathbf{d}$.

Auxiliary algorithms. We now describe the two supplementary algorithms $\mathsf{SimSetup}$ and $\mathsf{Extract}$ that are critical for analyzing the unforgeability of the proposed GEOT system.

$\mathsf{SimSetup}(1^\lambda)$ This algorithm is almost the same as the real Setup described above. The only difference is that, instead of running $\mathsf{ZK.Setup}(1^\lambda)$, it runs $\mathsf{ZK.ExtSetup}(1^\lambda)$ to obtain a crs associated with an extraction trapdoor τ_{ext}.

$\mathsf{Extract}\big(\tau_{\mathsf{ext}}, (\mathsf{pp}, \tau, \psi)\big)$ Given the extraction trapdoor τ_{ext} and a valid ciphertext $\psi = (\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$ at time τ, define $x = (\mathsf{vk}_{\mathsf{GM}}, \mathsf{ek}_{\mathsf{OA}}, \mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \mathcal{S}, F_\tau, P_\tau)$. This algorithm computes $\xi' \leftarrow \mathsf{ZK.Extract}(\mathsf{crs}, \tau_{\mathsf{ext}}, x, \pi)$ of the form $\xi' = (w', \mathsf{id}', \mathsf{cert}'_{\mathsf{id}}, s'_{\mathsf{R}}, s'_{\mathsf{OA}})$. It then returns (id', w').

3.3 Correctness and Security Analyses

Correctness. If the employed building blocks $\mathcal{SIG}, \mathcal{E}_{\mathcal{R}}, \mathcal{E}_{\mathcal{OA}}$ are correct and $\mathcal{NIZK}$ is complete, then the proposed GEOT scheme satisfies correctness as defined in Definition 1.

First of all, during the $\langle \mathsf{Join}(\mathsf{info}), \mathsf{Issue}(\mathsf{sk}_{\mathsf{GM}}, \mathsf{info}, \mathbf{reg}) \rangle$ protocol, for a newly registered user id, one has $\mathsf{S.Ver}(\mathsf{vk}_{\mathsf{GM}}, \mathsf{id}, \mathsf{cert}_{\mathsf{id}}) = 1$, thanks to the correctness of $\mathcal{SIG}$. As a result, the protocol is always successfully executed by honest parties. Next, during the encryption process, as long as $F_\tau(w) = 1$ and $\mathsf{id} \notin \mathcal{S}$, an honest ciphertext sender should be able to obtain a satisfying witness $\xi = (w, \mathsf{id}, \mathsf{cert}_{\mathsf{id}}, s_{\mathsf{R}}, s_{\mathsf{OA}})$ for the relation $\mathcal{R}$, and hence, should be able to generate a NIZK argument π. As a result, Enc should successfully output a ciphertext $\psi = (\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$. Now, thanks to the completeness of $\mathcal{NIZK}$, the argument π contained in ψ should be accepted by $\mathsf{ZK.Ver}$. Therefore, the verification algorithm $\mathsf{Verify}(\tau, \psi)$ should output 1.

Moreover, the correctness of $\mathcal{E}_{\mathcal{R}}$ implies that $w = w' = \mathsf{E}_{\mathsf{R}}.\mathsf{Dec}(\mathsf{sk}_{\mathsf{id}}, \mathbf{c}_{\mathsf{R}})$. Similarly, the correctness of $\mathcal{E}_{\mathcal{OA}}$ ensures that $\mathsf{E}_{\mathsf{OA}}.\mathsf{Dec}(\mathsf{sk}_{\mathsf{OA}}, \mathbf{c}_{\mathsf{OA}})$ returns $\mathbf{d} = (1 - P_\tau(\mathsf{id}, w)) \cdot \mathsf{id}$. Consequently, algorithm Open returns id if $P_\tau(\mathsf{id}, w) = 0$ and it returns $\perp$ if $P_\tau(\mathsf{id}, w) = 1$.

Security. The security properties of the proposed GEOT system are based on those of the employed cryptographic building blocks. In particular, `message secrecy` relies on the CCA2-security of $\mathcal{E}_{\mathcal{R}}$ and the ZK property of $\mathcal{NIZK}$. The latter, together with the CCA2-security and key-privacy of $\mathcal{E}_R$ and CCA2-security of $\mathcal{E}_{\mathcal{OA}}$, ensure `anonymity` of the system. Furthermore, `unforgeability` is based on the extractability of $\mathcal{NIZK}$ and the unforgeability of $\mathcal{SIG}$. Formally, we have the following theorem.

Theorem 1. *Assume that $\mathcal{SIG}$ is an EUF-CMA secure signature scheme, $\mathcal{E}_{\mathcal{R}}$ is a key-private CCA2-secure encryption scheme, $\mathcal{E}_{\mathcal{OA}}$ is a CCA2-secure encryption scheme, and $\mathcal{NIZK}$ is a dual-mode NIZK argument system for relation $\mathcal{R}$. Then the presented* GEOT *system satisfies* `message secrecy`*,* `Type-1 anonymity`*,* `Type-2 anonymity`*, and* `unforgeability`*.*

We prove Theorem 1 via the following lemmas.

Lemma 1 (Message Secrecy). *The proposed* GEOT *scheme satisfies message secrecy if $\mathcal{E}_{\mathcal{R}}$ is CCA2-secure and $\mathcal{NIZK}$ is statistically ZK in the hiding mode.*

Proof. We consider a sequence of hybrid games, the first of which is the original experiment $\mathbf{Exp}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda)$ in Definition 2 between a challenger and an adversary $\mathcal{A}$ who is given the secret keys of both the GM and the OA. The last one no longer depends on bit b chosen by the challenger. We will show that any two consecutive games in the sequence are computationally/statistically indistinguishable.

Game 1. This is exactly the original experiment. In this game, the challenge ciphertext ψ has the form $(\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$, with $\mathbf{c}_{\mathsf{R}} = \mathsf{E}_{\mathsf{R}}.\mathsf{Enc}(\mathsf{id}, w_b, s_{\mathsf{R}})$, $\mathbf{c}_{\mathsf{OA}} = \mathsf{E}_{\mathsf{OA}}.\mathsf{Enc}(\mathsf{ek}_{\mathsf{OA}}, d \cdot \mathsf{id}, s_{\mathsf{OA}})$, and $d = (1 - P_\tau(\mathsf{id}, w_b)) = (1 - P_\tau(\mathsf{id}, w_{1-b}))$ is essentially independent of b, and π is a faithfully generated NIZK argument.

Game 2. This game is exactly as **Game 1** with the exception that the simulation trapdoor τ_{sim} is kept by the challenger instead of being discarded from the output of $\mathsf{ZK}.\mathsf{Setup}(1^\lambda)$. The two games are identical from $\mathcal{A}$'s viewpoint.

Game 3. This game differs from **Game 2** in how the proof π is generated. Instead of generating a legitimate proof for the statement x using witness ξ, this game simulates π as $\pi \leftarrow \mathsf{ZK}.\mathsf{Sim}(\mathsf{crs}, \tau_{\mathsf{sim}}, x)$. Since $\mathcal{NIZK}$ is statistically ZK in the hiding mode, **Game 2** and **Game 3** are statistically indistinguishable.

Game 4. This game differs from **Game 3** in how the ciphertext $\mathbf{c}_{\mathsf{R}}$ is generated. Specifically, we replace $\mathbf{c}_{\mathsf{R}} = \mathsf{E}_{\mathsf{R}}.\mathsf{Enc}(\mathsf{id}, w_b, s_{\mathsf{R}})$ by a ciphertext of a randomly chosen plaintext under the same public key. Thanks to the CCA2-security of $\mathcal{E}_{\mathcal{R}}$, the two ciphertexts are computationally indistinguishable from $\mathcal{A}$'s viewpoint. Hence, **Game 4** and **Game 3** are computationally indistinguishable.

Observe that the challenge ciphertext ψ in **Game 4** is independent of the bit b chosen by the challenger in **Game 1**. Thus, the probability that $\mathcal{A}$ in **Game 4** outputs b' such that $b' = b$ is $\frac{1}{2}$. As **Game** 1 and **Game** 4 are computationally indistinguishable, the advantage $\mathbf{Adv}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda)$ of $\mathcal{A}$ in **Game 1** is negligible. This concludes the lemma. □

Lemma 2 (Type-1 Anonymity). *The proposed* GEOT *scheme satisfies type-1-anonymity if* $\mathcal{E}_{\mathcal{R}}$ *is key-private,* $\mathcal{E}_{\mathcal{OA}}$ *is CCA2-secure, and* $\mathcal{NIZK}$ *is statistically ZK in the hiding mode.*

Proof. We consider a sequence of hybrid games, the first of which is the original experiment $\mathbf{Exp}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda)$ in Definition 3 between a challenger and an adversary $\mathcal{A}$ who is not given the secret key of the OA. The last one no longer depends on bit b chosen by the challenger. We will show that any two consecutive games in the sequence are computationally/statistically indistinguishable.

Game 1. This is the original experiment. Here, the challenge ciphertext ψ has the form $(\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$, where $\mathbf{c}_{\mathsf{R}} = \mathsf{E}_{\mathsf{R}}.\mathsf{Enc}(\mathsf{id}_b, w, s_{\mathsf{R}})$, $\mathbf{c}_{\mathsf{OA}} = \mathsf{E}_{\mathsf{OA}}.\mathsf{Enc}(\mathsf{ek}_{\mathsf{OA}}, (1 - P_\tau(\mathsf{id}_b, w)) \cdot \mathsf{id}_b, s_{\mathsf{OA}})$, and π is an honestly generated NIZK argument.

Game 2. This game is exactly as **Game 1** with the exception that the simulation trapdoor τ_{sim} is kept by the challenger instead of being discarded from the output of $\mathsf{ZK}.\mathsf{Setup}(1^\lambda)$. The two games are identical from $\mathcal{A}$'s viewpoint.

Game 3. This game differs from **Game 2** in how the proof π is generated. Instead of generating a legitimate proof for the statement x using witness ξ, this game simulates π as $\pi \leftarrow \mathsf{ZK}.\mathsf{Sim}(\mathsf{crs}, \tau_{\mathsf{sim}}, x)$. Since $\mathcal{NIZK}$ is statistically ZK in the hiding mode, **Game 2** and **Game 3** are statistically indistinguishable.

Game 4. We modify how the ciphertext $\mathbf{c}_{\mathsf{OA}}$ is generated. Note that in **Game 3**, $\mathbf{c}_{\mathsf{OA}} = \mathsf{E}_{\mathsf{OA}}.\mathsf{Enc}(\mathsf{ek}_{\mathsf{OA}}, (1 - P_\tau(\mathsf{id}_b, w)) \cdot \mathsf{id}_b, s_{\mathsf{OA}})$. Here, instead of encrypting $(1 - P_\tau(\mathsf{id}_b, w)) \cdot \mathsf{id}_b$, we generate $\mathbf{c}_{\mathsf{OA}}$ as an encryption of a randomly chosen element of the plaintext space of $\mathcal{E}_{\mathcal{OA}}$. The IND-CCA2 security of $\mathcal{E}_{\mathcal{OA}}$ then ensures that $\mathcal{A}$ can distinguish between the two ciphertexts only with negligible advantage. Therefore, **Game 4** and **Game 3** are computationally indistinguishable.

Game 5. This game differs from **Game 4** in how the ciphertext $\mathbf{c}_{\mathsf{R}}$ is generated. Specifically, we replace $\mathbf{c}_{\mathsf{R}} = \mathsf{E}_{\mathsf{R}}.\mathsf{Enc}(\mathsf{id}_b, w, s_{\mathsf{R}})$ by a ciphertext of the same plaintext w under under a randomly generated public key (obtained from $\mathsf{E}_{\mathsf{R}}.\mathsf{Kg}(1^\lambda)$). Thanks to the key-privacy property of $\mathcal{E}_{\mathcal{R}}$, the two ciphertexts are computationally indistinguishable from $\mathcal{A}$'s viewpoint. Hence, **Game 5** and **Game 4** are computationally indistinguishable.

Observe that the challenge ciphertext ψ in **Game 5** is independent of the bit b chosen by the challenger in **Game 1**. Thus, the advantage of $\mathcal{A}$ in **Game 5** is 0. As **Game** 1 and **Game** 5 are computationally indistinguishable, the advantage $\mathbf{Adv}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda)$ of $\mathcal{A}$ in **Game 1** is negligible. This concludes the lemma. □

Lemma 3 (Type-2 Anonymity). *The* GEOT *scheme satisfies type-2-anonymity if $\mathcal{E}_{\mathcal{R}}$ is key-private and $\mathcal{NIZK}$ is statistically ZK in the hiding mode.*

The proof of Lemma 3 is very similar to the one for Lemma 2, except that we do not require the CCA2-security of $\mathcal{E}_{\mathcal{OA}}$. In other words, there is no need to consider **Game 4**. The details are hence omitted.

Lemma 4 (Extractability). *The* GEOT *scheme is extractable if $\mathcal{NIZK}$ has CRS indistinguishability and statistical extractability in the binding mode.*

Proof. By the design of algorithms Setup and SimSetup, the only difference lies in the generation of the common reference strings crs. By the CRS indistinguishability of the employed $\mathcal{NIZK}$, the crs's outputted by ZK.Setup and ZK.ExtSetup are indistinguishable, so are the outputs of Setup and SimSetup.

Next, let us consider the experiment $\mathbf{Exp}^{\mathrm{extract}}_{\mathcal{A}}(1^\lambda)$ in Definition 5. The adversary $\mathcal{A}$ is given $(\mathsf{pp}, \mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}})$. At some point, $\mathcal{A}$ outputs a valid pair (τ, ψ), where $\psi = (\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$.

Let $x = (\mathsf{vk}_{\mathsf{GM}}, \mathsf{ek}_{\mathsf{OA}}, \mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \mathcal{S}, F_\tau, P_\tau)$ and let $\xi' \leftarrow \mathsf{ZK.Extract}(\mathsf{crs}, \tau_{\mathsf{ext}}, x, \pi)$ of the form $\xi' = (w', \mathsf{id}', \mathsf{cert}'_{\mathsf{id}}, s'_{\mathsf{R}}, s'_{\mathsf{OA}})$. Also, let $\mathsf{id}^* \leftarrow \mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$.

Since $\mathcal{NIZK}$ has statistical extractability in the binding mode, $(x, \xi') \in \mathcal{R}$ with overwhelming probability. In particular, one has $F_\tau(w') = 1$ and $\mathsf{id}^* = (1 - P_\tau(\mathsf{id}', w')) \cdot \mathsf{id}'$. Consequently, either $P_\tau(\mathsf{id}', w') = 1$ and $\mathsf{id}^* = \perp$, or $P_\tau(\mathsf{id}', w') = 0$ and $\mathsf{id}^* \neq \perp$. In other words, $\mathbf{Adv}^{\mathsf{extract}}_{\mathcal{A}}(1^\lambda)$ is negligible in λ. □

Lemma 5 (Unforgeability). *The* GEOT *scheme is unforgeable if $\mathcal{SIG}$ is EUF-CMA secure and $\mathcal{NIZK}$ has statistical extractability in the binding mode.*

Proof. Let us consider experiment $\mathbf{Exp}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda)$ in Definition 6. The adversary $\mathcal{A}$ is given $(\mathsf{pp}, \mathsf{sk}_{\mathsf{OA}})$. At some point, $\mathcal{A}$ outputs a valid pair (τ, ψ), where $\psi = (\mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \pi)$. Let $x = (\mathsf{vk}_{\mathsf{GM}}, \mathsf{ek}_{\mathsf{OA}}, \mathbf{c}_{\mathsf{R}}, \mathbf{c}_{\mathsf{OA}}, \mathcal{S}, F_\tau, P_\tau)$, where $\mathcal{S}$ is the list of

revoked users at time τ, and let $\xi' \leftarrow \mathsf{ZK.Extract}(\mathsf{crs}, \tau_{\mathsf{ext}}, x, \pi)$ be of the form $\xi' = (w', \mathsf{id}', \mathsf{cert}'_{\mathsf{id}}, s'_{\mathsf{R}}, s'_{\mathsf{OA}})$. Also, let $\mathsf{id}^* \leftarrow \mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$. Since $\mathcal{NIZK}$ has statistical extractability, $(x, \xi') \in \mathcal{R}$ with overwhelming probability. In particular, $F_\tau(w') = 1$ and $\mathsf{id}^* = (1 - P_\tau(\mathsf{id}', w')) \cdot \mathsf{id}'$. As a result, either $P_\tau(\mathsf{id}', w') = 1$ and $\mathsf{id}^* = \perp$, or $P_\tau(\mathsf{id}', w') = 0$ and $\mathsf{id}^* = \mathsf{id}'$. Consequently, the adversary can possibly win the experiment if id' is not active at time τ. For the sake of contradiction, assume that this is the case.

Note that the extractability of $\mathcal{NIZK}$ also implies that $\mathsf{id}' \notin \mathcal{S}$. Hence, we can exclude the case that id' has joined the system, but is revoked at time τ. This leaves only the case where the adversary has not properly joined the system, i.e., there is no corresponding record for id' in the table **reg**. This means that "message" id' was not signed by the GM. However, as we have $\mathsf{S.Ver}(\mathsf{vk}_{\mathsf{GM}}, \mathsf{id}', \mathsf{cert}'_{\mathsf{id}}) = 1$ (again, thanks to the extractability of $\mathcal{NIZK}$), we can deduce that the pair $(\mathsf{id}', \mathsf{cert}'_{\mathsf{id}})$ yields a valid forgery for $\mathcal{SIG}$. Thus, by the EUF-CMA security of $\mathcal{SIG}$, the probability that id' is not active at time τ is negligible. This concludes the proof. □

4 Code-Based Instantiation

In this section, using the high-level ideas of Section 3, we will develop a code-based GEOT scheme based on the FDGE scheme from [59]. Recall that the scheme from [59] uses an updatable code-based Merkle tree [61] (which can be seen as a weak form of signature) to manage the enrollments and revocations of users[2], CCA2-secure versions [62,57] of McEliece [53] for the encryption layers. The difference from [59] is that we employ VOLEitH-based zero-knowledge protocol [9,63] instead of Stern-like [69] zero-knowledge protocols [30,61] for proving ciphertext validity. (The details of these code-based cryptographic tools and VOLEitH-based ZK proofs are provided in Appendix B and Appendix C.)

To upgrade the scheme from [59] to a GEOT scheme, we introduce extensions with tracing and filtering policies. Our code-based GEOT scheme can work smoothly with both general policies. In particular, we model tracing policies and filtering policies as polynomial-size Boolean circuits. The supporting ZK techniques to prove correct evaluations of policies are presented in Appendix C.2.

In the random oracle model, the correctness and security properties of the scheme rely on those of the employed ingredients (i.e., code-based Merkle trees, McEliece encryption and VOLEitH-based ZK protocols) and the fact that, in the non-traceable case, even if the Type-2-anonymity adversary is given OA's secret key, it can learn no additional information about the receiver's identity.

4.1 Description of Our Code-Based Construction

In the scheme description below, we will generically use F_τ and P_τ to denote the policies without specifying their formulations.

[2] Hence, compared to the construction in Section 3, our code-based instantiation here does not need a full-fledged signature and the revocation list $\mathcal{S}$ is handled differently.

$\mathsf{Setup}(1^\lambda)$ Given the security parameter 1^λ, this setup algorithm first generates the following public parameters, including descriptions of $\mathcal{ID}, \mathcal{W}, \mathcal{T}, \mathcal{P}, \mathcal{F}$.

(1) Integers $n, c \in \mathbb{Z}^+$ such that $n = \mathcal{O}(\lambda)$, $c = \mathcal{O}(1)$, and $c \mid n$. Set $m = 2 \cdot 2^c \cdot n/c$. Let the user identifier space be $\mathcal{ID} = \{0,1\}^n$.

(2) An integer $\ell = \ell(\lambda)$ that specifies the expected number $N = 2^\ell$ of users in the system.

(3) An integer t that specifies the message space $\mathcal{W} = \{0,1\}^t$.

(4) An ordered time space $\mathcal{T} = \{\tau_1, \tau_2, \ldots\}$.

(5) A predicate family $\mathcal{P} = \{P_\tau : \{0,1\}^{n+t} \to \{0,1\}\}_{\tau \in \mathcal{T}}$.

(6) A message filtering policy $\mathcal{F} = \{F_\tau : \mathcal{W} \to \{0,1\}\}_{\tau \in \mathcal{T}}$.

(7) Two sets of parameters $(n_1, k_1, t_1, k_1 - n, n)$ and $(n_2, k_2, t_2, k_2 - t, t)$ for the McEliece encryption scheme with regular noise. (See Appendix B.2 for this variant). Thus, we also specify integers $k_{e,1}, k_{e,2}, c_{e,1}, c_{e,2}$ such that $c_{e,1} \mid k_{e,1}$, $c_{e,2} \mid k_{e,2}$ and $n_1 = k_{e,1}/c_{e,1} \cdot 2^{c_{e,1}}$, $n_2 = k_{e,2}/c_{e,2} \cdot 2^{c_{e,2}}$. We require that $n \mid n_2$ for ease of presentation. Looking ahead, the first set is used by the OA for tracing purpose while the second set is used by the users for encrypting messages.

(8) A random matrix $\mathbf{B} \xleftarrow{\$} \{0,1\}^{n \times m}$, which specifies a code-based hash function $h_{\mathbf{B}}$ (as in Definition 10). This hash function is then employed in the Merkle tree accumulator (see Appendix B.1) to accumulate user public keys and identifiers.

(9) A small integer $q = 2^r$ defining the finite field $\mathbb{F}_q$ and a positive integer δ such that $\max\{c+1, c_{e,1}, c_{e,2}\}/q^\delta = \mathsf{negl}(\lambda)$. The parameter δ defines a $[\delta, 1, \delta]$-linear code over $\mathbb{F}_q$. Looking ahead, these will serve as auxiliary parameters for the $\mathcal{F}_{\mathsf{sVOLE}}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}$ functionality, which is called upon executing VOLEitH-based zero-knowledge proof (see Appendix C.1).

(10) A hash function $\mathcal{H}_{\mathsf{FS}}$ modeled as a random oracle in the security proof.

Let

$$\mathsf{pp}' = \{n, c, m, t, \ell, N, \mathcal{T}, \mathcal{P}, \mathcal{F}, n_1, k_{e,1}, c_{e,1}, k_1, t_1, n_2, k_{e,2}, c_{e,2}, k_2, t_2, \mathbf{B}, q, \delta, \mathcal{H}_{\mathsf{FS}}\}.$$

Next, this algorithm generates secret keys for GM and OA as follows.

(i) Run the McEliece key generation $\mathsf{McKeygen}(n_1, k_1, t_1)$ twice, obtaining $(\mathbf{G}_{\mathsf{oa},0}, \mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)})$ and $(\mathbf{G}_{\mathsf{oa},1}, \mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},1)})$. Define $\mathsf{sk}_{\mathsf{OA}} = (\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)}, \mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},1)})$.

(ii) Sample $\mathsf{sk}_{\mathsf{GM}} \xleftarrow{\$} \{0,1\}^{2n}$ and computes $\mathbf{y}_{\mathsf{GM}} = \mathbf{B} \cdot \mathsf{RE}(\mathsf{sk}_{\mathsf{GM}})$. (See Appendix B for the definition of the regular encoding algorithm RE.)

(iii) Initialize a registration table $\mathbf{reg} = (\mathbf{reg}[0], \ldots, \mathbf{reg}[N-1])$, where for each i, $\mathbf{reg}[i] = (\mathbf{reg}[i][0], \mathbf{reg}[i][1], \mathbf{reg}[i][2], \mathbf{reg}[i][3]) = (\mathbf{0}^n, \mathbf{0}, -1, -1)$. Here $\mathbf{reg}[i][0]$ stores a hash value, $\mathbf{reg}[i][1]$ stores the membership certificate, $\mathbf{reg}[i][2]$ and $\mathbf{reg}[i][3]$ are the epochs at which the user joins and leaves the group, respectively.

(iv) Initialize a Merkle tree MTr built from $(\mathbf{reg}[0][0], \ldots, \mathbf{reg}[N-1][0])$ using the hash function $h_{\mathbf{B}}$. Note that all nodes in MTr are zero at this stage.

(v) Initialize state $j = 0$ and group information $\mathsf{info} := \emptyset$.

Return $\mathsf{pp} = \{\mathsf{pp}', \mathbf{G}_{\mathsf{oa},0}, \mathbf{G}_{\mathsf{oa},1}, \mathbf{y}_{\mathsf{GM}}\}$, $\mathsf{sk}_{\mathsf{OA}}$, and $\mathsf{sk}_{\mathsf{GM}}$. We assume that only one with the key $\mathsf{sk}_{\mathsf{GM}}$ can edit $\mathbf{reg}$ and info, and the well-formedness of these databases can be publicly verified.

$\langle \mathsf{Join}(\mathsf{info}), \mathsf{Issue}(\mathsf{sk}_{\mathsf{GM}}, \mathsf{info}, \mathbf{reg}) \rangle$ This protocol is triggered if a user requests to join the group at τ_{current}. Let the state of the GM be j and the user identifier be $\mathsf{id}_j \in \{0,1\}^n$. The user first performs the following steps.

1. Generate its encryption key by running the algorithm $\mathsf{McKeygen}(n_2, k_2, t_2)$ twice, resulting in two key pairs $(\mathbf{G}_{j,0}, \mathsf{sk}_{\mathsf{Mc}}^{(j,0)})$ and $(\mathbf{G}_{j,1}, \mathsf{sk}_{\mathsf{Mc}}^{(j,1)})$. Set $\mathsf{pk}_j = (\mathbf{G}_{j,0}, \mathbf{G}_{j,1}) \in (\{0,1\}^{n_2 \times k_2})^2$ and $\mathsf{sk}_j = (\mathsf{sk}_{\mathsf{Mc}}^{(j,0)}, \mathsf{sk}_{\mathsf{Mc}}^{(j,1)})$.
2. Next, it computes a hash $\mathbf{d}'_j$ of the key pk_j as follows:
   - For $b \in \{0,1\}$, denote $\mathbf{G}_{j,b} = [\mathbf{g}_{j,k_2 \cdot b} | \mathbf{g}_{j,k_2 \cdot b+1} | \cdots | \mathbf{g}_{j,k_2 \cdot b+k_2-1}]$.
   - For each $l = 0, \ldots, k_2 - 1$, split the column vectors $\mathbf{g}_{j,k_2 \cdot b+l} \in \{0,1\}^{n_2}$ into n_2/n column vectors $\mathbf{g}_{j,k_2 \cdot b+l,0}, \ldots, \mathbf{g}_{j,k_2 \cdot b+l,n_2/n-1} \in \{0,1\}^n$. Note that, we should have
   $$\mathbf{g}_{j,k_2 \cdot b+l} = \left(\mathbf{g}_{j,k_2 \cdot b+l,0} \| \ldots \| \mathbf{g}_{j,k_2 \cdot b+l,n_2/n-1}\right)$$
   - Let $D_j = \{\mathbf{g}_{j,0,0}, \ldots, \mathbf{g}_{j,0,n_2/n-1}, \ldots, \mathbf{g}_{j,2k_2-1,0}, \ldots, \mathbf{g}_{j,2k_2-1,n_2/n-1}\}$. By "padding" zero vectors $\mathbf{0}^n$ to D_j such that $|D_j|$ is a power of 2, the sender can run the accumulating algorithm $\mathsf{TAccu}_{\mathbf{B}}(D_j)$ (see Appendix B.1) to obtain $\mathbf{d}'_j \in \{0,1\}^n$.
3. Let $\mathbf{B} = [\mathbf{B}_0 | \mathbf{B}_1]$. Compute $\mathbf{d}_j = \mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{d}'_j) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathsf{id}_j)$.
4. If $\mathbf{d}_j$ is non-zero, the user sends $(\mathsf{id}_j, \mathsf{pk}_j, \mathbf{d}_j)$ to the GM. Otherwise, it repeats the above steps. We remark that $\mathbf{d}_j$ is non-zero with overwhelming probability for large enough n.

Upon receiving the tuple from the user, the GM checks the following: (i) the authenticity of the user identifier id_j; (ii) both $\mathbf{G}_{j,0}$ and $\mathbf{G}_{j,1}$ are not registered by any other user; (iii) the non-zero $\mathbf{d}_j$ is computed from id_j and pk_j. If either condition is not satisfied, the GM rejects. Else, the GM sends a membership identifier $j = (j_{\ell-1}, \ldots, j_0)_2$ to the user and sets $\mathsf{cert}_{\mathsf{id}_j} = (\mathsf{bin}(j), \mathsf{id}_j, \mathsf{pk}_j, \mathbf{d}_j)$. In addition, the GM performs the following.

- Update $\mathbf{reg}[j][0] = \mathbf{d}_j$, $\mathbf{reg}[j][1] = \mathsf{cert}_{\mathsf{id}_j}$, and $\mathbf{reg}[j][2] = \tau_{\text{current}}$.
- Update the Merkle tree MTr by running $\mathsf{TUpdate}_{\mathbf{B}}(j, \mathbf{d}_j)$.
- Increase the state j to $j + 1$.

$\mathsf{GUpdate}(\mathsf{sk}_{\mathsf{GM}}, \mathcal{S}, \mathsf{info}, \mathbf{reg})$ The group manager runs this algorithm to advance the time epoch to τ_{new} and update the group information to $\mathsf{info}_{\tau_{\text{new}}}$. Concretely, it proceeds as follows.

1. Let $\mathcal{S} = \{i_1, i_2, \ldots, i_r\}$ be the membership identifiers of to-be-revoked users. If $\mathcal{S} = \emptyset$, go to Step 2. Else, for each $k \in [1, r]$, the GM updates MTr by running $\mathsf{TUpdate}_{\mathbf{B}}(i_k, \mathbf{0}^n)$ and sets $\mathbf{reg}[i_k][3] = \tau_{\text{new}}$.

2. Let $\mathcal{D} = \{\mathbf{d}_j\}_j$ be the set of non-zero values accumulated in the updated tree MTr. For each j, let $w^{(j)} = \mathsf{TWitGen}_{\mathbf{B}}(\mathcal{D}, \mathbf{d}_j) \in \{0,1\}^\ell \times (\{0,1\}^n)^\ell$ be the witness that $\mathbf{d}_j$ is accumulated in the root $\mathbf{u}_{\tau_{\mathrm{new}}}$ of the updated tree. Then GM outputs $\mathsf{info}_{\tau_{\mathrm{new}}}$ as

$$\mathsf{info}_{\tau_{\mathrm{new}}} = (\mathbf{u}_{\tau_{\mathrm{new}}}, \{w^{(j)}\}_j).$$

We remark that the updated group information is for active users only. This is because each of the zero leaves in MTr corresponds to a user who has been revoked from the group or has not joined the group yet and thus is not allowed to receive any ciphertext at τ_{new}.

$\mathsf{Enc}(\tau, \mathbf{w}, \mathsf{id}_j, \mathsf{cert}_{\mathsf{id}_j})$ Parse $\mathsf{cert}_{\mathsf{id}_j} = ((j_{\ell-1}, \ldots, j_0)^\top, \mathsf{id}_j, \mathbf{G}_{j,0}, \mathbf{G}_{j,1}, \mathbf{d}_j)$, and extract $(\mathbf{G}_{\mathsf{oa},0}, \mathbf{G}_{\mathsf{oa},1})$ from pp. This algorithm is run by a sender who wishes to send a message $\mathbf{w} \in \{0,1\}^t$ to user $\mathsf{id}_j = (x_{j,0}, \ldots, x_{j,n-1})^\top \in \{0,1\}^n$ with respect to time epoch τ. The sender checks that $F_\tau(\mathbf{w}) = 1$, and id_j is active at τ. If either of the two conditions does not hold, abort. The sender then downloads from info_τ the associated witness $w^{(j)} = (\mathsf{bin}(j), \mathbf{w}_0, \ldots, \mathbf{w}_{\ell-1})$ and performs the following steps.

1. Encrypt $\mathbf{w}$ under the keys $\mathbf{G}_{j,0}$ and $\mathbf{G}_{j,1}$.
   - Sample vectors $\mathbf{r}_{w,0}, \mathbf{r}_{w,1} \xleftarrow{\$} \{0,1\}^{k_2 - t}$ and $\mathbf{e}_{w,0}, \mathbf{e}_{w,1} \xleftarrow{\$} \mathbb{F}_2^{k_{e,2}}$.
   - For $i \in \{0,1\}$, compute

$$\mathbf{c}_{w,i} = \mathbf{G}_{j,i} \cdot \begin{pmatrix} \mathbf{r}_{w,i} \\ \mathbf{w} \end{pmatrix} \oplus \mathsf{RE}(\mathbf{e}_{w,i}) \in \{0,1\}^{n_2}, \tag{2}$$

   where RE is the regular encoding function (see Appendix B). Let $\mathbf{c}_w = (\mathbf{c}_{w,0}, \mathbf{c}_{w,1}) \in \{0,1\}^{n_2} \times \{0,1\}^{n_2}$.
2. Let $b = P_\tau(\mathsf{id}_j, \mathbf{w})$. Encrypt $(1-b) \cdot \mathbf{d}_j \in \{0,1\}^n$ under keys $\mathbf{G}_{\mathsf{oa},0}$, $\mathbf{G}_{\mathsf{oa},1}$.
   - Sample vectors $\mathbf{r}_{\mathsf{oa},0}, \mathbf{r}_{\mathsf{oa},1} \xleftarrow{\$} \{0,1\}^{k_1 - n}$ and $\mathbf{e}_{\mathsf{oa},0}, \mathbf{e}_{\mathsf{oa},1} \xleftarrow{\$} \mathbb{F}_2^{k_{e,1}}$.
   - For $i \in \{0,1\}$, compute

$$\mathbf{c}_{\mathsf{oa},i} = \mathbf{G}_{\mathsf{oa},i} \cdot \begin{pmatrix} \mathbf{r}_{\mathsf{oa},i} \\ (1-b) \cdot \mathbf{d}_j \end{pmatrix} \oplus \mathsf{RE}(\mathbf{e}_{\mathsf{oa},i}) \in \{0,1\}^{n_1}. \tag{3}$$

   Let $\mathbf{c}_{\mathsf{oa}} = (\mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}) \in \{0,1\}^{n_1} \times \{0,1\}^{n_1}$.
3. Let ξ be of the following form

$$\begin{aligned} \xi \ = (\ & \mathbf{r}_{w,0},\ \mathbf{r}_{w,1},\ \mathbf{e}_{w,0},\ \mathbf{e}_{w,1},\ \mathbf{w},\ \mathbf{r}_{\mathsf{oa},0},\ \mathbf{r}_{\mathsf{oa},1},\ \mathbf{e}_{\mathsf{oa},0},\ \mathbf{e}_{\mathsf{oa},1}, \\ & b,\ \mathsf{bin}(j),\ \mathsf{id}_j,\ \mathbf{G}_{j,0},\ \mathbf{G}_{j,1},\ \mathbf{d}'_j,\ \mathbf{d}_j,\ \mathbf{w}_0,\ \ldots,\ \mathbf{w}_{\ell-1}\ ). \end{aligned} \tag{4}$$

   Generate an NIZK argument of knowledge of ξ such that the following conditions hold.
   (a) $\mathbf{c}_{w,0}, \mathbf{c}_{w,1}$ encrypt $\mathbf{w}$ under the keys $\mathbf{G}_{j,0}, \mathbf{G}_{j,1}$, respectively. Specifically, for $i \in \{0,1\}$, the equation (2) holds.

(b) The plaintext $\mathbf{w}$ satisfies the filtering relation $F_\tau(\mathbf{w}) = 1$.

(c) $\mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}$ encrypt $(1-b)\cdot\mathbf{d}_j$ under the keys $\mathbf{G}_{\mathsf{oa},0}, \mathbf{G}_{\mathsf{oa},1}$, i.e., the equation (3) holds for $i \in \{0,1\}$. Therefore, the non-zero $\mathbf{d}_j$ is encrypted if and only if $b = P_\tau(\mathsf{id}_j, \mathbf{w}) = 0$.

(d) The non-zero vector $\mathbf{d}_j$ is accumulated in the root $\mathbf{u}_\tau$ at epoch τ, i.e., the equation $\mathsf{TVerify}_{\mathbf{B}}(\mathbf{u}_\tau, \mathbf{d}_j, w^{(j)}) = 1$ holds.

(e) The keys $\mathbf{G}_{j,0}, \mathbf{G}_{j,1}$ and the user identifier id_j are hashed to a non-zero $\mathbf{d}_j$. Concretely, $\mathbf{d}'_j = \mathsf{TAccu}_{\mathbf{B}}(\mathbf{G}_{j,0}, \mathbf{G}_{j,1})$, $\mathbf{d}_j = \mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{d}'_j) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathsf{id}_j)$, and $\mathbf{d}_j \neq \mathbf{0}^n$. The latter two conditions are to ensure that id_j is a certified and active user at τ.

To this end, the sender converts the above conditions to a set of t_{proof} polynomial equations over $\mathbb{F}_2$, which have degrees at most $d_{\mathsf{proof}} = \max\{c+1, c_{e,1}, c_{e,2}\}$ and are satisfied by a witness in $\{0,1\}^{l_{\mathsf{proof}}}$. We specify the conversion and the values $t_{\mathsf{proof}}, d_{\mathsf{proof}}$ and l_{proof} in Appendix C.2. Then the sender executes the VOLEitH-based NIZK protocol in Appendix C.1. Let the final proof be π_ψ. Return $\psi = (\mathbf{c}_{w,0}, \mathbf{c}_{w,1}, \mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}, \pi_\psi)$.

$\mathsf{Verify}(\tau,\ \psi)$ Let ψ be as above. This algorithm simply runs the verification algorithm of the VOLEitH proof system. Return 1 if the verification algorithm returns 1, and 0 otherwise.

$\mathsf{Dec}(\mathsf{sk}_{\mathsf{id}_j}, \tau, \psi)$ Let $\psi = (\mathbf{c}_{w,0}, \mathbf{c}_{w,1}, \mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}, \pi_\psi)$ and $\mathsf{sk}_{\mathsf{id}_j} = (\mathsf{sk}_{\mathsf{Mc}}^{(j,0)}, \mathsf{sk}_{\mathsf{Mc}}^{(j,1)})$. If $\mathsf{Verify}(\tau, \psi) = 0$, return $\perp$. Otherwise, user id_j decrypts ψ by running the algorithm $\mathbf{w}' \leftarrow \mathsf{McDec}(\mathsf{sk}_{\mathsf{Mc}}^{(j,0)}, \mathbf{c}_{w,0})$ (see Appendix B.2).

$\mathsf{Open}(\mathsf{sk}_{\mathsf{OA}}, \tau, \psi)$ Let $\psi = (\mathbf{c}_{w,0}, \mathbf{c}_{w,1}, \mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}, \pi_\psi)$ and $\mathsf{sk}_{\mathsf{OA}} = (\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)}, \mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},1)})$. If $\mathsf{Verify}(\tau, \psi) = 0$, return $\perp$. Otherwise, the OA decrypts ψ by running the algorithm $\widetilde{\mathbf{d}} \leftarrow \mathsf{McDec}(\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)}, \mathbf{c}_{\mathsf{oa},0})$.

1. If $\widetilde{\mathbf{d}} = \mathbf{0}^n$, indicating that ψ is non-traceable, return $\perp$.
2. If $\widetilde{\mathbf{d}} \neq \mathbf{0}^n$, find the index j in $\mathbf{reg}$ such that $\mathbf{reg}[j][0] = \widetilde{\mathbf{d}}$. Let $\mathbf{reg}[j][1] = (\mathsf{bin}(j), \mathsf{id}_j, \mathsf{pk}_j, \mathbf{d}_j)$, return id_j. If no such index is found, return $\perp$.

4.2 Efficiency, Correctness, and Security Analyses

Efficiency. The asymptotic efficiency of our construction is as follows.

(a) The secret key $\mathsf{sk}_{\mathsf{GM}}$ of the GM has bit-size $\mathcal{O}(\lambda)$, while the secret key $\mathsf{sk}_{\mathsf{OA}}$ of the OA has bit-size $\mathcal{O}(\lambda^2)$.

(b) The size of public parameters pp is dominated by that of matrices $\mathbf{G}_{\mathsf{oa},0}, \mathbf{G}_{\mathsf{oa},1}$, $\mathbf{B}$, which have bit size $\mathcal{O}(\lambda^2)$.

(c) The membership certificate and secret key of users have bit-size $\mathcal{O}(\lambda^2)$.

(d) The message sender needs to download information of bit-size $\mathcal{O}(\lambda \cdot \log \lambda)$. The verifier only needs to download $\mathbf{u}_\tau$, which has bit-size $\mathcal{O}(\lambda)$.

(e) The proof π_ψ has bit-size $\mathcal{O}(L_{\mathsf{proof}}) = \mathcal{O}(\lambda^2) + \mathcal{O}(|F_\tau|) + \mathcal{O}(|P_\tau|)$ where $|F_\tau|$ and $|P_\tau|$ denote the circuit sizes of filtering and tracing policies respectively. (See Appendix C.2 for more information.)

We provide an example parameter set aiming to achieve 128-bit security in Table 2. These parameters are used in the comparisons provided in Table 1. We choose the McEliece parameter according to the document [3] with some modifications. One suggested parameter set (n_1, k_1, t_1) in [3] is $(3488, 2720, 64)$. Since we employ the regular noise variant of the McEliece cryptosystem, which reduces the security level by a few bits (see, e.g., [50,51, Table 1]), we thus increase n_1 to 4096. As a result, the new parameter set becomes $(4096, 3328, 64)$. It can achieve 156-bit security, according to the estimator[3] provided in [50,51]. Even when considering a new distinguishing attack [66] against binary Goppa codes, the parameter set can still achieve 128-bit security. The parameters for the VOLEitH proof systems follow the specification in [8] to ensure all proof components achieve 128-bit security. For instance, $r = 8$ and $\delta = 18$ ensure that soundness error in Theorem 3 is at most 2^{-128}. Finally, parameters for the 2-RNSD problems are chosen according to the best known attacks proposed in [4]. For the chosen parameter $(n, c, m) = (1024, 8, 2^{16})$, the GBA attack has complexity 2^{256} and the ISD attack has complexity 2^{161}, thus satisfying our requirements.

Table 2: Parameters for 128-bit security level

Parameters	Description	Value
λ	Security level	128
$N = 2^{\ell}$	Expected No. of users	1024
t	Message bit length	128
n	Hash $h_{\mathbf{B}}$ output length	1024
c	$2 - \mathsf{RNSD}$ parameter	8
$m = \frac{2n}{c} \cdot 2^c$	$h_{\mathbf{B}}$ input length	2^{16}
(n_1, k_1, t_1)	McEliece parameters set 1	$(4096, 3328, 64)$
$(k_{e,1}, c_{e,1})$	Regular noise encoding	$(384, 6)$
(n_2, k_2, t_2)	McEliece parameters set 2	$(4096, 3328, 64)$
$(k_{e,2}, c_{e,2})$	Regular noise encoding	$(384, 6)$
$\lvert P_\tau \rvert$	Circuit size of P_τ	10^5
$\lvert F_\tau \rvert$	Circuit size of F_τ	10^5
$q = 2^r$	Extension field	2^8
δ	Repetition of VOLEitH	18
$s = \lambda + 16$	Universal hash parameter	144
$h = \lambda + 16$	Universal hash parameter	144
t_{proof}	Number of poly. equations	$\approx 2.72 \cdot 10^7$
l_{proof}	Number of variables	$\approx 2.73 \cdot 10^7$
d_{proof}	Max. degree of poly. equations	9

[3] `https://gist.github.com/hansliu1024/21c87609e75f6cc52decdc69981e1d5b`

Sender's Complexity. We remark that that the runtime Enc algorithm is dominated by the time for generating a VOLEitH-based NIZK proof. Using the benchmark provided in Rust library gf256, we estimate that the proof time is around 260 seconds. The details are provided in Appendix C.2.

Correctness. The correctness of the above construction relies on the following facts: (i) The VOLEitH-based NIZK argument system for generating π_ψ is perfectly complete; (ii) The employed McEliece encryption schemes are correct.

First, for an honestly generated ciphertext $\psi = (\mathbf{c}_w, \mathbf{c}_{\mathsf{oa}}, \pi_\psi)$ that encrypts $\mathbf{w}$ to id, the perfect completeness of the system that generates π_ψ ensures that the Verify algorithm returns 1. Next, the Dec algorithm outputs $\mathbf{w}' = \mathbf{w}$ with overwhelming probability due to Fact (ii). Suppose $(1-b)\cdot\mathbf{d}$ is originally encrypted, then the outputted $\widetilde{\mathbf{d}}$ in the Open algorithm should be exactly $(1-b)\cdot\mathbf{d}$ with all but negligible probability. Here $\mathbf{d}$ is a hash value corresponding to id. If $b = P_\tau(\mathsf{id}, \mathbf{w}) = 1$, then $\widetilde{\mathbf{d}} = \mathbf{0}^n$ and Open returns $\perp$. If $P_\tau(\mathsf{id}, \mathbf{w}) = 0$, then $\widetilde{\mathbf{d}} \neq \mathbf{0}^n$. In particular, $\widetilde{\mathbf{d}} = \mathbf{d}$. Therefore, the opening algorithm is able to identify the receiver id by looking up the registration table $\mathbf{reg}$.

Security. In the following theorem, we prove that the above construction satisfies our stringent requirements in Section 2.2 and Section 2.3.

Theorem 2. *Assume that the VOLEitH-based NIZK argument system generating the proof π_ψ is simulation-sound extractable and ZK, the randomized McEliece cryptosystems are CPA-secure and key-private, and the AFS hash function is collision-resistant. Then in the random oracle model, the* GEOT *scheme satisfies message secrecy, type-1-anonymity, type-2-anonymity, and unforgeability.*

In the random oracle model, the above assumptions are based on the following.

1. The decisional McEliece problems $\mathsf{DMcE}(n_1, k_1, t_1)$, $\mathsf{DMcE}(n_2, k_2, t_2)$, and the decisional learning parity with regular noise problems $\mathsf{DLPN}(n_1, k_1 - n, \mathsf{Regular}(k_{e,1}, c_{e,1}))$, $\mathsf{DLPN}(n_2, k_2 - t, \mathsf{Regular}(k_{e,2}, c_{e,2}))$ problems are hard. This is to ensure that the randomized McEliece cryptosystems have pseudorandom ciphertexts (and hence, CPA-security and key-privacy). Via the Naor-Yung transformation [57], the obtained variant of McEliece encryption scheme is CCA2-secure.
2. The $2-\mathsf{RNSD}_{n,2n,c}$ problem is hard. Thus the used AFS hash function is collision resistant.

In the following, we will provide the proof for Type-1-anonymity of the scheme. Due to space restrictions, the proofs for Type-2-anonymity, message secrecy and unforgeability, are deferred to Appendix D.

Lemma 6 (Type 1 - Anonymity). *Assume that* $\mathsf{DMcE}(n_1, k_1, t_1)$, $\mathsf{DMcE}(n_2, k_2, t_2)$ *and* $\mathsf{DLPN}(n_1, k_1 - n, \mathsf{Regular}(k_{e,1}, c_{e,1}))$, $\mathsf{DLPN}(n_2, k_2 - t, \mathsf{Regular}(k_{e,2}, c_{e,2}))$ *are hard, the VOLEitH-based NIZK argument system used to generate π_ψ is simulation-sound and zero-knowledge, then the given* GEOT *construction is Type* 1 *- anonymous in the random oracle model.*

Proof. We prove this lemma using a series of indistinguishable games, where the first one is $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-b}(1^\lambda)$ and the last one no longer relies on b. Here $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-b}(1^\lambda)$ is the experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}}(1^\lambda)$ by fixing b. Denote W_i as the event that Game i returns 1.

Game 0**:** This is $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-b}(1^\lambda)$. The challenger $\mathcal{C}$ runs the initialization algorithm to produce public parameters pp, secret key $\mathsf{sk}_{\mathsf{OA}}$ for the OA, and secret key $\mathsf{sk}_{\mathsf{GM}}$ for the GM. $\mathcal{C}$ also maintains two lists HUL and CUL. The adversary $\mathcal{A}$ fully corrupts the GM by having the key $\mathsf{sk}_{\mathsf{GM}}$ as well as modifying **reg** and info at its will as long as both are well-formed and consistent. Throughout the experiment, $\mathcal{A}$ is allowed to enroll honest users to the group via the oracle USER and to learn their secret keys via $\mathsf{RevealU}$, and has decryption access to the OA's secret key $\mathsf{sk}_{\mathsf{OA}}$ via the oracle OPEN.
At some point, $\mathcal{A}$ outputs a challenge tuple $(\tau, \mathbf{w}, \mathsf{id}_{j_0}, \mathsf{id}_{j_1})$. This game proceeds only if the challenge tuple satisfies the specified conditions. Specifically, both users have to be active at τ and in the set $\mathsf{HUL}\backslash\mathsf{CUL}$, and $F_\tau(\mathbf{w}) = 1$. Let the membership certificate of id_{j_b} be $(\mathsf{bin}(j_b), \mathsf{id}_{j_b}, \mathbf{G}_{j_b,0}, \mathbf{G}_{j_b,1}, \mathbf{d}_{j_b})$. Next, $\mathcal{C}$ computes a challenge ciphertext $\psi^* = (\mathbf{c}^*_{w,0}, \mathbf{c}^*_{w,1}, \mathbf{c}^*_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}, \pi_{\psi^*})$, where $\mathbf{c}^*_{w,0}, \mathbf{c}^*_{w,1}$ encrypt $\mathbf{w}$ under the key $\mathbf{G}_{j_b,0}, \mathbf{G}_{j_b,1}$ while $\mathbf{c}^*_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}$ encrypt $(1 - P_\tau(\mathsf{id}_{j_b}, \mathbf{w})) \cdot \mathbf{d}_{j_b}$ under the key $\mathbf{G}_{\mathsf{oa},0}, \mathbf{G}_{\mathsf{oa},1}$. The adversary still has decryption access to $\mathsf{sk}_{\mathsf{OA}}$ as well as $\mathsf{sk}_{\mathsf{id}_{j_0}}$ and $\mathsf{sk}_{\mathsf{id}_{j_1}}$ with the exception that ψ is not allowed. Finally, $\mathcal{A}$ outputs a bit b' and this game returns 1 if and only if $b' = b$. By definition $\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-b}(1^\lambda) = 1] = \mathsf{Pr}[W_0]$.

Game 1**:** In this game, we use $\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},1)}$ instead of $\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)}$ to decrypt $\mathbf{c}_{\mathsf{oa},1}$ when $\mathcal{A}$ probes the OPEN oracle with a tuple (τ, ψ), where $\psi = (\mathbf{c}_{w,0}, \mathbf{c}_{w,1}, \mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}, \pi_\psi)$. Also, we use $\mathsf{sk}_{\mathsf{Mc}}^{(j_0,1)}$ (instead of $\mathsf{sk}_{\mathsf{Mc}}^{(j_0,0)}$) and $\mathsf{sk}_{\mathsf{Mc}}^{(j_1,1)}$ (instead of $\mathsf{sk}_{\mathsf{Mc}}^{(j_0,0)}$) to decrypt $\mathbf{c}_{w,1}$ when $\mathcal{A}$ invokes DEC w.r.t. id_{j_0} and id_{j_1}, respectively. The view of $\mathcal{A}$ remains the same as in **Game** 0 unless $\mathcal{A}$ comes up with a different tuple (τ', ψ') with $\psi' = (\mathbf{c}'_{w,0}, \mathbf{c}'_{w,1}, \mathbf{c}'_{\mathsf{oa},0}, \mathbf{c}'_{\mathsf{oa},1}, \pi_{\psi'})$ such that $\pi_{\psi'}$ is valid yet either $\mathbf{c}'_{\mathsf{oa},0}, \mathbf{c}'_{\mathsf{oa},1}$ encrypt different strings or $\mathbf{c}'_{w,0}, \mathbf{c}'_{w,1}$ encrypt different messages. However, this would breach the soundness of the argument system used to generate $\pi_{\psi'}$. Hence, $|\mathsf{Pr}[W_1] - \mathsf{Pr}[W_0]| \leq \mathbf{Adv}^{\mathsf{sound}} \leq \mathsf{negl}(\lambda)$.

Game 2**:** In this game, we resort to the zero-knowledge simulator of the argument system to obtain a simulated proof π_{ψ^*} when generating the challenge ciphertext. Since the system used to generate π_{ψ^*} is zero-knowledge, the view of $\mathcal{A}$ in this game is computationally close to that in Game 1, i.e., $|\mathsf{Pr}[W_2] - \mathsf{Pr}[W_1]| \leq \mathsf{negl}(\lambda)$ holds.

Game 3**:** This game changes the challenge ciphertext in the following way. Sample random vectors $\mathbf{c}^\star_{\mathsf{oa},0} \xleftarrow{\$} \{0,1\}^{n_1}$ and $\mathbf{c}^\star_{w,0} \xleftarrow{\$} \{0,1\}^{n_2}$ and replace $\psi^* = (\mathbf{c}^*_{w,0}, \mathbf{c}^*_{w,1}, \mathbf{c}^*_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}, \pi_{\psi^*})$ with $\psi^{**} = (\mathbf{c}^\star_{w,0}, \mathbf{c}^*_{w,1}, \mathbf{c}^\star_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}, \pi_{\psi^{**}})$. Since $\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)}$, $\mathsf{sk}_{\mathsf{Mc}}^{(j_0,0)}$, and $\mathsf{sk}_{\mathsf{Mc}}^{(j_1,0)}$ are no longer used for replying the oracle queries, any noticeable change in $\mathcal{A}$'s output can be employed to construct another adversary $\mathcal{B}$ who can distinguishes a McEliece ciphertext from random. However, the hardness of the problems $\mathsf{DMcE}(n_1, k_1, t_1)$, $\mathsf{DMcE}(n_2, k_2, t_2)$, $\mathsf{DLPN}(n_1, k_1 - n, \mathsf{Regular}(k_{e,1}, c_{e,1}))$ and $\mathsf{DLPN}(n_2, k_2 - t, \mathsf{Regular}(k_{e,2}, c_{e,2}))$

implies that $\mathcal{A}$ cannot tell whether this is Game 3 or Game 2 with non-negligible advantage. Thus $|\mathsf{Pr}[W_3] - \mathsf{Pr}[W_2]| \leq \mathsf{negl}(\lambda)$.

Game 4**:** This game switches back to $\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)}$, $\mathsf{sk}_{\mathsf{Mc}}^{(j_0,0)}$, and $\mathsf{sk}_{\mathsf{Mc}}^{(j_1,0)}$ and relies on those keys to compute the replies for the queries to oracles OPEN and DEC. The adversary is not aware of this modification unless $\mathcal{A}$ produces (τ', ψ') such that (i) (τ', ψ') is different from the challenge tuple $(\tau, \psi^{\star\star})$; (ii) $\pi_{\psi'}$ is valid and (iii) either $\mathbf{c}'_{\mathsf{oa},0}, \mathbf{c}'_{\mathsf{oa},1}$ encrypt different strings or $\mathbf{c}'_{w,0}, \mathbf{c}'_{w,1}$ encrypt different messages. This however, violates the simulation-soundness of the NIZK argument system used to generate $\pi_{\psi'}$. Note that $\mathcal{A}$ sees a simulated proof $\pi_{\psi^{\star\star}}$ for a false statement, where $\mathbf{c}^{\star}_{w,0}, \mathbf{c}^{\star}_{\mathsf{oa},0}$ are random while $\mathbf{c}^{*}_{w,1}, \mathbf{c}^{*}_{\mathsf{oa},0}$ encrypt $\mathbf{w}$ and $(1 - P_{\tau}(\mathsf{id}_{j_b}, \mathbf{w})) \cdot \mathbf{d}_{j_b}$, respectively. As a result, $|\mathsf{Pr}[W_4] - \mathsf{Pr}[W_3]| \leq \mathbf{Adv}^{\mathsf{sim-sound}} \leq \mathsf{negl}(\lambda)$.

Game 5**:** We further modify the challenge ciphertext $\psi^{\star\star}$ in this game. Sample random vectors $\mathbf{c}^{\star}_{\mathsf{oa},1} \xleftarrow{\$} \{0,1\}^{n_1}$ and $\mathbf{c}^{\star}_{w,1} \xleftarrow{\$} \{0,1\}^{n_2}$ and replace $\psi^{\star\star} = (\mathbf{c}^{\star}_{w,0}, \mathbf{c}^{*}_{w,1}, \mathbf{c}^{\star}_{\mathsf{oa},0}, \mathbf{c}^{*}_{\mathsf{oa},1}, \pi_{\psi^{\star\star}})$ with $\psi^{\star} = (\mathbf{c}^{\star}_{w,0}, \mathbf{c}^{\star}_{w,1}, \mathbf{c}^{\star}_{\mathsf{oa},0}, \mathbf{c}^{\star}_{\mathsf{oa},1}, \pi_{\psi^{\star}})$. Due to the security of the McEliece cryptosystems and the fact that $\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},1)}$, $\mathsf{sk}_{\mathsf{Mc}}^{(j_0,1)}$, and $\mathsf{sk}_{\mathsf{Mc}}^{(j_1,1)}$ are not used for replying the oracle queries, $\mathcal{A}$ cannot notice the modification significantly. Thus $|\mathsf{Pr}[W_5] - \mathsf{Pr}[W_4]| \leq \mathsf{negl}(\lambda)$.

Note that $\mathcal{A}$'s view in **Game** 5 does not depend on b. Hence, $\mathsf{Pr}[W_5] = \frac{1}{2}$. We then obtain $|\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-b}(1^{\lambda}) = b] - \frac{1}{2}| \leq \mathsf{negl}(\lambda)$, and

$$\begin{aligned}
\mathbf{Adv}_{\mathcal{A}}^{\mathsf{anony}}(1^{\lambda}) &= |\frac{1}{2}\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-1}(1^{\lambda}) = 1] + \frac{1}{2}\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-0}(1^{\lambda}) = 0] - \frac{1}{2}| \\
&\leq \frac{1}{2}|\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-1}(1^{\lambda}) = 1] - \frac{1}{2}| + \frac{1}{2}|\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{anony}-0}(1^{\lambda}) = 0] - \frac{1}{2}| \\
&\leq \mathsf{negl}(\lambda).
\end{aligned}$$

This ends the proof. □

Acknowledgements

This work is partially supported by the National Cryptologic Science Fund of China under grant 2025NCSF02039, by the National Science Foundation of China under grants 12401693 and 12361141818, and by Singapore Ministry of Education Academic Research Fund Tier 2 Grant MOE-T2EP20223-0016, the National Research Foundation, Singapore and Infocomm Media Development Authority under its Trust Tech Funding Initiative. W. Susilo is supported by the Australian Laureate Fellowship (FL230100033). This work is also supported by the open project funding of the Key Laboratory of Intelligent Sensing System and Security (Ministry of Education), Hubei University. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Infocomm Media Development Authority.

References

1. L. E. Aimani and M. Joye. Toward practical group encryption. In *ACNS 2013*, volume 7954 of *LNCS*, pages 237–252. Springer, 2013.
2. Q. Alamélou, O. Blazy, S. Cauchie, and P. Gaborit. A code-based group signature scheme. *Des. Codes Cryptography*, 82(1-2):469–493, 2017.
3. M. R. Albrecht, D. J. Bernstein, T. Chou, C. Cid, J. Gilcher, T. Lange, V. Maram, I. Von Maurich, R. Misoczki, R. Niederhagen, et al. Classic mceliece: conservative code-based cryptography. 2022. `https://classic.mceliece.org/nist.html`.
4. D. Augot, M. Finiasz, P. Gaborit, S. Manuel, and N. Sendrier. Sha-3 proposal: Fsb. *Submission to NIST*, pages 81–85, 2008.
5. D. Augot, M. Finiasz, and N. Sendrier. A fast provably secure cryptographic hash function. *IACR Cryptol. ePrint Arch.*, 2003:230, 2003.
6. D. Augot, M. Finiasz, and N. Sendrier. A family of fast syndrome based cryptographic hash functions. In *Mycrypt 2005*, volume 3715 of *LNCS*, pages 64–83. Springer, 2005.
7. C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, and P. Scholl. One tree to rule them all: Optimizing GGM trees and owfs for post-quantum signatures. In *ASIACRYPT 2024*, volume 15484 of *LNCS*, pages 463–493. Springer, 2024.
8. C. Baum, L. Braun, C. D. de Saint Guilhem, M. Klooß, C. Majenz, S. Mukherjee, S. Ramacher, C. Rechberger, E. Orsini, L. Roy, et al. Faest: Algorithm specifications. 2023.
9. C. Baum, L. Braun, C. D. de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, and P. Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from vole-in-the-head. In *CRYPTO 2023*, volume 14085 of *LNCS*, pages 581–615. Springer, 2023.
10. M. Belenkiy, M. Chase, M. Kohlweiss, and A. Lysyanskaya. P-signatures and noninteractive anonymous credentials. In *TCC 2008*, volume 4948 of *LNCS*, pages 356–374. Springer, 2008.
11. M. Bellare, A. Boldyreva, A. Desai, and D. Pointcheval. Key-privacy in public-key encryption. In *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 566–582. Springer, 2001.
12. M. Bellare, D. Micciancio, and B. Warinschi. Foundations of group signatures: Formal definitions, simplified requirements, and a construction based on general assumptions. In *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 614–629. Springer, 2003.
13. M. Bellare, H. Shi, and C. Zhang. Foundations of group signatures: The case of dynamic groups. In *CT-RSA 2005*, volume 3376 of *LNCS*, pages 136–153. Springer, 2005.
14. S. Bettaieb, L. Bidoux, P. Gaborit, and M. Kulkarni. Modelings for generic pok and applications: Shorter SD and PKP based signatures. *IACR Cryptol. ePrint Arch.*, page 1668, 2024.
15. L. Bidoux, T. Feneuil, P. Gaborit, R. Neveu, and M. Rivain. Dual support decomposition in the head: Shorter signatures from rank SD and minrank. In *ASIACRYPT 2024*, volume 15485 of *LNCS*, pages 38–69. Springer, 2024.
16. J. Bootle, A. Cerulli, P. Chaidos, E. Ghadafi, and J. Groth. Foundations of fully dynamic group signatures. In *ACNS 2016*, volume 9696 of *LNCS*, pages 117–136. Springer, 2016.

17. J. Bootle, V. Lyubashevsky, N. K. Nguyen, and A. Sorniotti. A framework for practical anonymous credentials from lattices. In *CRYPTO 2023*, volume 14082 of *LNCS*, pages 384–417. Springer, 2023.
18. P. Branco and P. Mateus. A code-based linkable ring signature scheme. In *ProvSec 2018*, volume 11192 of *LNCS*, pages 203–219. Springer, 2018.
19. G. Brassard, D. Chaum, and C. Crépeau. Minimum disclosure proofs of knowledge. *J. Comput. Syst. Sci.*, 37(2):156–189, 1988.
20. J. Camenisch and A. Lysyanskaya. A signature scheme with efficient protocols. In *SCN 2002*, volume 2576 of *LNCS*, pages 268–289. Springer, 2002.
21. J. Camenisch and A. Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In M. K. Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 56–72. Springer, 2004.
22. J. Cathalo, B. Libert, and M. Yung. Group encryption: Non-interactive realization in the standard model. In *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 179–196. Springer, 2009.
23. D. Chaum and E. van Heyst. Group signatures. In *EUROCRYPT 1991*, volume 547 of *LNCS*, pages 257–265. Springer, 1991.
24. S. Cheng, K. Nguyen, and H. Wang. Policy-based signature scheme from lattices. *Des. Codes Cryptogr.*, 81(1):43–74, 2016.
25. J. H. Chiang, I. Damgård, W. R. Duro, S. Engan, S. Kolby, and P. Scholl. Post-quantum threshold ring signature applications from vole-in-the-head. In *ACM CCS 2025*, pages 4664–4678. ACM, 2025.
26. N. T. Courtois, M. Finiasz, and N. Sendrier. How to achieve a mceliece-based digital signature scheme. In C. Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 157–174. Springer, 2001.
27. H. Cui, H. Liu, D. Yan, K. Yang, Y. Yu, and K. Zhang. Resolved: Shorter signatures from regular syndrome decoding and vole-in-the-head. In *PKC 2024*, volume 14601 of *LNCS*, pages 229–258. Springer, 2024.
28. L. Dallot and D. Vergnaud. Provably secure code-based threshold ring signatures. In *IMACC 2009*, volume 5921 of *LNCS*, pages 222–235. Springer, 2009.
29. T. Debris-Alazard, N. Sendrier, and J. Tillich. Wave: A new family of trapdoor one-way preimage sampleable functions based on codes. In S. D. Galbraith and S. Moriai, editors, *ASIACRYPT 2019*, volume 11921 of *LNCS*, pages 21–51. Springer, 2019.
30. M. F. Ezerman, H. T. Lee, S. Ling, K. Nguyen, and H. Wang. A provably secure group signature scheme from code-based assumptions. In *ASIACRYPT 2015*, volume 9452 of *LNCS*, pages 260–285. Springer, 2015.
31. A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO 1986*, volume 263 of *LNCS*, pages 186–194. Springer, 1986.
32. C. Ganesh, C. Orlandi, M. Pancholi, A. Takahashi, and D. Tschudi. Fiat-shamir bulletproofs are non-malleable (in the random oracle model). *J. Cryptol.*, 38(1):11, 2025.
33. O. Goldreich, S. Micali, and A. Wigderson. How to prove all np-statements in zero-knowledge, and a methodology of cryptographic protocol design. In *CRYPTO 1986*, volume 263 of *LNCS*, pages 171–185. Springer, 1986.
34. S. Goldwasser and S. Micali. Probabilistic encryption and how to play mental poker keeping secret all partial information. In *STOC 1982*, pages 365–377. ACM, 1982.
35. S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.*, 18(1):186–208, 1989.

36. S. Goldwasser, S. Micali, and R. L. Rivest. A "paradoxical" solution to the signature problem (extended abstract). In *FOCS 1984*, pages 441–448. IEEE Computer Society, 1984.
37. J. Groth, R. Ostrovsky, and A. Sahai. Perfect non-interactive zero knowledge for NP. In *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 339–358. Springer, 2006.
38. M. Izabachène, D. Pointcheval, and D. Vergnaud. Mediated traceable anonymous encryption. In *LATINCRYPT 2010*, volume 6212 of *LNCS*, pages 40–60. Springer, 2010.
39. A. Jain, S. Krenn, K. Pietrzak, and A. Tentes. Commitments and efficient zero-knowledge proofs from learning parity with noise. In *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 663–680. Springer, 2012.
40. C. Jeudy, A. Roux-Langlois, and O. Sanders. Lattice signature with efficient protocols, application to anonymous credentials. In *CRYPTO 2023*, volume 14082 of *LNCS*, pages 351–383. Springer, 2023.
41. A. Kiayias, Y. Tsiounis, and M. Yung. Traceable signatures. In *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 571–589. Springer, 2004.
42. A. Kiayias, Y. Tsiounis, and M. Yung. Group encryption. In *ASIACRYPT 2007*, volume 4833 of *LNCS*, pages 181–199. Springer, 2007.
43. E. Kiltz. Chosen-ciphertext security from tag-based encryption. In *TCC 2006*, volume 3876 of *LNCS*, pages 581–600. Springer, 2006.
44. E. Kiltz, J. Pan, and H. Wee. Structure-preserving signatures from standard assumptions, revisited. In *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 275–295. Springer, 2015.
45. B. Libert, S. Ling, F. Mouhartem, K. Nguyen, and H. Wang. Signature schemes with efficient protocols and dynamic group signatures from lattice assumptions. In *ASIACRYPT 2016*, volume 10032 of *LNCS*, pages 373–403, 2016.
46. B. Libert, S. Ling, F. Mouhartem, K. Nguyen, and H. Wang. Zero-knowledge arguments for matrix-vector relations and lattice-based group encryption. *Theor. Comput. Sci.*, 759:72–97, 2019.
47. B. Libert, K. Nguyen, T. Peters, and M. Yung. Bifurcated signatures: Folding the accountability vs. anonymity dilemma into a single private signing scheme. In *EUROCRYPT 2021*, volume 12698 of *LNCS*, pages 521–552. Springer, 2021.
48. B. Libert, M. Yung, M. Joye, and T. Peters. Traceable group encryption. In *PKC 2014*, volume 8383 of *LNCS*, pages 592–610. Springer, 2014.
49. S. Ling, K. Nguyen, D. H. Phan, K. H. Tang, H. Wang, and Y. Xu. Fully dynamic attribute-based signatures for circuits from codes. In *PKC 2024*, volume 14601 of *LNCS*, pages 37–73. Springer, 2024.
50. H. Liu, X. Wang, K. Yang, and Y. Yu. The hardness of LPN over any integer ring and field for PCG applications. *IACR Cryptol. ePrint Arch.*, page 712, 2022.
51. H. Liu, X. Wang, K. Yang, and Y. Yu. The hardness of LPN over any integer ring and field for PCG applications. In *EUROCRYPT 2024*, volume 14656 of *LNCS*, pages 149–179. Springer, 2024.
52. H. K. Maji, M. Prabhakaran, and M. Rosulek. Attribute-based signatures. In *CT-RSA 2011*, volume 6558 of *LNCS*, pages 376–392. Springer, 2011.
53. R. J. McEliece. A public-key cryptosystem based on algebraic coding theory. *Coding Thv*, 4244:114–116, 1978.
54. C. A. Melchor, P. Cayrel, and P. Gaborit. A new efficient threshold ring signature scheme based on coding theory. In *PQCrypto 2008*, volume 5299 of *LNCS*, pages 1–16. Springer, 2008.

55. C. A. Melchor, P. Cayrel, P. Gaborit, and F. Laguillaumie. A new efficient threshold ring signature scheme based on coding theory. *IEEE Trans. Inf. Theory*, 57(7):4833–4842, 2011.
56. K. Morozov and T. Takagi. Zero-knowledge protocols for the mceliece encryption. In *ACISP 2012*, volume 7372 of *LNCS*, pages 180–193. Springer, 2012.
57. M. Naor and M. Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *STOC 1990*, pages 427–437. ACM, 1990.
58. K. Nguyen, F. Guo, W. Susilo, and G. Yang. Multimodal private signatures. In *CRYPTO 2022*, volume 13508 of *LNCS*, pages 792–822. Springer, 2022.
59. K. Nguyen, R. Safavi-Naini, W. Susilo, H. Wang, Y. Xu, and N. Zeng. Group encryption: Full dynamicity, message filtering and code-based instantiation. In *PKC 2021*, volume 12711 of *LNCS*, pages 678–708. Springer, 2021.
60. K. Nguyen, R. Safavi-Naini, W. Susilo, H. Wang, Y. Xu, and N. Zeng. Group encryption: Full dynamicity, message filtering and code-based instantiation. *Theor. Comput. Sci.*, 1007:114678, 2024.
61. K. Nguyen, H. Tang, H. Wang, and N. Zeng. New code-based privacy-preserving cryptographic constructions. In *ASIACRYPT 2019*, volume 11922 of *LNCS*, pages 25–55. Springer, 2019.
62. R. Nojima, H. Imai, K. Kobara, and K. Morozov. Semantic security for the mceliece cryptosystem without random oracles. *Des. Codes Cryptogr.*, 49(1-3):289–305, 2008.
63. Y. Ouyang, D. Tang, and Y. Xu. Code-based zero-knowledge from vole-in-the-head and their applications: Simpler, faster, and smaller. In *ASIACRYPT 2024*, volume 15488 of *LNCS*, pages 436–470. Springer, 2024.
64. J. Pan, X. Chen, F. Zhang, and W. Susilo. Lattice-based group encryption with full dynamicity and message filtering policy. In *ASIACRYPT 2021*, volume 13093 of *LNCS*, pages 156–186. Springer, 2021.
65. C. Peikert and S. Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In *CRYPTO 2019*, volume 11692 of *LNCS*, pages 89–114. Springer, 2019.
66. H. Randriambololona. The syzygy distinguisher. In *EUROCRYPT 2025*, volume 15606 of *LNCS*, pages 324–354. Springer, 2025.
67. L. Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In *CRYPTO 2022*, pages 657–687, 2022.
68. A. Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *FOCS '99*, pages 543–553, 1999.
69. J. Stern. A new paradigm for public key identification. *IEEE Transactions on Information Theory*, 42(6):1757–1768, 1996.
70. M. Trolin and D. Wikström. Hierarchical group signatures. In *ICALP 2005*, volume 3580 of *LNCS*, pages 446–458. Springer, 2005.
71. Y. Xu, R. Safavi-Naini, K. Nguyen, and H. Wang. Traceable policy-based signatures and instantiation from lattices. *Inf. Sci.*, 607:1286–1310, 2022.
72. K. Yang, P. Sarkar, C. Weng, and X. Wang. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *ACM CCS 2021*. ACM, 2021.
73. R. Yang, M. H. Au, Z. Zhang, Q. Xu, Z. Yu, and W. Whyte. Efficient lattice-based zero-knowledge arguments with standard soundness: Construction and applications. In *CRYPTO 2019*, volume 11692 of *LNCS*, pages 147–175. Springer, 2019.

A Cryptographic Primitives

A.1 Digital Signatures

A digital signature [36] is a tuple of polynomial-time algorithms $(\mathsf{S.Kg}, \mathsf{S.Sign}, \mathsf{S.Ver})$ defined as follows.

$\mathsf{S.Kg}(1^\lambda)$**.** On input the security parameter λ, the key generation algorithm returns a key pair $(\mathsf{vk}, \mathsf{sk})$, where vk is the verification key and sk is the signing key.

$\mathsf{S.Sign}(\mathsf{sk}, m)$**.** On input a signing key sk and a message m to be signed, the signing algorithm returns the signature σ_m.

$\mathsf{S.Ver}(\mathsf{vk}, m, \sigma_m)$**.** On input the verification key, the message m and its signature σ_m, the verification algorithm returns 1 to denote the validity of the signature or 0 otherwise.

Correctness. Correctness of a signature scheme requires that a signature σ_m on any message m generated by any key pair $(\mathsf{vk}, \mathsf{sk})$ from the key generation algorithm can pass the signature verification.

Security. A signature scheme is said to be existentially unforgeable against adaptively chosen-message attacks if for any PPT adversary $\mathcal{A}$, one has the following advantage

$$\Pr\left[\mathsf{S.Ver}(\mathsf{vk}, m^*, \sigma_{m^*}) = 1 \wedge m^* \notin \mathcal{L}_{\mathsf{sig}} \;\middle|\; \begin{array}{r}(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{S.Kg}(1^\lambda),\\ (m^*, \sigma_{m^*}) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{sig}}}(\mathsf{vk})\end{array}\right] = \mathsf{negl}(\lambda),$$

which is negligible in the function of λ. Here, $\mathcal{O}_{\mathsf{sig}}$ denotes the signing oracle that input any message, it returns its signature computed with sk. $\mathcal{L}_{\mathsf{sig}}$ is the list recording all generated signed messages (m_i, σ_{m_i}) returned from the signing oracle $\mathcal{O}_{\mathsf{sig}}$.

A.2 Public-key Encryption

A public-key encryption scheme [34] is a tuple of polynomial-time algorithms $(\mathsf{E.Kg}, \mathsf{E.Enc}, \mathsf{E.Dec})$ defined as follows.

$\mathsf{E.Kg}(1^\lambda)$**.** On input the security parameter 1^λ, the key generation algorithm returns a key pair $(\mathsf{pk}, \mathsf{sk})$, where pk is the encryption key and sk is the decryption key.

$\mathsf{E.Enc}(\mathsf{pk}, m, r)$**.** On input an encryption key ek, a message m to be encrypted, and the randomness r, the encryption algorithm returns a ciphertext c.

$\mathsf{E.Dec}(\mathsf{sk}, \mathsf{c})$**.** On input the decryption key sk and the ciphertext c, the decryption algorithm returns the decryption result or $\perp$ (failure).

CORRECTNESS. Correctness of a public-key encryption scheme requires that a ciphertext c that is encryption on any message m with any randomness r and any encryption key (pk) generated from the key generation algorithm can be correctly decrypted. Namely, we have

$$\Pr\left[\mathsf{E.Dec}(\mathsf{sk},\mathsf{c}) = m \;\middle|\; \begin{array}{c}(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{E.Kg}(1^\lambda),\\ \mathsf{c} \leftarrow \mathsf{E.Enc}(\mathsf{pk}, m, r)\end{array}\right] = 1$$

IND-CCA2. A public key encryption scheme is said to be indistinguishable against adaptively chosen-ciphertext attacks if for any PPT adversary $\mathcal{A}$, one has the following advantage

$$\Pr\left[(b'=b) \wedge (\mathsf{c}^* \notin \mathcal{L}_{\mathsf{dec}}) \;\middle|\; \begin{array}{r}(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{E.Kg}(1^\lambda),\\ (m_0,m_1) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{dec}}}(\mathsf{pk}),\\ b \leftarrow \{0,1\}, r \leftarrow R,\\ \mathsf{c}^* \leftarrow \mathsf{E.Enc}(\mathsf{pk}, m_b, r),\\ b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{dec}}}(\mathsf{pk},\mathsf{c}^*).\end{array}\right] = \frac{1}{2} + \mathsf{negl}(\lambda),$$

where $\mathsf{negl}(\lambda)$ is negligible in the function of λ. Here, $\mathcal{O}_{\mathsf{dec}}$ denotes the decryption oracle that input any ciphertext, it returns its decryption result computed with sk. $\mathcal{L}_{\mathsf{dec}}$ is the list recording all submitted ciphertexts to the decryption oracle $\mathcal{O}_{\mathsf{dec}}$.

IK-CCA2. A public key encryption scheme is said to be key private against adaptively chosen-ciphertext attacks [11] if for any PPT adversary $\mathcal{A}$, one has the following advantage

$$\Pr\left[(b'=b) \wedge (\mathsf{c}^* \notin \mathcal{L}_{\mathsf{dec}}^{\mathsf{sk}_0} \cup \mathcal{L}_{\mathsf{dec}}^{\mathsf{sk}_1}) \;\middle|\; \begin{array}{r}\{(\mathsf{pk}_0,\mathsf{sk}_0),(\mathsf{pk}_1,\mathsf{sk}_1)\} \leftarrow \mathsf{E.Kg}(1^\lambda),\\ m \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{dec}}^{\mathsf{sk}_0},\mathcal{O}_{\mathsf{dec}}^{\mathsf{sk}_1}}(\mathsf{pk}_0,\mathsf{pk}_1),\\ b \leftarrow \{0,1\}, r \leftarrow R,\\ \mathsf{c}^* \leftarrow \mathsf{E.Enc}(\mathsf{pk}_\mathsf{b}, m, r),\\ b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{dec}}^{\mathsf{sk}_0},\mathcal{O}_{\mathsf{dec}}^{\mathsf{sk}_1}}(\mathsf{pk}_0,\mathsf{pk}_1,\mathsf{c}^*).\end{array}\right] = \frac{1}{2} + \mathsf{negl}(\lambda),$$

where $\mathsf{negl}(\lambda)$ is negligible in the function of λ. Here, $\mathcal{O}_{\mathsf{dec}}^{\mathsf{sk}}$ denotes the decryption oracle that input any ciphertext, it returns its decryption result computed with sk. $\mathcal{L}_{\mathsf{dec}}^{\mathsf{sk}}$ is the list recording all submitted ciphertexts to the decryption oracle $\mathcal{O}_{\mathsf{dec}}^{\mathsf{sk}}$.

A.3 Commitment Schemes

Definition 7. *A commitment scheme* $\mathcal{COM} = (\mathsf{Setup},\ \mathsf{Com},\ \mathsf{Open})$ *is a triple of algorithms defined as follows:*

$\mathsf{C.Setup}(1^\lambda)$. *On input the security parameter* 1^λ*, it returns system parameters* $\mathsf{pp_C}$.

$\mathsf{C.Com}(\mathsf{pp_C}, m)$. *On input* $\mathsf{pp_C}$ *and a message* m*, this probabilistic algorithm samples a random string* $\mathbf{r}$ *and computes commitment* C*. It then returns* C *and* $(m, \mathbf{r})$.

$\mathsf{C.Open}(\mathsf{pp_C}, C, m, \mathbf{r})$. *On input* $\mathsf{pp_C}$*,* C*, a message* m*, and randomness* $\mathbf{r}$*, this deterministic algorithm outputs* 1 *if* $C = \mathsf{C.Com}(\mathsf{pp_C}, m, \mathbf{r})$ *or* 0 *otherwise.*

We require a commitment scheme $\mathcal{COM}$ to satisfy the following correctness, hiding, and binding properties.

Correctness. We say that a commitment scheme $\mathcal{COM}$ is correct if for all $\lambda \in \mathbb{N}$, all m, we have that

$$\mathsf{Pr}\left[\mathsf{C.Open}(\mathsf{pp_C}, C, m, \mathbf{r}) = 1) \;\middle|\; \begin{array}{l} \mathsf{pp_C} \leftarrow \mathsf{C.Setup}(1^\lambda), \\ (C, (m, \mathbf{r})) \leftarrow \mathsf{C.Com}(\mathsf{pp_C}, m). \end{array}\right] = 1.$$

Statistical/Computational hiding. For any $\mathsf{pp_C} \leftarrow \mathsf{C.Setup}(1^\lambda)$ and for any m_0, m_1, the distributions of $\{\mathsf{C.Com}(\mathsf{pp_C}, m_0)\}$ and $\{\mathsf{C.Com}(\mathsf{pp_C}, m_1)\}$ are statistically/computationally indistinguishable.

Computational binding. For any PPT adversary $\mathcal{A}$,

$$\mathsf{Pr}\left[\begin{array}{l} \mathsf{C.Com}(m_0, \mathbf{r}_0) = \mathsf{C.Com}(m_1, \mathbf{r}_1) \\ \wedge\ (m_0 \neq m_1) \end{array} \;\middle|\; \begin{array}{l} \mathsf{pp_C} \leftarrow \mathsf{C.Setup}(1^\lambda), \\ (m_0, \mathbf{r}_0;\ m_1, \mathbf{r}_1) \leftarrow \mathcal{A}(\mathsf{pp_C}). \end{array}\right] = \mathsf{negl}(\lambda).$$

A.4 Dual-Mode NIZKs

Here, we recall the definitions and security properties of dual-mode NIZK systems. Our presentation follows [47].

A dual-mode NIZK system [37,65] is a NIZK that has two modes of operation, whose corresponding common reference strings are computationally indistinguishable. In the hiding mode, the system provides statistical ZK and computational soundness. In the binding mode, it offers computational ZK and statistical soundness. In the latter mode, when the common reference string contains an extraction trapdoor, the system can additionally satisfy statistical extractability.

Definition 8. *A dual-mode NIZK for NP-relation* $\mathcal{R}$ *is a tuple of algorithms* $\mathcal{NIZK} = (\mathsf{ZK.Setup}, \mathsf{ZK.ExtSetup}, \mathsf{ZK.Prove}, \mathsf{ZK.Ver}, \mathsf{ZK.Sim}, \mathsf{ZK.Extr})$*, defined as follows.*

$\mathsf{ZK.Setup}(1^\lambda) \to (\mathsf{crs}, \tau_{\mathsf{sim}})$. *On input* 1^λ*, outputs a common reference string* crs *and a simulation trapdoor* τ_{sim}.

$\mathsf{ZK.ExtSetup}(1^\lambda) \to (\mathsf{crs}, \tau_{\mathsf{ext}})$. *On input* 1^λ*, outputs a common reference string* crs *and an extraction trapdoor* τ_{ext}.

$\mathsf{ZK.Prove}(\mathsf{crs}, x, w) \to \pi$. *On input* crs*, a statement* x *and a witness* w*, outputs a proof/argument* π.

$\mathsf{ZK.Ver}(\mathsf{crs}, x, \pi) \to \{0,1\}$. *On input* crs*, a statement* x *and a purported proof or argument* π*, outputs either* 1 *or* 0*.*

$\mathsf{ZK.Sim}(\mathsf{crs}, \tau_{\mathsf{sim}}, x) \to \{\pi^\star, \perp\}$. *On input* crs*, a simulation trapdoor* τ_{sim} *and statement* x*, outputs a simulated argument* $\pi^\star$ *or symbol* $\perp$ *indicating failure.*

$\mathsf{ZK.Extr}(\mathsf{crs}, \tau_{\mathsf{ext}}, x, \pi)$. *On input* crs*, an extraction trapdoor* τ_{ext}*, a statement* x *and a proof* π*, outputs a witness* w*.*

CRS INDISTINGUISHABILITY. The distributions of the common reference strings generated via $\mathsf{ZK.Setup}(1^\lambda)$ and $\mathsf{ZK.ExtSetup}(1^\lambda)$ are computationally indistinguishable.

PERFECT COMPLETENESS IN BOTH MODES. For any $(x, w) \in \mathcal{R}$, we have

$$\Pr\left[\mathsf{ZK.Ver}(\mathsf{crs}, x, \pi) = 1 \;\middle|\; \begin{array}{c}(\mathsf{crs}, \tau_{\mathsf{sim}}) \leftarrow \mathsf{ZK.Setup}(1^\lambda), \\ \pi \leftarrow \mathsf{ZK.Prove}(\mathsf{crs}, x, w)\end{array}\right] = 1,$$

$$\Pr\left[\mathsf{ZK.Ver}(\mathsf{crs}, x, \pi) = 1 \;\middle|\; \begin{array}{c}(\mathsf{crs}, \tau_{\mathsf{ext}}) \leftarrow \mathsf{ZK.ExtSetup}(1^\lambda), \\ \pi \leftarrow \mathsf{ZK.Prove}(\mathsf{crs}, x, w)\end{array}\right] = 1,$$

STATISTICAL SOUNDNESS IN BINDING MODE. For every adversary $\mathcal{A}$,

$$\Pr\left[\mathsf{ZK.Ver}(\mathsf{crs}, x, \pi) = 1 \wedge x \notin \mathcal{L} \;\middle|\; \begin{array}{c}(\mathsf{crs}, \tau_{\mathsf{ext}}) \leftarrow \mathsf{ZK.ExtSetup}(1^\lambda), \\ (x, \pi) \leftarrow \mathcal{A}(\mathsf{crs})\end{array}\right] = \mathsf{negl}(\lambda).$$

STATISTICAL EXTRACTABILITY IN BINDING MODE. For every adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{c}\mathsf{ZK.Ver}(\mathsf{crs}, x, \pi) = 1 \quad \wedge \\ \big(x, \mathsf{ZK.Extr}(\mathsf{crs}, \tau_{\mathsf{ext}}, x, \pi)\big) \notin \mathcal{R}\end{array} \;\middle|\; \begin{array}{c}(\mathsf{crs}, \tau_{\mathsf{ext}}) \leftarrow \mathsf{ZK.ExtSetup}(1^\lambda), \\ (x, \pi) \leftarrow \mathcal{A}(\mathsf{crs})\end{array}\right] = \mathsf{negl}(\lambda).$$

STATISTICAL ZERO-KNOWLEDGE IN HIDING MODE. This property requires that, for any $(\mathsf{crs}, \tau_{\mathsf{sim}}) \leftarrow \mathsf{ZK.Setup}(1^\lambda)$, and for all $(x, w) \in \mathcal{R}$, the distributions $\{\mathsf{ZK.Prove}(\mathsf{crs}, x, w)\}$ and $\{\mathsf{ZK.Sim}(\mathsf{crs}, \tau_{\mathsf{sim}}, x)\}$ are statistically indistinguishable.

B Some Background on Code-Based Cryptography

Notations. Denote $\mathbb{Z}^+$ as the set of all positive integers. The Hamming weight of a binary vector $\mathbf{x} \in \{0,1\}^n$ is denoted by $wt(\mathbf{x})$. For $n, \omega \in \mathbb{Z}^+$, let $\mathsf{B}(n, \omega)$ represent the set that contains all binary vectors of length n with Hamming weight ω. All vectors are column vectors. For $\mathbf{x} \in \mathbb{Z}^{n_1}$ and $\mathbf{y} \in \mathbb{Z}^{n_2}$, we use $(\mathbf{x} \| \mathbf{y}) \in \mathbb{Z}^{n_1+n_2}$ to denote the column concatenation of these two vectors and $[n_1, n_2]$ to denote the set $\{n_1, n_1+1, \ldots, n_2\}$.

For an integer $X \in [0, 2^L - 1]$, we use $(x_{L-1}, \ldots, x_0)_2$ to denote its binary representation and write $X = (x_{L-1}, \ldots, x_0)_2$ if no ambiguity is caused. Denote $\mathsf{bin}(X) = (x_{L-1}, \ldots, x_0)^\top \in \{0,1\}^L$ to represent the vector corresponding to X. In the following, let $k, c \in \mathbb{Z}^+$ and $c \mid k$.

$\mathsf{Regular}(k,c)$ is the set of all vectors $\mathbf{w} = (\mathbf{w}_0 \| \cdots \| \mathbf{w}_{k/c-1}) \in \{0,1\}^{2^c \cdot k/c}$, where each $\mathbf{w}_i \in \mathsf{B}(2^c, 1)$. Such a vector $\mathbf{w}$ is known as a regular word.

$\mathsf{re} : \{0,1\}^c \to \{0,1\}^{2^c}$ maps $\mathbf{x} = (x_0, \ldots, x_{c-1})^\top$ to a vector in $\mathsf{B}(2^c, 1)$, in which the sole 1-entry is in the t-th position with $t = \sum_{i=0}^{c-1} (2^{c-1-i} \cdot x_i)$. It is easy to check that $\mathsf{re}(\mathbf{x}) \in \mathsf{Regular}(c, c)$.

$\mathsf{RE} : \{0,1\}^k \to \{0,1\}^{2^c \cdot k/c}$ maps $\mathbf{x} = (\mathbf{x}_0 \| \cdots \| \mathbf{x}_{k/c-1}) \in \{0,1\}^k$, where each $\mathbf{x}_i \in \{0,1\}^c$, to $(\mathsf{re}(\mathbf{x}_0) \| \cdots \| \mathsf{re}(\mathbf{x}_{k/c-1})) \in \{0,1\}^{2^c \cdot k/c}$. It is straightforward to verify that $\mathsf{RE}(\mathbf{x}) \in \mathsf{Regular}(k, c)$.

$2{-}\mathsf{Regular}(k,c)$ is the set of all vectors $\mathbf{x} = \mathbf{v} \oplus \mathbf{w}$ such that $\mathbf{v}, \mathbf{w} \in \mathsf{Regular}(k, c)$. Such a vector $\mathbf{x}$ is called as a 2-regular word.

We now recall the $2{-}\mathsf{RNSD}$ problem introduced by Augot, Finiasz, and Sendrier [5,6], and a collision resistant hash function that relies on the hardness of this problem.

Definition 9 (The $2{-}\mathsf{RNSD}_{n,k,c}$ Problem). *Let $n, k, c \in \mathbb{Z}^+$, $c \mid k$, and $m = 2^c \cdot k/c$. Given a uniformly random matrix $\mathbf{B} \in \{0,1\}^{n \times m}$, the $2{-}\mathsf{RNSD}_{n,k,c}$ problem asks to find a non-zero vector $\mathbf{x} \in 2{-}\mathsf{Regular}(k,c) \subseteq \{0,1\}^m$ such that $\mathbf{B} \cdot \mathbf{x} = \mathbf{0} \bmod 2$.*

Definition 10 (The AFS Hash Function). *Let $n, c \in \mathbb{Z}^+$ such that n is divisible by c, $m = 2 \cdot 2^c \cdot n/c$. Let $\mathcal{H} = \{h_{\mathbf{B}} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n\}$ be a function family. Here $\mathbf{B} = [\mathbf{B}_0 | \mathbf{B}_1] \in \{0,1\}^{n \times m}$, where each $\mathbf{B}_i \in \{0,1\}^{n \times m/2}$, $h_{\mathbf{B}}$ is defined as $\mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{x}_0) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{x}_1)$ for $(\mathbf{x}_0, \mathbf{x}_1) \in \{0,1\}^n \times \{0,1\}^n$.*

The AFS hash function is collision resistant if the $2{-}\mathsf{RNSD}_{n,2n,c}$ problem is hard [61].

B.1 An updatable Code-Based Merkle Tree Accumulator

Let us now recall an updatable code-based Merkle tree accumulator suggested in [61,59] that uses the above AFS hash function. The details are as follows.

$\mathsf{TSetup}(1^\lambda)$**.** Given the security parameter 1^λ, this algorithm chooses $n = \mathcal{O}(\lambda)$ and $c = \mathcal{O}(1)$ with $c \mid n$. Set $m = 2^{c+1} \cdot n/c$. It then samples a random matrix $\mathbf{B} \in \{0,1\}^{n \times m}$ and returns public parameters $\mathsf{pp} = \{n, c, m, \mathbf{B}\}$.

$\mathsf{TAccu}_{\mathbf{B}}\big(R = \{\mathbf{d}_0, \ldots, \mathbf{d}_{N-1}\} \subseteq (\{0,1\}^n)^N\big)$**.** Assume $N = 2^\ell = \mathsf{poly}(\lambda)$ for $\ell \in \mathbb{Z}^+$ without loss of generality. This algorithm proceeds as follows. It first rewrites $\mathbf{d}_j$ as $\mathbf{u}_{0,j}$, and treats them as the leaves [4] of a binary tree of height ℓ. Next, for $k \in [1, \ell]$ and $i \in [0, 2^{\ell-k} - 1]$, compute $\mathbf{u}_{k,i} = h_{\mathbf{B}}(\mathbf{u}_{k-1,2i}, \mathbf{u}_{k-1,2i+1})$. Output the root value $\mathbf{u} = \mathbf{u}_{\ell,0}$.

$\mathsf{TWitGen}_{\mathbf{B}}(R, \mathbf{d})$**.** This algorithm generates a witness w to demonstrate that $\mathbf{d} \in R$. It returns $\perp$ if $\mathbf{d} \notin R$. Otherwise, it outputs w as follows.

- First, find the index j such that $\mathbf{d} = \mathbf{u}_{0,j}$. Denote $j = (j_{\ell-1}, \ldots, j_0)_2$.

[4] Leaves are on the level 0 while the root is on the level ℓ.

– Consider the path from $\mathbf{u}_{0,j}$ to the root $\mathbf{u}_{\ell,0}$. The witness w contains all the siblings of the nodes on the path together with $\mathsf{bin}(j)$. Denote $w = (\mathsf{bin}(j), (\mathbf{w}_0, \ldots, \mathbf{w}_{\ell-1})) \in \{0,1\}^\ell \times (\{0,1\}^n)^\ell$.

$\mathsf{TVerify}_{\mathbf{B}}(\mathbf{u}, \mathbf{d}, w)$**.** Let w be as above. This algorithm first computes $\mathbf{v}_0, \ldots, \mathbf{v}_\ell$ as follows. Set $\mathbf{v}_0 = \mathbf{d}$. For $i \in [1, \ell]$, it computes

$$\mathbf{v}_i = \overline{j}_{i-1} \cdot h_{\mathbf{B}}(\mathbf{v}_{i-1}, \mathbf{w}_{i-1}) \oplus j_{i-1} \cdot h_{\mathbf{B}}(\mathbf{w}_{i-1}, \mathbf{v}_{i-1}).$$

Next, it returns 1 if and only if $\mathbf{v}_\ell = \mathbf{u}$.

$\mathsf{TUpdate}_{\mathbf{B}}(j, \mathbf{d}^*)$**.** This algorithm updates the j-th leaf node to a new value $\mathbf{d}^*$ without reconstructing the whole tree. Let $\mathbf{d}_j$ be the current value on the j-th leaf node. It runs the algorithm $\mathsf{TWitGen}_{\mathbf{B}}(R, \mathbf{d}_j)$, obtaining a witness of the form $w = (\mathsf{bin}(j), \mathbf{w}_0, \ldots, \mathbf{w}_{\ell-1})$. Next, it sets $\mathbf{v}_0 = \mathbf{d}^*$ and computes $\mathbf{v}_1, \ldots, \mathbf{v}_\ell$ as in the algorithm $\mathsf{TVerify}_{\mathbf{B}}$. Finally, for $k \in [0, \ell]$, update $\mathbf{u}_{k, \lfloor \frac{j}{2^k} \rfloor}$ to $\mathbf{v}_k$. Specifically, the root value $\mathbf{u}_{\ell,0}$ is updated to $\mathbf{v}_\ell$.

Definition 11. *An accumulator* $(\mathsf{TSetup}, \mathsf{TAccu}, \mathsf{TWitGen}, \mathsf{TVerify}, \mathsf{TUpdate})$ *is secure if for all honestly generated parameter* pp*, the probability of generating a tuple* $(R, \mathbf{d}^*, w^*)$ *such that* $(\mathsf{TVerify}(\mathsf{TAccu}(R), \mathbf{d}, ^*, w^*) = 1) \wedge (\mathbf{d}^* \notin R)$ *is negligible for all PPT adversaries.*

The security of the above accumulator relies on the collision resistance of the AFS hash function $h_{\mathbf{B}}$ [61], which in turn relies on the hardness of the $2-\mathsf{RNSD}_{n,2n,c}$ problem.

B.2 The Randomized McEliece Encryption Scheme

In this section, we first describe a randomized version of the McEliece [53] encryption scheme as proposed in [62], followed by the hard problems on which the scheme relies. A McEliece encryption scheme consists of the following polynomial-time algorithms.

$\mathsf{McSetup}(1^\lambda)$**.** On input 1^λ, this algorithm chooses parameters $n_e, k_e, t_e \in \mathsf{poly}(\lambda)$ for a binary $[n_e, k_e, 2t_e+1]$ Goppa code. Next, it specifies the plaintext space $\{0,1\}^{k_2}$ for some $k_2 < k_e$. Set $k_1 = k_e - k_2$.

$\mathsf{McKeygen}(n_e, k_e, t_e)$**.** This algorithm generates a public-secret key pair for the McEliece encryption scheme. It performs the following steps.

– Choose a random $[n_e, k_e, 2t_e+1]$ Goppa code and let $\mathbf{G}' \in \{0,1\}^{n_e \times k_e}$ be its generator matrix. Select a random invertible matrix $\mathbf{S} \in \{0,1\}^{k_e \times k_e}$ and a random permutation matrix $\mathbf{P} \in \{0,1\}^{n_e \times n_e}$, and then compute $\mathbf{G} = \mathbf{P} \cdot \mathbf{G}' \cdot \mathbf{S}$. Set $\mathsf{pk}_{\mathsf{Mc}} = \mathbf{G}$ and $\mathsf{sk}_{\mathsf{Mc}} = (\mathbf{P}, \mathbf{G}', \mathbf{S})$.

$\mathsf{McEnc}(\mathsf{pk}_{\mathsf{Mc}}, \mathbf{m})$**.** This algorithm samples a random vector $\mathbf{r} \in \{0,1\}^{k_1}$ and a random noise $\mathbf{e} \in \mathsf{B}(n_e, t_e)$ and outputs ciphertext $\mathbf{c} = \mathbf{G} \cdot \begin{pmatrix} \mathbf{r} \\ \mathbf{m} \end{pmatrix} \oplus \mathbf{e}$.

$\mathsf{McDec}(\mathsf{sk}_{\mathsf{Mc}}, \mathbf{c})$**.** This algorithm unveils the underlying message as follows.

– It runs the error-correcting procedure of the Goppa code on $\mathbf{P}^{-1} \cdot \mathbf{c} = \mathbf{G}' \cdot \mathbf{S} \cdot \begin{pmatrix} \mathbf{r} \\ \mathbf{m} \end{pmatrix} \oplus \mathbf{P}^{-1} \cdot \mathbf{e}$, obtaining $\mathbf{m}'' = \mathbf{S} \cdot \begin{pmatrix} \mathbf{r} \\ \mathbf{m} \end{pmatrix}$.
– Compute $\mathbf{m}' = \mathbf{S}^{-1} \cdot \mathbf{m}'' = \begin{pmatrix} \mathbf{r} \\ \mathbf{m} \end{pmatrix}$.
– Output $\mathbf{m}$.

The above scheme is correct and has pseudorandom ciphertexts if the decisional McEliece problem $\mathsf{DMcE}(n_e, k_e, t_e)$ and the decisional learning parity with (fixed Hamming weight) noise problem $\mathsf{DLPN}(n_e, k_1, \mathsf{B}(n_e, t_e))$ are hard. For completeness, these two problems are recalled below.

Definition 12 (The $\mathsf{DMcE}(n, k, t)$ Problem). *Let n, k, t be positive integers. Given a matrix $\mathbf{G} \in \{0,1\}^{n \times k}$, the $\mathsf{DMcE}(n, k, t)$ problem asks to determine whether $\mathbf{G}$ is a uniformly random matrix or is generated from the aforementioned $\mathsf{McKeygen}(n, k, t)$ algorithm.*

Definition 13 (The $\mathsf{DLPN}(n, k, \mathsf{B}(n, t))$ Problem). *Let $n, k, t \in \mathbb{Z}^+$. Given a pair $(\mathbf{A}, \mathbf{c}) \in \{0,1\}^{n \times k} \times \{0,1\}^n$, this problem asks to decide if the pair is uniformly chosen from the set $\{0,1\}^{n \times k} \times \{0,1\}^n$ or is obtained by sampling uniformly random $\mathbf{A} \in \{0,1\}^{n \times k}$, $\mathbf{r} \in \{0,1\}^k$, $\mathbf{e} \in \mathsf{B}(n, t)$ and outputting $(\mathbf{A}, \mathbf{A} \cdot \mathbf{r} \oplus \mathbf{e})$.*

A variant with regular noise. In this work, we consider a variant of McEliece encryption scheme such that the noise $\mathbf{e}$ is a regular word. More specifically, let k, c be two integers with $c|k$ such that $\frac{k}{c} \cdot 2^c = n_e$. Then the noise is computed as $\mathbf{e} = \mathsf{RE}(\mathbf{e}')$ with $\mathbf{e} \xleftarrow{\$} \mathbb{F}_2^k$. The security of this variant will then rely on the hardness of decisional LPN problem with regular noise. As shown by Liu et al. [Table 1][50,51], the usage of regular noise for LPN problems only reduces the security levels by a few bits (smaller than 10).

B.3 Universal Hash Function

When realizing the $\mathcal{F}_{\mathsf{sVOLE}}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}$ functionality, we utilize an $\mathbb{F}_p^l$-hiding and universal linear hash function for consistency check. We recall its definitions following [9,67].

Definition 14. *A linear ϵ-almost universal family of hashes is a family of matrices $\mathcal{H} \subseteq \mathbb{F}_q^{s \times l}$ such that for any nonzero $\mathbf{v} \in \mathbb{F}_q^l$, $\mathsf{Pr}_{\mathsf{H} \leftarrow \mathcal{H}}[\mathsf{H}\mathbf{v} = 0] \leq \epsilon$.*

Definition 15. *Let p and $q = p^k$ be prime powers. A matrix $\mathsf{H} \in \mathbb{F}_q^{s \times (l+h)}$ is $\mathbb{F}_p^l$-hiding if the distribution of $\mathbf{H} \cdot \mathbf{v}$ is independent from $\mathbf{v}_{[1,l]}$ for a uniformly random $\mathbf{v} \in \mathbb{F}_p^{l+h}$. Equivalently, if $\mathsf{H}' \in \mathbb{F}_p^{sk \times (l+h)}$ is interpreted as an $\mathbb{F}_p$-linear map, the the column space of H' must equal the column space of $\mathsf{H}_{[l+1,l+h]}$. A hash family $\mathcal{H} \subseteq \mathbb{F}_q^{s \times (l+h)}$ is $\mathbb{F}_p^l$-hiding if every $\mathsf{H} \in \mathcal{H}$ is $\mathbb{F}_p^l$-hiding.*

C Supporting Zero-Knowledge Protocols

In this section, we recall the technical description of the VOLEitH-based ZK protocol in [63] and explain how the code-based instantiation in Section 4 makes use of this ZK protocol for proving the defining relations of GEOT.

C.1 VOLEitH-based ZK Protocols

Vector Oblivious Linear Evaluation (VOLE) VOLE is a two-party functionality $\mathcal{F}_{\mathsf{VOLE}}^{l,\tau}$ between a sender and receiver. It allows the sender to obtain $\mathsf{M} \in \mathbb{F}_{p^\tau}^l$ and $\mathbf{u} \in \mathbb{F}_p^l$ and the receiver to obtain $\mathsf{K} \in \mathbb{F}_{p^\tau}^l$ and $\Delta \in \mathbb{F}_{p^\tau}$ such that $\mathsf{K} = \mathsf{M} + \Delta \cdot \mathbf{u}$. These VOLE correlations can be used to authenticate $\mathbf{u}$. We denote such authenticated values by $[\![\mathbf{u}]\!]$, indicating that the sender obtains $\mathbf{u}$ and M while the receiver obtains Δ and K. It is not hard to see that the sender cannot alter $\mathbf{u}$ to a different $\mathbf{u}'$ without guessing Δ correctly. It is also easy to verify that VOLE correlations are additively homomorphic. In particular, given public coefficients $c_0, \ldots, c_l \in \mathbb{F}_{p^\tau}$, two parties can locally compute $[\![y]\!] = \sum_{i=1}^{l} c_i \cdot [\![u_i]\!] + c_0$, where the sender computes $y := \sum_{i=1}^{l} c_i \cdot u_i + c_0$ and $\mathsf{M}_y = \sum_{i=1}^{l} c_i \cdot \mathsf{M}_{u_i}$, and the receiver computes $\mathsf{K}_y := \sum_{i=1}^{l} c_i \cdot \mathsf{K}_{u_i} + c_0 \cdot \Delta$.

There is an extension of VOLE called Vector Oblivious Polynomial Evaluation (VOPE), introduced by Yang et al. [72]. Roughly speaking, VOPE correlations generalize VOLE correlations to high-degree polynomials. In a VOPE functionality, the sender gets $A_0, \ldots, A_d \in \mathbb{F}_{p^r}$ while the receiver gets $B \in \mathbb{F}_{p^r}$ and $\Delta \in \mathbb{F}_p$ such that $B = \sum_{i=0}^{d} A_i \cdot \Delta^i$.

VOLE-Based and VOPE-Based ZK Proofs VOLE and VOPE allow the design of ZK protocols proving linear equation or polynomial satisfiability. As shown in [72], one can utilize a VOPE functionality to prove a set of degree-d polynomials on totally l distinct variables with communication cost of $l + d$ field elements, which is independent of the number of multiplications to compute all polynomials. Let $f_1, \ldots, f_t$ be a set of l-variate degree-d polynomials over $\mathbb{F}_p$. For simplicity, we represent each polynomial in a degree-separated format, i.e., $f_i(X_1, \ldots, X_l) = \sum_{h=0}^{d} g_{i,h}(X_1, \ldots, X_l)$ such that all terms in $g_{i,h}$ have degree exactly h. A prover $\mathcal{P}$ wants to prove that $f_i(w_1, \ldots, w_l) = 0$ for $i \in [1, t]$ with $\mathbf{w} = (w_1, \ldots, w_l)^\top \in \mathbb{F}_p^l$. In more detail, suppose a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ obtain $[\![w_1]\!], \ldots, [\![w_l]\!]$ such that $\mathsf{K}_i = \mathsf{M}_i + w_i \cdot \Delta$. Then

$$\begin{aligned} B_i &= \sum_{h=0}^{d} g_{i,h}(\mathsf{K}_1, \ldots, \mathsf{K}_l) \cdot \Delta^{d-h} = \sum_{h=0}^{d} g_{i,h}(\mathsf{M}_1 + w_1 \cdot \Delta, \ldots, \mathsf{M}_l + w_l \cdot \Delta) \cdot \Delta^{d-h} \\ &= \underbrace{f(w_1, \ldots, w_n)}_{0 \text{ if } \mathcal{P} \text{ is honest}} \cdot \Delta^d + \underbrace{A_{i,0}}_{\text{known to } \mathcal{P}} + \underbrace{A_{i,1}}_{\text{known to } \mathcal{P}} \cdot \Delta + \cdots + \underbrace{A_{i,d-1}}_{\text{known to } \mathcal{P}} \cdot \Delta^{d-1} \end{aligned} \tag{5}$$

Therefore, checking that $f(w_1, \ldots, w_l) = 0$ can be converted to checking the degree-$(d-1)$ VOPE relation $B_i = A_{i,0} + A_{i,1} \cdot \Delta + \cdots + A_{i,d-1} \cdot \Delta^{d-1}$ in (5).

Moreover, we can use random linear combination to reduce checking t VOPE relations (corresponding to t polynomials equation) to checking a single VOPE relation.

Functionality $\mathcal{F}_{\mathsf{sVOLE}}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}$

The functionality interacts with a sender $\mathcal{P}$, a receiver $\mathcal{V}$ and an adversary $\mathcal{A}$. It is parametrized by integers l; a prime p and an integer q such that $q = p^k$; and an $[n_\mathcal{C}, k_\mathcal{C}, d_\mathcal{C}]_p$ linear code $\mathcal{C}$ over $\mathbb{F}_p$ with a generator matrix $\mathbf{G}_\mathcal{C} \in \mathbb{F}_p^{k_\mathcal{C} \times n_\mathcal{C}}$.

1. Upon receiving (`init`) from the prover $\mathcal{P}$ and the verifier $\mathcal{V}$, sample $\mathbf{U} \leftarrow \mathbb{F}_p^{l \times k_C}$, $\mathbf{V} \leftarrow \mathbb{F}_q^{l \times n_C}$ and $\mathbf{\Delta} \leftarrow S_\Delta \subseteq \mathbb{F}_q^{n_C}$ and set $\mathbf{Q} := \mathbf{V} + \mathbf{U}\mathbf{G}_C\mathsf{diag}(\mathbf{\Delta})$.
   - If $\mathcal{P}$ is corrupt, receive $\mathbf{U}, \mathbf{V}$ from the adversary $\mathcal{A}$, and recompute $\mathbf{Q}$ as above.
   - If $\mathcal{V}$ is corrupt, receive $\mathbf{\Delta}, \mathbf{Q}$ from the adversary $\mathcal{A}$, and compute $\mathbf{V} := \mathbf{Q} - \mathbf{U}\mathbf{G}_C\mathsf{diag}(\mathbf{\Delta})$.
   - Send $(\mathbf{U}, \mathbf{V})$ to $\mathcal{P}$.
   - If $\mathcal{P}$ is corrupt, receive a leakage query $L \in \mathcal{L}$ from $\mathcal{A}$.
2. Upon receiving (`get`) from $\mathcal{V}$, if $\mathbf{\Delta} \notin L$, send (`check-failed`) to $\mathcal{V}$ and abort. Otherwise, send $(\mathbf{\Delta}, \mathbf{Q})$ to $\mathcal{V}$.

VOLE-in-the-Head Technique The drawback of VOLE-based proof systems is being designated-verifier (DV) since $\mathcal{V}$ has to know its part of VOLE correlations so as to verify proofs. Baum et al. [9] introduced a technique called VOLE-in-the-head (VOLEitH) that transforms the above DVZK proofs to public-coin protocols, which in turn can be made non-interactive via the Fiat-Shamir heuristic [31]. At a high level, Baum et al. [9] employed a VOLE functionality $\mathcal{F}_{\mathsf{sVOLE}}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}$ that allows $\mathcal{P}$ to first generate its values K_i, u_i independent of Δ, M_i and to generate Δ, M_i after all proof messages have been "committed". This VOLE functionality can then be realized via all-but-one oblivious transfer, which is further realized by GGM-based vector commitments (VC).

VOLEitH-based ZK proof for Degree-d Constraints We describe the protocol $\Pi_{\mathsf{dD-Rep}}^t$ of [63], which proves the following relation

$$\mathcal{R}_{d,t} = \{((f_1, \ldots, f_t), \mathbf{w}) \in \left(\mathbb{F}_{p^k}[X_1, \ldots, X_l]_{\leq d}\right)^t \times \mathbb{F}_p^l : \forall i \in \{1, \ldots, t\}, f_i(\mathbf{w}) = 0\}. \tag{6}$$

The goal of prover $\mathcal{P}$ is proving the knowledge of $\mathbf{w} \in \mathbb{F}_q^l$ such that $f_i(\mathbf{w}) = 0$ for $i \in [1, t]$, where $\{f_1, \ldots, f_t\}$ are polynomials of l variables and of degree at most d. At a high level, the protocol works as follows:

- In the commit phase, both parties invoke the functionality $\mathcal{F}_{\mathsf{sVOLE}}$. The prover obtains $\mathbf{u} \in \mathbb{F}_p^{l+(d-1)r\delta}$ and $\mathbf{V}$, while the verifer $\mathcal{V}$ will obtain $\mathbf{Q}$ and $\mathbf{\Delta}$ satisfying $\mathbf{Q} = \mathbf{V} + \mathbf{u} \cdot \mathbf{G}_\mathcal{C}\mathrm{diag}(\mathbf{\Delta})$ (after receiving messages from $\mathcal{P}$ in the prove phase). Next, $\mathcal{P}$ commits to its witness $\mathbf{w}$ by sending $\mathbf{d} = \mathbf{w} - \mathbf{u}_{[1,l]}$ to $\mathcal{V}$.

– In the challenge phase, $\mathcal{V}$ samples uniformly random and independent coefficients $\chi_1, \ldots, \chi_t \in \mathbb{F}_{q^\delta}$ and sends them to $\mathcal{P}$. These coefficients will be used for the random linear combination performed by $\mathcal{P}$ in the following phase.
– In the prove phase, $\mathcal{P}$ basically reduces the task of proving $f_i(\mathbf{w}) = 0$ for $i \in [1, t]$ into the task of proving the satisfiability of (5). In the process, both parties employ the remaining $(d-1)r\delta$ relations related to $\mathbf{u}_{[l+1,l+(d-1)r\delta]}$ to generate a single VOPE correlation to mask a random linear combination of the t equations.

Details of the generalization are given below.

Protocol $\Pi^t_{\mathsf{dD-Rep}}$

Parameters: Code $\mathcal{C}_{\mathrm{Rep}} = [\delta, 1, \delta]$ with $\mathcal{G}_c = (1, \ldots, 1) \in \mathbb{F}_q^{1\times\delta}$ and $q = p^r$. Assume there is one-to-one correspondence between elements in $\mathbb{F}_q$ and $[1, q]$. Define $S_\Delta = \mathbb{F}_q^\delta$.
Inputs: Polynomials $f_i = \sum_{h=0}^{d} f_{i,h} \in \mathbb{F}_{p^k}[X_1, \ldots, X_l]_{\le d}, i \in [t]$ with $k|(\delta r)$. $\mathcal{P}$ holds a witness $\mathbf{w} = (w_1, \ldots, w_l)^\top \in \mathbb{F}_p^l$ such that $f_i(w_1, \ldots, w_l) = 0$ for all $i \in [t]$.
Round 1. $\mathcal{P}$ performs the following steps:

1. Call the functionality $\mathcal{F}_{\mathsf{VOLE}}^{p,q,S_\Delta,\mathcal{C}_{\mathrm{Rep}},l+(d-1)\delta r,\mathcal{L}}$ and receive $\mathbf{u} \in \mathbb{F}_p^{l+(d-1)\delta r}$. $\mathcal{V}$ receives `done`.
2. Compute $\mathbf{d} = \mathbf{w} - \mathbf{u}_{[1,l]} \in \mathbb{F}_p^l$ and send $\mathbf{d}$ to $\mathcal{V}$.
3. For $i \in [l+1, l+(d-1)\delta r]$, embed the i-th element $u_i \in \mathbb{F}_p$ of $\mathbf{u}$ to $u_i \in \mathbb{F}_q^r$. For $i \in [l + (d-1)\delta r]$, lift the i-th row $\mathbf{v}_i \in \mathbb{F}_q^r$ of $\mathbf{V}$ into $\mathbf{v}_i \in \mathbb{F}_q^r$. For $i \in [l]$, also embed the i-th element w_i of witness $\mathbf{w}$ to $w_i \in \mathbb{F}_{q^\delta}$.

Round 2. $\mathcal{V}$ samples uniformly random $\chi_i \xleftarrow{\$} \mathbb{F}_{q^\delta}$, $i \in [t]$ and sends to $\mathcal{P}$.
Round 3. After receiving $\chi_1, \ldots, \chi_t$, $\mathcal{P}$ does the following:

1. For each $i \in [t]$, compute $A_{i,0}, A_{i,1}, \ldots, A_{i,d-1} \in \mathbb{F}_{q^\delta}$ such that

$$c_i(Y) = \sum_{h=0}^{d} \overline{f_{i,h}}(v_1 + w_1 Y, \ldots, v_l + w_l Y) Y^{d-h} = \overline{f}_i(w_1, \ldots, w_l) Y^d + \sum_{j=0}^{d-1} A_{i,j} Y^j,$$

   where $\overline{f_{i,h}} \in \mathbb{F}_{q^\delta}[X_1, \ldots, X_l]$ is the embedding of $f_{i,h} \in \mathbb{F}_{p^k}[X_1, \ldots, X_l]$.
2. **Generation of a VOPE correlation.**
   (a) For $j \in [d-1]$, compute

$$u_j^* = \sum_{i \in [\delta r]} u_{l+(j-1)\delta r+i} X^{i-1} \in \mathbb{F}_{q^\delta}, \quad v_j^* = \sum_{i \in [\delta r]} v_{l+(j-1)\delta r+i} X^{i-1} \in \mathbb{F}_{q^\delta},$$

   where $\mathbb{F}_{q^\delta} \cong \mathbb{F}_p[X]/F(X)$ with $F(X) \in \mathbb{F}_p[X]$ being an irreducible polynomial of degree δr.
   (b) Define $g_1(x) = v_1^* + u_1^* \cdot x$. For $i \in [1, d-2]$, compute $g_{i+1}(x) = g_i(x)(v_{i+1}^* + u_{i+1}^* \cdot x)$. Then $\mathcal{P}$ is able to compute the coefficients $A_0^*, \ldots, A_{d-1}^* \in \mathbb{F}_{q^r}$ such that $g_{d-1}(x) = \sum_{j=0}^{d-1} A_j^* \cdot x^j$.

3. For $j = 0, 1, \ldots, d-1$, compute $\tilde{a}_j = \sum_{i\in[t]} \chi_i \cdot A_{i,j} + A_j^*$ and send $\tilde{a}_j$ to $\mathcal{V}$.

Verification. After receiving all responses, $\mathcal{V}$ runs the following checks:

1. Call $\mathcal{F}_{\mathsf{VOLE}}^{p,q,S_\Delta,\mathcal{C}_{\mathrm{Rep}},l+(d-1)\delta r,\mathcal{L}}$ on input `get` and obtain $\mathbf{\Delta} \in S_\Delta, \mathbf{Q} \in \mathbb{F}_q^{(l+(d-1)\delta r)\times\delta}$ such that $\mathbf{Q} = \mathbf{V} + \mathbf{u}G_{\mathcal{C}}\mathsf{diag}(\mathbf{\Delta})$. Let $\mathbf{q}_i$ be the i-th row vector of $\mathbf{Q}$, for $i \in [l+1, l+(d-1)\delta r]$.
2. Compute $\mathbf{Q}^* = \mathbf{Q}_{[1,l]} + \mathbf{d} \cdot G_{\mathcal{C}} \cdot \mathsf{diag}(\mathbf{\Delta})$, which is supposed to be $\mathbf{V}_{[1,l]} + \mathbf{w} \cdot G_{\mathcal{C}} \cdot \mathsf{diag}(\mathbf{\Delta})$. Let $\mathbf{q}_1^*, \ldots, \mathbf{q}_l^* \in \mathbb{F}_q^\delta$ be the rows of $\mathbf{Q}^*$.
3. Lift $\mathbf{\Delta} \in \mathbb{F}_q^\delta$ into $\Delta \in \mathbb{F}_{q^\delta}$. Also, lift $\mathbf{q}_1^*, \ldots, \mathbf{q}_l^*, \mathbf{q}_{l+1}, \ldots, \mathbf{q}_{l+(d-1)\delta r} \in \mathbb{F}_q^\delta$ into $q_1^*, \ldots, q_l^*, q_{l+1}, \ldots, q_{l+(d-1)\delta r} \in \mathbb{F}_{q^\delta}$.
4. **Generation of a VOPE correlation.**
   (a) For $j \in [d-1]$, compute $q_{l+j}^* = \sum_{i\in[\delta r]} q_{l+(j-1)\delta r+i} X^{i-1} \in \mathbb{F}_{q^\delta}$, which should satisfy $q_{l+j}^* = v_j^* + u_j^* \cdot \Delta$.
   (b) Let $B_1^* = q_{l+1}^*$. Then for $i \in [d-2]$, compute $B_{i+1}^* = B_i^* \cdot q_{l+i+1}^*$. Define $B^* = B_{d-1}^*$. Then one can verify that $B^* = \sum_{j=0}^{d-1} A_j^* \Delta^j$.
5. For each $i \in [t]$, compute

$$c_i(\Delta) = \sum_{h=0}^{d} \overline{f_{i,h}}(q_1^*, \ldots, q_l^*) \cdot \Delta^{d-h}.$$

6. Compute $\tilde{c} = \sum_{i\in[t]} \chi_i \cdot c_i(\Delta) + B^*$ and check if $\tilde{c} = \sum_{j=0}^{d-1} \tilde{a}_j \cdot \Delta^j$.

Theorem 3 ([63]). *In the $\mathcal{F}_{\mathsf{sVOLE}}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}$-hybrid model [9], the protocol $\Pi_{\mathsf{dD-Rep}}^t$ proves the relation $\mathcal{R}_{d,t}$ defined in (6), is zero-knowledge against a semi-honest verifier and has soundness error upper bounded by $1/p^{r\delta} + d/|S_\Delta| = (d+1)/p^{r\delta}$.*

Communication cost. In $\Pi_{\mathsf{dD-Rep}}^t$, in addition to the cost of the sVOLE steps, $\mathcal{P}$ sends the initial commitment $\mathbf{d} \in \mathbb{F}_p^l$ and $\{\tilde{a}_i \in \mathbb{F}_{q^\delta}\}_{i\in[0,d-1]}$. Therefore, the total cost is summarized as follows:

$$\mathsf{Com}_{\Pi_{\mathsf{ID-Rep}}^t} = \mathsf{Com}_{\mathsf{sVOLE}} + l \cdot \log_2 p + d \cdot r \cdot \delta \cdot \log_2 p. \tag{7}$$

Additionally, the verifier sends t values in $\mathbb{F}_q^\delta$ but this can be removed via the Fiat-Shamir transform in the non-interactive setting and does not affect the final proof size. When instantiating the $\mathcal{F}_{\mathsf{sVOLE}}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}$ functionality with GGM-based VC (see [9, Section 3.1, Figure 3, Figure 4]), the cost of sVOLE steps is

$$\begin{aligned}\mathsf{Com}_{\mathsf{sVOLE}} = 2\lambda &+ (l + (d-1) \cdot r \cdot \delta + h) \cdot (\delta - 1) \cdot \log_2 p \\ &+ (s + s \cdot \delta) \cdot \log_2 p + (2\lambda + r \cdot \lambda) \cdot \delta.\end{aligned} \tag{8}$$

The process of the instantiation employs an $\mathbb{F}_p^l$-hiding and ϵ-universal hash function $\mathsf{H} \in \mathbb{F}_p^{s\times(l+(d-1)r\delta+h)}$ (see Appendix B.3).

Prover's Complexity. Following the analysis of [72], the prover's complexity is dominated by the computation in round 3, which involves operations over the

extension field $\mathbb{F}_{q^\delta}$. From $\Pi^t_{\mathsf{dD-Rep}}$, the prover's computation in round 3 involves: (1) computing the coefficients $A_{i,j}$ of the polynomials $c_i(Y) \in \mathbb{F}_{q^\delta}[X_1, \ldots, X_l]$; (2a) computing the field elements $u^\star_j, v^\star_j \in \mathbb{F}_{q^\delta}$, which takes $2d\delta r$ multiplications and $2d(\delta r - 1)$ additions; (2b) computing the coefficients of the polynomial $\prod_{i=1}^{d-1}(v^\star_i + u^\star_i \cdot x)$, which takes around $d^2/2$ multiplications and $d^2/2$ additions; and (3) computing the response $\tilde{a}_0, \ldots, \tilde{a}_j \in \mathbb{F}_{q^\delta}$, which takes td multiplications and td additions.

For step (1), the coefficients $A_{i,j}$ can be computed via *Lagrange interpolation*. Namely, since $c_i(Y)$ is a polynomial of degree d, we can evaluate $c_i(Y)$ at any fixed $d+1$ evaluation point then interpolate the coefficient. Thus, computing $A_{i,j}$ requires: (i) evaluating the polynomials $\overline{f_{i,h}}$ at $d+1$ interpolation points, where each evaluation costs at most hz multiplications and $z-1$ additions where z is the maximum number of monomial terms in the degree-h homogenous polynomials $\overline{f_{i,h}} = f_{i,h}$, and; (ii) reconstruct the coefficients via the Lagrange interpolation polynomials, which cost another $d(d+1)$ multiplications and d^2 additions if the Lagrange polynomials are precomputed. Therefore, the total cost of computing $A_{i,j}$ is at most $t(d+1)dz + td(d+1) = \mathcal{O}(td^2z)$ multiplications and $t(d+1)(z-1) + td^2 = \mathcal{O}(tdz)$ additions. In practice, the polynomials $f_{i,h}$ may have certain special structures that potentially speeds up the interpolation process, or allows computing the values $A_{i,j}$ without the need of interpolation at all.

VOLEitH-based NIZK from Fiat-Shamir transformation. As shown by Baum et al. [9], applying the Fiat-Shamir transformation to $\Pi^t_{\mathsf{dD-Rep}}$ (with the functionality $\mathcal{F}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}_{\mathsf{sVOLE}}$ instantiated by GGM-based VC) results in a non-interactive zero-knowledge argument of knowledge with a straightline knowledge extractor. Ganesh et al. [32] also showed that the resulting non-interactive argument is simulation-extractable in the programmable random oracle model. At a high level, simulation-extractability guarantees that extractability holds even when the adversary sees simulated proofs, and implies simulation-soundness [68]. This suffices to prove the security of our code-based instantiation.

VOLEitH-based ZK Proof for Regular Encoding We recall the technique of [63] to prove the regular encoding process within the VOLEitH paradigm. This technique will be crucial in the ZK proofs supporting the code-based GEOT scheme presented in the subsequent section.

Consider the encoding function $\mathsf{re} : \mathbb{F}_2^c \to \mathbb{F}_2^{2^c}$ in Appendix B. Observe that re can be seen as 2^c number of c-variate Boolean functions $f_1(\cdot), \ldots, f_{2^c}(\cdot)$. If we focus on the first output bit, then the truth table of the corresponding Boolean function $f_1(\cdot)$ is the unit vector $\mathbf{e}_1 \in \mathbb{F}_2^{2^c}$ with 1 in the first position. Through Lagrange interpolation, one can obtain $f_1(\cdot) \triangleq f_{(0,\ldots,0)}(X_1, \ldots, X_c) = \prod_{i=1}^{c}(1 + 0 + X_i)$. Similarly, for the j-th output bit, the truth table of $f_j(\cdot)$ is the unit vector $\mathbf{e}_j \in \mathbb{F}_2^{2^c}$, and the Boolean function is $f_j(\cdot) \triangleq f_{(j_1,\ldots,j_c)}(X_1, \ldots, X_c) = \prod_{i=1}^{c}(1 + j_i + X_i)$, where $(j_1, \ldots, j_c)^\top = \mathsf{bin}(j-1)$. In short, we can represent

the non-linear encoding process as degree-c relations

$$\mathsf{re}(x_1,\ldots,x_c) = (f_{(0,\ldots,0)}(x_1,\ldots,x_c),\ldots, \\ f_{(j_1,\ldots,j_c)}(x_1,\ldots,x_c),\ldots,f_{(1,\ldots,1)}(x_1,\ldots,x_c))^\top.$$

Now consider the encoding function $\mathsf{RE}:\mathbb{F}_2^k \to \mathbb{F}_2^m$, for $c|k$, $m = 2^c \cdot \frac{k}{c}$. Then by definition, for $\mathbf{x} \in \mathbb{F}_2^k$, we have that $\mathsf{RE}(\mathbf{x}) = \left(\mathsf{re}(\mathbf{x}_0),\ldots,\mathsf{re}\left(\mathbf{x}_{k/c-1}\right)\right)$. As such, we can write $\mathsf{RE}(\mathbf{x}) = (f_1(\mathbf{x}),\ldots,f_m(\mathbf{x}))^\top$, and $\deg(f_i) = c$ for $i \in [1,m]$ without explicitly describing the details of f_i. In fact, $f_j(X_1,\ldots,X_c) = f_{2^c+j}(X_1,\ldots,X_c) = \ldots = f_{(\frac{k}{c}-1)\cdot 2^c+j}(X_1,\ldots,X_c)$ for $j \in [1,2^c]$. Note that $f_j(\mathbf{x})$ only selects c inputs and ignores other inputs.

Therefore, to show that $\mathbf{z} = (z_1,\ldots,z_m)^\top \in \mathsf{Regular}(k,c)$ is indeed a regular encoding of $\mathbf{x} = (x_1,\ldots,x_k)^\top \in \mathbb{F}_2^k$, it suffices to show that $z_j = f_j(\mathbf{x})$ for all $j \in [1,m]$. Since these m constraints are degree-c relations, and thus can be proved in zero-knowledge using the protocol $\Pi^t_{\mathsf{dD}-\mathsf{Rep}}$.

C.2 Zero-Knowledge Arguments Supporting the Code-Based GEOT Scheme

In this section, we present the zero-knowledge protocol invoked by the sender who has encrypted a message $\mathbf{w}$ to a receiver id_j in the code-based GEOT scheme for circuits. In short, we need to prove a statement involving the conditions (3a)-(3e) in the Enc algorithm in Section 4.1, that are

- $\mathbf{c}_{w,0}$ and $\mathbf{c}_{w,1}$ are McEliece ciphertexts of message $\mathbf{w}$ satisfying $F_\tau(\mathbf{w}) = 1$;
- $\mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}$ are ciphertexts of $(1-b)\cdot\mathbf{d}_j$ satisfying $b = P_\tau(\mathsf{id}_j,\mathbf{w})$;
- The keys $\mathbf{G}_{j,0},\mathbf{G}_{j,1} \in \{0,1\}^{n_2\times k_2}$ are honestly computed to a hash value $\mathbf{d}'_j \in \{0,1\}^n$. In other words, $\mathbf{d}'_j = \mathsf{TAccu}_{\mathbf{B}}(\mathbf{G}_{j,0},\mathbf{G}_{j,1})$. Here, $\mathbf{G}_{j,0},\mathbf{G}_{j,1}$ and $\mathbf{d}'_j$ are all secret.
- The value $\mathbf{d}_j \in \{0,1\}^n$ is accumulated in the root $\mathbf{u}_\tau$, i.e., the prover knows $\mathbf{d}_j$ and a corresponding witness

$$w^{(j)} = ((j_{\ell-1},\ldots,j_0)^\top,\mathbf{w}_0,\ldots,\mathbf{w}_{\ell-1}) \in \{0,1\}^\ell \times (\{0,1\}^n)^\ell$$

  such that $\mathsf{TVerify}_{\mathbf{B}}(\mathbf{u}_\tau,\mathbf{d}_j,w^{(j)}) = 1$.
- The prover possesses user identifier $\mathsf{id}_j = (x_{j,0},\ldots,x_{j,n-1})^\top \in \{0,1\}^n$ such that

$$\begin{cases}\mathbf{d}_j = \mathbf{B}_0\cdot\mathsf{RE}(\mathbf{d}'_j)\oplus\mathbf{B}_1\cdot\mathsf{RE}(\mathsf{id}_j);\\ \mathbf{d}_j \neq \mathbf{0}^n.\end{cases}$$

Note that, the keys $(\mathbf{G}_{j,0},\mathbf{G}_{j,1})$ that encrypt $\mathbf{w}$ should be kept secret as well. We can summarize the above statements as follows

- The public input consists of τ, F_τ, P_τ and

$$\mathbf{G}_{\mathsf{oa},0},\mathbf{G}_{\mathsf{oa},1} \in \{0,1\}^{n_1\times k_1},\mathbf{c}_{w,0},\mathbf{c}_{w,1},\mathbf{u}_\tau \in \{0,1\}^n,\mathbf{c}_{\mathsf{oa},0},\mathbf{c}_{\mathsf{oa},1} \in \{0,1\}^{n_1}$$

– The prover's secret input consists of

$$\begin{cases} \mathbf{w} \in \{0,1\}^t,\ b,\ \mathsf{id}_j = (x_{j,0}, \ldots, x_{j,n-1}), \in \{0,1\}^n,\ \mathbf{d}_j \in \{0,1\}^n, \\ \mathbf{G}_{j,0}, \mathbf{G}_{j,1} \in \{0,1\}^{n_2 \times k_2},\ \mathbf{r}_{w,0}, \mathbf{r}_{w,1} \in \{0,1\}^{k_2 - t},\ \mathbf{e}_{w,0}, \mathbf{e}_{w,1} \in \mathbb{F}_2^{k_{e,2}}, \\ \mathbf{r}_{\mathsf{oa},0}, \mathbf{r}_{\mathsf{oa},1} \in \{0,1\}^{k_1 - n},\ \ \mathbf{e}_{\mathsf{oa},0}, \mathbf{e}_{\mathsf{oa},1} \in \mathbb{F}_2^{k_{e,1}}, \\ \mathbf{d}'_j, \mathbf{w}_0, \ldots, \mathbf{w}_{\ell-1} \in \{0,1\}^n. \end{cases} \tag{9}$$

– The prover's goal is to prove knowledge of the above secret input such that the following hold.

$$\forall i \in \{0,1\} : \mathbf{c}_{w,i} = \mathbf{G}_{j,i} \cdot \begin{pmatrix} \mathbf{r}_{w,i} \\ \mathbf{w} \end{pmatrix} \oplus \mathsf{RE}(\mathbf{e}_{w,i}) \in \{0,1\}^{n_2}; \tag{10}$$

$$\forall i \in \{0,1\} : \mathbf{c}_{\mathsf{oa},i} = \mathbf{G}_{\mathsf{oa},i} \cdot \begin{pmatrix} \mathbf{r}_{\mathsf{oa},i} \\ (1-b) \cdot \mathbf{d}_j \end{pmatrix} \oplus \mathsf{RE}(\mathbf{e}_{\mathsf{oa},i}) \in \{0,1\}^{n_1}; \tag{11}$$

$$b = P_\tau(\mathsf{id}_j, \mathbf{w}); \tag{12}$$

$$F_\tau(\mathbf{w}) = 1; \tag{13}$$

$$\mathbf{d}'_j = \mathsf{TAccu}_{\mathbf{B}}(\mathbf{G}_{j,0}, \mathbf{G}_{j,1}); \tag{14}$$

$$\mathsf{TVerify}_{\mathbf{B}}(\mathbf{u}_\tau, \mathbf{d}_j, w^{(j)}) = 1; \tag{15}$$

$$\mathbf{d}_j = \mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{d}'_j) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathsf{id}_j); \mathbf{d}_j \neq \mathbf{0}^n. \tag{16}$$

The strategy is to transform the above constraints to an instance of $\mathcal{R}_{d,t}$, which can be handled by VOLEitH-based ZK proof in Appendix C.1.

Proving (10). Observe that we can rewrite (10) as

$$\forall i \in \{0,1\} : \mathbf{G}_{j,i} \cdot \begin{pmatrix} \mathbf{r}_{w,i} \\ \mathbf{w} \end{pmatrix} \oplus \mathsf{RE}(\mathbf{e}_{w,i}) \oplus \mathbf{c}_{w,i} = \mathbf{0}^{n_2} \in \{0,1\}^{n_2},$$

where $\mathsf{RE}(\mathbf{e}_{w,i})$ is a regular vector with $k_{e,2}/c_{e,2}$ blocks. As the multiplication $\mathbf{G}_{j,i} \cdot \begin{pmatrix} \mathbf{r}_{w,i} \\ \mathbf{w} \end{pmatrix}$ can be expressed as degree-2 polynomials the left hand side of the above equation is a collection of $t_{\mathbf{c}_w} = 2n_2$ polynomials in $l_{\mathbf{c}_w} = 2(n_2 k_2 + k_{e,2} + k_2) - t$ variables and of degree $d_{\mathbf{c}_w} = \max\{c_{e,2}, 2\}$. These polynomials are satisfied by a witness vector in $\mathbb{F}_2^{l_{\mathbf{c}_w}}$ with entries taken from $\mathbf{G}_{j,0}$, $\mathbf{G}_{j,1}$, $\mathbf{r}_{w,0}$, $\mathbf{r}_{w,1}$, $\mathbf{e}_{w,0}$, $\mathbf{e}_{w,1}$, $\mathbf{w}$.

Proving (11). Similarly, (11) can be rewritten as

$$\forall i \in \{0,1\} : \mathbf{G}_{\mathsf{oa},i} \cdot \begin{pmatrix} \mathbf{r}_{\mathsf{oa},i} \\ (1-b) \cdot \mathbf{d}_j \end{pmatrix} \oplus \mathsf{RE}(\mathbf{e}_{\mathsf{oa},i}) \oplus \mathbf{c}_{\mathsf{oa},i} = \mathbf{0}^{n_1} \in \{0,1\}^{n_1},$$

where $\mathsf{RE}(\mathbf{e}_{\mathsf{oa},i})$ is a regular vector with $k_{e,1}/c_{e,1}$ blocks. As such, proving (11) is equivalent to proving a set of $t_{\mathbf{c}_{\mathsf{oa}}} = 2n_1$ polynomials in $l_{\mathbf{c}_{\mathsf{oa}}} = 2(k_1 - n) + n + 2k_{e,1} + 1$ variables and of degree $d_{\mathbf{c}_{\mathsf{oa}}} = \max\{c_{e,1}, 2\}$. These polynomials are satisfied by a witness vector in $\mathbb{F}_2^{l_{\mathbf{c}_{\mathsf{oa}}}}$ with entries taken from $\mathbf{r}_{\mathsf{oa},0}, \mathbf{r}_{\mathsf{oa},1}, \mathbf{d}_j, \mathbf{e}_{\mathsf{oa},0}, \mathbf{e}_{\mathsf{oa},1}, b$.

Proving (12) and (13). Let C be an arbitrary Boolean circuit represented by addition and multiplication gates. Our goal is to prove the knowledge of

$x = (x_0, \ldots, x_{L-1})$ such that $C(x) = 1$. Let K be the number of multiplication gates in C. We then observe that it suffices to prove that

$$x_{i,0} \cdot x_{i,1} = x_{i,2} \quad \text{for } i \in [0, K-1], \tag{17}$$

where $x_{i,0}, x_{i,1}, x_{i,2}$ are the two inputs and output of the i-th multiplication gate. In addition, $x_{i,0}, x_{i,1}$ can be derived from the inputs or the outputs of some multiplication gates. Therefore, we introduce intermediate values $x_{0,2}, x_{1,2}, \ldots, x_{K-1,2}$ and prove equations (17) hold.

Therefore, to prove (12), let $L_{P_\tau} = n + t, K_{P_\tau}$ be the numbers of inputs and multiplication gates in the circuit computing P_τ, respectively. Let $x_0, \ldots, x_{L_{P_\tau}-1}$ be the assignments to L_{P_τ} input wires of P_τ and $x_{L_{P_\tau}}, \ldots, x_{L_{P_\tau}+K_{P_\tau}-1}$ be the assignments to K_{P_τ} output wires of P_τ. Then proving (12) is equivalent to proving the knowledge of

$$(x_0, \ldots, x_{L_{P_\tau}-1}, x_{L_{P_\tau}}, \ldots, x_{L_{P_\tau}+K_{P_\tau}-1})^\top \in \mathbb{F}_2^{l_{P_\tau}}$$

that satisfies a set of $t_{P_\tau} = K_{P_\tau}$ degree-2 polynomials defined from the topology of P_τ. Here $l_{P_\tau} = L_{P_\tau} + K_{P_\tau}$.

Similarly, to prove (13), let $L_{F_\tau} = t, K_{F_\tau}$ be the numbers of inputs and multiplication gates in the circuit computing F_τ, respectively. Let $x_0, \ldots, x_{L_{F_\tau}-1}$ be the assignments to L_{F_τ} input wires of F_τ and $x_{L_{F_\tau}}, \ldots, x_{L_{F_\tau}+K_{F_\tau}-1}$ be the assignments to K_{F_τ} output wires of F_τ. As F_τ is evaluated to 1, $x_{L_{F_\tau}+K_{F_\tau}-1} = 1$. Then proving (12) is equivalent to proving the knowledge of

$$(x_0, \ldots, x_{L_{F_\tau}-1}, x_{L_{F_\tau}}, \ldots, x_{L_{F_\tau}+K_{F_\tau}-2})^\top \in \mathbb{F}_2^{l_{F_\tau}}$$

that satisfies a set of $t_{F_\tau} = K_{F_\tau}$ degree-2 polynomials defined from the topology of F_τ. Here $l_{F_\tau} = L_{F_\tau} + K_{F_\tau} - 1$.

Proving (14). We need to prove knowledge of $\mathbf{d}'_j$ and $\mathbf{G}_{j,0}, \mathbf{G}_{j,1}$ such that $\mathbf{d}'_j = \mathsf{TAccu}_{\mathbf{B}}(D_j)$, where:

- $\mathbf{G}_{j,b} = [\mathbf{g}_{j,k_2 \cdot b} | \mathbf{g}_{j,k_2 \cdot b+1} | \cdots | \mathbf{g}_{j,k_2 \cdot b+k_2-1}]$;
- $\mathbf{g}_{j,k_2 \cdot b+l} = \left(\mathbf{g}_{j,k_2 \cdot b+l,0} \| \ldots \| \mathbf{g}_{j,k_2 \cdot b+l,n_2/n-1}\right)$ and $\mathbf{g}_{j,k_2 \cdot b+l,0}, \ldots, \mathbf{g}_{j,k_2 \cdot b+l,n_2/n-1} \in \{0,1\}^n$
- $D_j = \{\mathbf{g}_{j,0,0}, \ldots, \mathbf{g}_{j,0,n_2/n-1}, \ldots, \mathbf{g}_{j,2k_2-1,0}, \ldots, \mathbf{g}_{j,2k_2-1,n_2/n-1}, \mathbf{0}^n, \ldots, \mathbf{0}^n\}$, with the number of padded vectors $\mathbf{0}^n$ chosen such that $|D_j|$ is a power of 2.

Since $|D_j| = 2^v$ for some positive integer v, by rewriting $\mathbf{D}_j$ as $\{\mathbf{p}^j_{v,0}, \ldots, \mathbf{p}^j_{v,2^v-1}\}$; the condition $\mathbf{d}'_j = \mathsf{TAccu}_{\mathbf{B}}(D_j)$ is equivalent to

$$\begin{cases} \mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{p}^j_{1,0}) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{p}^j_{1,1}) \oplus \mathbf{d}'_j = \mathbf{0}^n; \\ \mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{p}^j_{2,2i}) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{p}^j_{2,2i+1}) \oplus \mathbf{p}^j_{1,i} = \mathbf{0}^n, \quad \forall i \in [0, 2^1 - 1]; \\ \cdots\cdots \\ \mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{p}^j_{v-1,2i}) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{p}^j_{v-1,2i+1}) \oplus \mathbf{p}^j_{v-2,i} = \mathbf{0}^n, \quad \forall i \in [0, 2^{v-2} - 1]; \\ \mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{p}^j_{v,2i}) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{p}^j_{v,2i+1}) \oplus \mathbf{p}^j_{v-1,i} = \mathbf{0}^n, \quad \forall i \in [0, 2^{v-1} - 1]; \end{cases} \tag{18}$$

Therefore, we need to prove that (18) is satisfied by

$$(\mathbf{d}'_j, \mathbf{p}^j_{1,0}, \mathbf{p}^j_{1,1}, \mathbf{p}^j_{2,0}, \ldots, \mathbf{p}^j_{2,3}, \ldots, \mathbf{p}^j_{v-1,0}, \ldots, \mathbf{p}^j_{v-1,2^{v-1}-1}, \mathbf{p}^j_{v,0}, \ldots, \mathbf{p}^j_{v,2^v-1}).$$

Note that, since there are $2^v - n_2/n \cdot 2k_2$ zero vectors in D_j, there are approximately $2(2^v - n_2/n \cdot 2k_2)$ vectors $\mathbf{p}^j$ that are zero vectors. In short, proving (14) is equivalent to proving a collection of $t_{\mathsf{acc}} \approx [(2^v - 1) - (2^v - n_2/n \cdot 2k_2)] \cdot n \approx 2n_2k_2 - n$ polynomials of degree $d_{\mathsf{acc}} = c$, satisfied by a witness

$$(\mathbf{d}'_j, \mathbf{p}^j_{1,0}, \mathbf{p}^j_{1,1}, \mathbf{p}^j_{2,0}, \ldots, \mathbf{p}^j_{2,3}, \ldots, \mathbf{p}^j_{v-1,0}, \ldots, \mathbf{p}^j_{v-1,2^{v-1}-1}, \mathbf{p}^j_{v,0}, \ldots, \mathbf{p}^j_{v,2^v-1})^\top \in \mathbb{F}_2^{l_{\mathsf{acc}}},$$

where $l_{\mathsf{acc}} \approx 2^{v+1} - 2(2^v - n_2/n \cdot 2k_2) = 4n_2k_2/n$.

Proving (15). The technique of [63] proving that $\mathsf{TVerify}_{\mathbf{B}}(\mathbf{u}_\tau, \mathbf{d}_j, w^{(j)}) = 1$ proceeds by rewriting this verification condition as

$$\begin{aligned}(j_{\ell-1} \oplus 1) \cdot (\mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{v}_{\ell-1}) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{w}_{\ell-1})) \\ \oplus\, j_{\ell-1} \cdot (\mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{w}_{\ell-1}) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{v}_{\ell-1})) \oplus \mathbf{u}_\tau = \mathbf{0}^n,\end{aligned} \tag{19}$$

and for all $\theta \in \{\ell - 2, \ell - 1, \ldots, 1\}$

$$\begin{aligned}(j_\theta \oplus 1) \cdot (\mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{v}_\theta) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{w}_\theta)) \\ \oplus\, j_\theta \cdot (\mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{w}_\theta) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathbf{v}_\theta)) \oplus \mathbf{v}_{\theta-1} = \mathbf{0}^n,\end{aligned} \tag{20}$$

where $\mathbf{v}_0 = \mathbf{d}_j$. Therefore, it suffices to prove knowledge of $w^{(j)}$ and $\mathbf{v}_0, \ldots, \mathbf{v}_{\ell-1}$ satisfying (19) and (20). As the regular encoding process is described by degree-c contraints, (19) and (20) are a collection of $t_{\mathsf{vacc}} = \ell n$ polynomial constraints of degree $d_{\mathsf{vacc}} = c+1$, which are satisfied by a witness $(w^{(j)}, \mathbf{v}_0, \ldots, \mathbf{v}_{\ell-1})^\top \in \mathbb{F}_2^{l_{\mathsf{vacc}}}$ where $l_{\mathsf{vacc}} = (2n + 1)\ell$.

Proving (16). To prove (16), first observe that the equation

$$\mathbf{B}_0 \cdot \mathsf{RE}(\mathbf{d}'_j) \oplus \mathbf{B}_1 \cdot \mathsf{RE}(\mathsf{id}_j) \oplus \mathbf{d}_j = \mathbf{0}$$

is a collection of n degree-c constraints over a witness vector with entries taken from $\mathbf{d}'_j, \mathsf{id}_j, \mathbf{d}_j$. If we let $\mathbf{d}_j = (d_{j,0}, \ldots, d_{j,n-1}) \in \mathbb{F}_2^n$, then $\mathbf{d}_j \neq \mathbf{0}^n$ is equivalent to

$$(d_{j,0} + 1) \cdot (d_{j,1} + 1) \cdots (d_{j,n-1} + 1) = 0. \tag{21}$$

By introducing intermediate witnesses $e_0 = (d_{j,0} + 1) \cdot (d_{j,1} + 1)$, and $e_i = e_{i-1} \cdot (d_{j,i+1} + 1)$ for $i \in [1, n-2]$. Then the equation (21) is equivalent to $(n-1)$ polynomials of degree 2. In short, proving (16) is equivalent to proving that a collection of $t_{\mathsf{zero}} = (2n - 1)$ polynomials of degree $d_{\mathsf{zero}} = \max\{1, 2, c\} = c$, satisfied by a witness $(\mathbf{d}'_j, \mathsf{id}_j, \mathbf{d}_j, e_0, \ldots, e_{n-2})^\top \in \mathbb{F}_2^{l_{\mathsf{zero}}}$, where $l_{\mathsf{zero}} = 4n - 2$.

Putting everything together. We transformed conditions (10)-(16) to a set of polynomials over $\mathbb{F}_2$ and of degree at most d_{proof}, where

$$d_{\mathsf{proof}} = \max\{d_{\mathbf{c}_w}, d_{\mathbf{c}_{\mathsf{oa}}}, d_{P_\tau}, d_{F_\tau}, d_{\mathsf{acc}}, d_{\mathsf{vacc}}, d_{\mathsf{zero}}\} = \max\{c + 1, c_{e,1}, c_{e,2}\}.$$

These polynomials are satisfied by a witness vector $\mathbb{F}_2^{l_{\mathsf{proof}}}$ that is defined from the prover's secret input in (9). Note that, we have

$$l_{\mathsf{proof}} = l_{\mathbf{c}_w} + l_{\mathbf{c}_{\mathsf{oa}}} + l_{P_\tau} + l_{F_\tau} + l_{\mathsf{acc}} + l_{\mathsf{vacc}} + l_{\mathsf{zero}}$$

and the number of polynomials describing conditions (10)-(16) is given by

$$t_{\mathsf{proof}} = t_{\mathbf{c}_w} + t_{\mathbf{c}_{\mathsf{oa}}} + t_{P_\tau} + t_{F_\tau} + t_{\mathsf{acc}} + t_{\mathsf{vacc}} + t_{\mathsf{zero}}.$$

It suffices to run the protocol $\Pi^t_{\mathsf{dD-Rep}}$, with a linear code $\mathcal{C}_{\mathrm{Rep}} = [\delta, 1, \delta]$ and $q = 2^r$ as parameters of the $\mathcal{F}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}_{\mathsf{sVOLE}}$ functionality. The protocol has soundness error bounded by $(d_{\mathsf{proof}} + 1)/2^{r\delta}$. One can make the soundness error negligible by choosing δ such that $(d_{\mathsf{proof}} + 1)/2^{r\delta} = \mathsf{negl}(\lambda)$.

Prover's Complexity. We estimate the prover's runtime for generating a VOLEitH-based NIZK proof for the satisfiability of the polynomial system $\{f_i\}_{i\in[t_{\mathsf{proof}}]}$ describing the conditions (10)-(16), using the parameters provided in Table 2. This task is done via the following steps:

1. The first step is invoking the $\mathcal{F}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}_{\mathsf{sVOLE}}$ functionality, instantiated via the GGM-based vector commitment [9,8]. This requires around $2L_{\mathsf{proof}}\delta$ calls to the PRG/hash on underlying the vector commitment, where $L_{\mathsf{proof}} = l_{\mathsf{proof}} + (d_{\mathsf{proof}} - 1)\delta r$. Each call to the PRG/hash applies the primitive to a string of fixed length, which depends on λ and δ only. With the parameters in Table 2, the total number of calls to PRG/hash is around $9.8 \cdot 10^8$.
2. The second step is computing the first message $\mathbf{d} \in \mathbb{F}_p^{l_{\mathsf{proof}}}$ in $\Pi^t_{\mathsf{dD-Rep}}$, which requires l_{proof} additions over the base field $\mathbb{F}_p$;
3. The last step is generating a response in $\Pi^t_{\mathsf{dD-Rep}}$, which is the most dominant overall. Using the same analysis as in Appendix C.1, the prover's complexity for generating a VOLEitH-based NIZK proof is dominated by the cost of computing the coefficients $A_{i,j}$ via interpolating the polynomials $f_{i,h}$, where for $h \in \{0, \ldots, d_{\mathsf{proof}}\}$ the degree-h polynomials $f_{i,h}$ is the homogenous polynomial in the homogenous decomposition of f_i. We remark that, the high-degree polynomials in the system $\{f_i\}_{i\in[t_{\mathsf{proof}}]}$ come from (10), (11), (14), (15) and (16) (i.e., equations involving RE). These polynomials in the above equations, in turn, are linear combinations of at most $2n_2 + 2n_1 + \ell n + 2n_2k_2 - n + 2n - 1 = 2(n_1 + n_2 + n_2k_2) + (\ell+1)n - 1$ polynomials describing the encoding RE. Each of the polynomials describing RE is of degree (at most) 8 and defined over (at most) 8 variables, thereby containing up to $2^8 = 256$ monomials terms. Evaluating those monomial terms at a single point requires 256 multiplications. Therefore, to evaluate the $f_{i,h}$'s at $d_{\mathsf{proof}} + 1$ interpolation points, we can evaluate the $(2(n_1 + n_2 + n_2k_2) + (\ell + 1)n - 1) \cdot 256$ monomials in the expansions of $f_{i,h}$'s, then take a linear combination. This results in at most $(2(n_1 + n_2 + n_2k_2) + (\ell + 1)n - 1) \cdot 256 \approx 6.98 \cdot 10^9$ field multiplications. Then, we reconstruct the coefficients $A_{i,j}$ by taking a linear combination of the (precomputed) Lagrange polynomials, which requires another $t_{\mathsf{proof}} \cdot d_{\mathsf{proof}} \cdot (d_{\mathsf{proof}} + 1) \approx 2.43 \cdot 10^9$ field multiplications. In total,

there are around $9.41 \cdot 10^9$ field multiplications. The cost multiplication over $\mathbb{F}_{q^\delta} = \mathbb{F}_{2^{144}}$ is 4 times the cost of multiplication in the subfield $\mathbb{F}_{2^{72}}$. From the benchmark provided in the Rust library gf256, we take the 6.93 ns cost of multiplication over $\mathbb{F}_{2^{64}}$ performed by a 3.5 GHz CPU as the cost of multiplication over $\mathbb{F}_{2^{72}}$. This results in around 260 seconds for field operations needed for the NIZK proof.

D Deferred Security Proofs for the Code-Based Construction

This section provides proofs of the remaining lemmas required for Theorem 2.

Lemma 7 (Type 2 - Anonymity). *Assume that problems* $\mathsf{DMcE}(n_1, k_1, t_1)$, $\mathsf{DMcE}(n_2, k_2, t_2)$ *and problems* $\mathsf{DLPN}(n_1, k_1 - n, \mathsf{Regular}(k_{e,1}, c_{e,1}))$, $\mathsf{DLPN}(n_2, k_2 - t, \mathsf{Regular}(k_{e,2}, c_{e,2}))$ *problems are hard, the VOLEitH-based NIZK argument system used to generate* π_ψ *is simulation-sound and zero-knowledge, then the given* GEOT *construction is Type 2 - anonymous in the random oracle model.*

Proof. Compared to $\mathbf{Exp}^{\mathsf{anony}}_{\mathcal{A}}(1^\lambda)$, the adversary in $\mathbf{Exp}^{\mathsf{anony-ntr}}_{\mathcal{A}}(1^\lambda)$ is given the key $\mathsf{sk}_{\mathsf{OA}}$ additionally. However, it is required that the challenge tuple also satisfies $P_\tau(\mathsf{id}_{j_0}, \mathbf{w}) = 1$ and $P_\tau(\mathsf{id}_{j_1}, \mathbf{w}) = 1$. This extra requirement guarantees that the challenge ciphertext ψ is non-traceable, i.e., both $\mathbf{c}_{\mathsf{oa},0}$ and $\mathbf{c}_{\mathsf{oa},1}$ encrypt an all-zero string $\mathbf{0}^n$. Thus giving out $\mathsf{sk}_{\mathsf{OA}}$ does not reveal extra information to $\mathcal{A}$. Thus the given construction is Type 2 - anonymous. The proof is similar to the one for proving Type 1 - anonymity, in which we no longer need to change $\mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}$ to random ones and switch between $\mathsf{sk}^{(\mathsf{oa},0)}_{\mathsf{Mc}}$ and $\mathsf{sk}^{(\mathsf{oa},1)}_{\mathsf{Mc}}$. Details are omitted here. □

Lemma 8 (Message Secrecy). *Assume the* $\mathsf{DMcE}(n_2, k_2, t_2)$ *and the* $\mathsf{DLPN}(n_2, k_2 - t, \mathsf{Regular}(k_{e,2}, c_{e,2}))$ *problems are hard, the VOLEitH-based NIZK argument system used to generate* π_ψ *is zero-knowledge and simulation-sound, then the given* GEOT *scheme satisfies message secrecy in the random oracle model.*

Proof. We follow the method of proving Type 1 - anonymity to prove message secrecy. Let $\mathbf{Exp}^{\mathsf{sec}-b}_{\mathcal{A}}(1^\lambda)$ the experiment $\mathbf{Exp}^{\mathsf{sec}}_{\mathcal{A}}(1^\lambda)$ by fixing b. Denote W_i as the event that Game i returns 1.

Game 0**:** This is $\mathbf{Exp}^{\mathsf{sec}-b}_{\mathcal{A}}(1^\lambda)$. The challenger $\mathcal{C}$ runs the initialization algorithm to produce public parameters pp, secret key $\mathsf{sk}_{\mathsf{OA}}$ for the OA, and secret key $\mathsf{sk}_{\mathsf{GM}}$ for the GM. $\mathcal{C}$ also maintains two lists HUL and CUL. The adversary $\mathcal{A}$ fully corrupts the GM and OA, obtaining their secret keys $\mathsf{sk}_{\mathsf{GM}}, \mathsf{sk}_{\mathsf{OA}}$. Throughout the experiment, $\mathcal{A}$ is allowed to enroll honest users to the group via the oracle USER and to learn their secret keys via RevealU, and has decryption access to the challenge user id_j (chosen at a later point) via the oracle DEC.
Let the challenge tuple outputted by $\mathcal{A}$ be $(\tau, \mathbf{w}_0, \mathbf{w}_1, \mathsf{id}_j)$. This game will proceed further only if id_j is an active user at τ and in the set HUL $\setminus$

CUL, and $F_\tau(\mathbf{w}_0) = F_\tau(\mathbf{w}_1) = 1$. Retrieve the membership certificate of id_j, denoted as $(\mathsf{bin}(j), \mathsf{id}_j, \mathsf{pk}_j, \mathbf{d}_j)$. Compute a challenge ciphertext $\psi^* = (\mathbf{c}^*_{w,0}, \mathbf{c}^*_{w,1}, \mathbf{c}^*_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}, \pi_{\psi^*}) \leftarrow \mathsf{Enc}(\tau, \mathbf{w}_b, \mathsf{id}_j, \mathsf{cert}_{\mathsf{id}_j})$, where $\mathbf{c}^*_{w,0}, \mathbf{c}^*_{w,1}$ encrypt $\mathbf{w}_b$ under pk_j while $\mathbf{c}^*_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}$ encrypt $(1-b)\cdot\mathbf{d}_j$ with $b = P_\tau(\mathsf{id}_j, \mathbf{w})$. By definition, $\mathsf{Pr}[\mathbf{Exp}^{\mathsf{sec}-b}_{\mathcal{A}}(1^\lambda) = 1] = \mathsf{Pr}[W_0]$. Note that having the key $\mathsf{sk}_{\mathsf{OA}}$ does not enable $\mathcal{A}$ to learn more information about which one of the plaintexts $\mathbf{w}_0$, $\mathbf{w}_1$ is encrypted. This is because opening ψ always produces the same value $(1-b)\cdot\mathbf{d}_j$ no matter which message is encrypted.

Game 1: This game uses $\mathsf{sk}^{(j,1)}_{\mathsf{Mc}}$ rather than $\mathsf{sk}^{(j,0)}_{\mathsf{Mc}}$ when a tuple (τ, ψ) is queried to the oracle $\mathsf{DEC}(\mathsf{sk}_{\mathsf{id}_j}, \cdot)$. The adversary's view is the same as in Game 0 unless $\mathcal{A}$ computes a ciphertext $\psi' = (\mathbf{c}'_{w,0}, \mathbf{c}'_{w,1}, \mathbf{c}'_{\mathsf{oa},0}, \mathbf{c}'_{\mathsf{oa},1}, \pi_{\psi'})$ such that $\mathbf{c}'_{w,0}, \mathbf{c}'_{w,1}$ encrypt different messages and $\pi_{\psi'}$ is a valid proof. Nevertheless, the probability that $\mathcal{A}$ outputs such a ciphertext is negligible due to the soundness of the underlying argument system. Thus $|\mathsf{Pr}[W_1] - \mathsf{Pr}[W_0]| \leq \mathbf{Adv}^{\mathsf{sound}} \leq \mathsf{negl}(\lambda)$.

Game 2: Instead of using real proofs π_{ψ^*} when generating the challenge ciphertext ψ^*, we turn to its zero-knowledge simulator and produce a simulated proof π_{ψ^*}. Since the system used to generate π_{ψ^*} is zero-knowledge, the view of $\mathcal{A}$ in this game is computationally close to that in Game 1, i.e., $|\mathsf{Pr}[W_2] - \mathsf{Pr}[W_1]| \leq \mathsf{negl}(\lambda)$ holds.

Game 3: This game modifies Game 2 by sampling $\mathbf{c}^\star_{w,0} \xleftarrow{\$} \{0,1\}^{n_2}$ and then substituting the challenge ciphertext $\psi^* = (\mathbf{c}^*_{w,0}, \mathbf{c}^*_{w,1}, \mathbf{c}^*_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}, \pi_{\psi^*})$ for a new challenge ciphertext $\psi^{\star\star} = (\mathbf{c}^\star_{w,0}, \mathbf{c}^*_{w,1}, \mathbf{c}^*_{\mathsf{oa},0}, \mathbf{c}^*_{\mathsf{oa},1}, \pi_{\psi^{\star\star}})$. Since the key $\mathsf{sk}^{(j,0)}_{\mathsf{Mc}}$ is not used for the decryption queries, $\mathcal{A}$ cannot tell the differences between Game 3 and Game 2 due to the hardness of $\mathsf{DMcE}(n_2, k_2, t_2)$ and $\mathsf{DLPN}(n_2, k_2 - t, \mathsf{Regular}(k_{e,2}, c_{e,2}))$. Therefore, $|\mathsf{Pr}[W_3] - \mathsf{Pr}[W_2]| \leq \mathsf{negl}(\lambda)$.

Game 4: We now shift back to $\mathsf{sk}^{(j,0)}_{\mathsf{Mc}}$ for the oracle $\mathsf{DEC}(\mathsf{sk}_{\mathsf{id}_j}, \cdot)$ queries. In the challenge ciphertext $\psi^{\star\star}$, $\pi_{\psi^{\star\star}}$ is a simulated proof for a false statement. This is because that $\mathbf{c}^\star_{w,0}$ is random while $\mathbf{c}^*_{w,1}$ still encrypts $\mathbf{w}_b$. In other words, $\mathcal{A}$ sees a simulated proof for a false statement. If $\mathcal{A}$ comes up with a tuple (τ', ψ') such that (i) (τ', ψ') is different from the challenge tuple $(\tau, \psi^{\star\star})$; (ii) $\pi_{\psi'}$ is valid and (iii) $\mathbf{c}'_{w,0}, \mathbf{c}'_{w,1}$ encrypt different messages, it breaks the simulation-soundness of the underlying argument system. If no such tuple is found, the view of $\mathcal{A}$ in this game and the previous game is the same. As a result, $|\mathsf{Pr}[W_4] - \mathsf{Pr}[W_3]| \leq \mathbf{Adv}^{\mathsf{sim}-\mathsf{sound}} \leq \mathsf{negl}(\lambda)$.

Game 5: Once we switch back to the key $\mathsf{sk}^{(j,0)}_{\mathsf{Mc}}$, we can replace $\mathbf{c}^*_{w,1}$ with a uniformly random vector $\mathbf{c}^\star_{w,1} \xleftarrow{\$} \{0,1\}^{n_2}$ without changing $\mathcal{A}$'s view significantly. This is again because of the hardness of the problems $\mathsf{DMcE}(n_2, k_2, t_2)$ and $\mathsf{DLPN}(n_2, k_2 - t, \mathsf{Regular}(k_{e,2}, c_{e,2}))$. This is how we modify Game 4 to obtain Game 5. Therefore, $|\mathsf{Pr}[W_5] - \mathsf{Pr}[W_4]| \leq \mathsf{negl}(\lambda)$.

We can see that Game 5 is completely independent of b. Thus, the probability that $\mathcal{A}$ guesses b correctly is $\frac{1}{2}$, i.e., $\mathsf{Pr}[W_5] = \frac{1}{2}$. Combining those results,

$|\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{sec}-b}(1^\lambda)=b]-\frac{1}{2}|\leq \mathsf{negl}(\lambda)$. Therefore,

$$\mathbf{Adv}_{\mathcal{A}}^{\mathsf{sec}}(1^\lambda)=|\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{sec}}(1^\lambda)=1]-\frac{1}{2}|=|\mathsf{Pr}[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{sec}-b}(1^\lambda)=b]-\frac{1}{2}|\leq \mathsf{negl}(\lambda).$$

□

Now we give a proof of unforgeability. Note that, as the VOLEitH-based NIZK is obtained via Fiat-Shamir transformation, the adversaries in experiments $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{extract}}$ and $\mathbf{Ex}_{\mathcal{A}}^{\mathsf{unforge}}$ in Section 2.3 have access to the random oracles and a NIZK proof verification oracle. Next, let us describe the $\mathsf{SimSetup}$ and $\mathsf{Extract}$ algorithms.

$\mathsf{SimSetup}(1^\lambda)$**.** This simulated algorithm produces pp and $\mathsf{sk}_{\mathsf{GM}},\mathsf{sk}_{\mathsf{OA}}$ as the real setup algorithm. The difference is that the PRGs [5] $\mathsf{PRG}_{\mathsf{VC}},\mathsf{G}_{\mathsf{VC}}$ and hash functions $\mathcal{H}_{\mathsf{VC}},\mathcal{H}_{\mathsf{FS}}$ are modeled as a random oracle.

$\mathsf{Extract}(\tau_{\mathrm{ext}},(\mathsf{pp},\tau,\psi))$**.** If $\mathsf{Verify}(\tau,\psi)=0$, return 0. Else, this algorithm runs the NIZK extractor described in [9, Lemma 4] to obtain a witness $\mathbf{w}'_{\mathsf{proof}}$. Then, backtracking all the steps we have performed on ξ of form (4), we are able to obtain ξ' such that all the conditions (3a)-(3e) in the Enc algorithm are satisfied with overwhelming probability. Let ξ' be of the following form.

$$\begin{aligned}\xi'=\big(\ &\mathbf{r}'_{w,0},\ \mathbf{r}'_{w,1},\ \mathbf{e}'_{w,0},\ \mathbf{e}'_{w,1},\ \mathbf{w}',\ \mathbf{r}'_{\mathsf{oa},0},\ \mathbf{r}'_{\mathsf{oa},1},\ \mathbf{e}'_{\mathsf{oa},0},\ \mathbf{e}'_{\mathsf{oa},1},\\ &b',\ \mathsf{bin}(j'),\ \mathsf{id}',\ \mathbf{G}'_0,\ \mathbf{G}'_1,\ \mathbf{d}'',\mathbf{d}',\mathbf{w}'_0,\dots,\mathbf{w}'_{\ell-1}\ \big).\end{aligned}\tag{22}$$

Then ξ' satisfies the following conditions.

(a) $\mathbf{c}_{w,0},\mathbf{c}_{w,1}$ encrypt $\mathbf{w}'$ under the keys $\mathbf{G}'_0,\mathbf{G}'_1$, respectively.

(b) The plaintext $\mathbf{w}'$ satisfies the filtering relation $F_\tau(\mathbf{w}')=1$.

(c) $b'=P_\tau(\mathsf{id}',\mathbf{w}')$, and $\mathbf{c}_{\mathsf{oa},0},\mathbf{c}_{\mathsf{oa},1}$ encrypt $(1-b')\cdot\mathbf{d}'$ under the keys $\mathbf{G}_{\mathsf{oa},0},\mathbf{G}_{\mathsf{oa},1}$.

(d) The keys $\mathbf{G}'_0,\mathbf{G}'_1$ are hashed to $\mathbf{d}''$ by using a Merkle tree accumulator specified by $h_{\mathbf{B}}$. In addition, $\mathbf{d}'=\mathbf{B}_0\cdot\mathsf{RE}(\mathbf{d}'')\oplus\mathbf{B}_1\cdot\mathsf{RE}(\mathsf{id}')$.

(e) $\mathbf{d}'$ satisfies $\mathsf{TVerify}_{\mathbf{B}}(\mathbf{u}_\tau,\mathbf{d}',w')=1$ and $\mathbf{d}'\neq\mathbf{0}^n$. Here w' is the witness $w'=(\mathsf{bin}(j'),\mathbf{w}'_0,\dots,\mathbf{w}'_{\ell-1})$.

Finally, return $(\mathsf{id}',\mathbf{w}')$.

Lemma 9. *If the* $2-\mathsf{RNSD}_{n,2n,c}$ *problem is hard, then our code-based* GEOT *scheme is extractable.*

Proof. First of all, the differences between the algorithms $\mathsf{SimSetup}$ and Setup lie in the treatment of hash functions $\mathsf{PRG}_{\mathsf{VC}},\mathsf{G}_{\mathsf{VC}},\mathcal{H}_{\mathsf{VC}},\mathcal{H}_{\mathsf{FS}}$. These, however, is indistinguishable to the adversary.

Next, we need to show that the advantage of any PPT adversary $\mathcal{A}$ in experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{extract}}(1^\lambda)$ in Section 2.3 is negligible. Let (τ,ψ) be a valid tuple outputted by $\mathcal{A}$. Run the algorithm $\mathsf{Extract}(\tau_{\mathrm{ext}},(\mathsf{pp},\tau,\psi))$, obtaining ξ' of

[5] Note that $\mathsf{PRG}_{\mathsf{VC}},\mathsf{G}_{\mathsf{VC}},\mathcal{H}_{\mathsf{VC}}$ are used to realize the $\mathcal{F}_{\mathsf{sVOLE}}^{p,q,S_\Delta,\mathcal{C},l,\mathcal{L}}$ functionality, and need to be modeled as random oracles.

form (22). Then following [9, Lemma 4], the conditions (3a)-(3e) in the Enc algorithm are satisfied by the extracted witness ζ', except with a probability bounded by $3(Q_{\mathsf{FS}}+Q_{\mathsf{Verify}})(d_{\mathsf{proof}}+1)/2^{r\delta}+\mathsf{negl}(\lambda)$; where Q_{FS} is the maximum number of queries to $\mathcal{H}_{\mathsf{FS}}$ and Q_{Verify} is the maximum number of queries to the NIZK proof verification oracle.

Note that $F_\tau(\mathbf{w}')=1$. Next, observe that $\mathsf{Open}(\mathsf{sk}_{\mathsf{OA}},(\tau,\psi))$, with $\mathsf{sk}_{\mathsf{OA}}=(\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)},\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},1)})$, simply runs $\mathsf{McDec}(\mathsf{sk}_{\mathsf{Mc}}^{(\mathsf{oa},0)},\mathbf{c}_{\mathsf{oa},0})$ and returns id^*. Due to the correctness of the McEliece encryption scheme, we would obtain $(1-b')\cdot\mathbf{d}'$ with $b'=P_\tau(\mathsf{id}',\mathbf{w}')$, which is $\mathbf{0}$ if $b'=1$ and $\mathbf{d}'$ if $b'=0$. Let id^* be the output of the Open algorithm.

Retrieve a certificate $\mathsf{cert}=(\mathsf{bin}(j'),\mathsf{id}_{j'},\mathbf{G}_{j',0},\mathbf{G}_{j',1},\mathbf{d}_{j'})$, such that the first element is $\mathsf{bin}(j')$. Let $\mathbf{d}'_{j'}=\mathsf{TAccu}_{\mathbf{B}}(\mathbf{G}_{j',0},\mathbf{G}_{j',1})$. We consider the following cases.

Case 1: $(\mathsf{id}',\mathbf{G}'_0,\mathbf{G}'_1,\mathbf{d}'',\mathbf{d}')\neq(\mathsf{id}_{j'},\mathbf{G}_{j',0},\mathbf{G}_{j',1},\mathbf{d}'_{j'},\mathbf{d}_{j'})$. Then it is guaranteed that one can find different pairs $(\mathbf{u},\mathbf{v}),(\mathbf{u}',\mathbf{v}')\in\{0,1\}^n\times\{0,1\}^n$ such that $h_{\mathbf{B}}(\mathbf{u},\mathbf{v})=h_{\mathbf{B}}(\mathbf{u}',\mathbf{v}')$. In the extreme case, $h_{\mathbf{B}}(\mathbf{u},\mathbf{v})=h_{\mathbf{B}}(\mathbf{u}',\mathbf{v}')=\mathbf{u}_\tau$. This however finds a solution to the $2{-}\mathsf{RNSD}_{n,2n,c}$ problem.

Case 2: $(\mathsf{id}',\mathbf{G}'_0,\mathbf{G}'_1,\mathbf{d}'',\mathbf{d}')=(\mathsf{id}_{j'},\mathbf{G}_{j',0},\mathbf{G}_{j',1},\mathbf{d}'_{j'},\mathbf{d}_{j'})$. We consider two subcases.

- $b'=P_\tau(\mathsf{id}',\mathbf{w}')=1$, in other words, $\mathbf{c}_{\mathsf{oa},0},\mathbf{c}_{\mathsf{oa},1}$ are ciphertexts of $\mathbf{0}^n$. Since the McEliece cryptosystem is correct, Step 1 of the Open algorithm obtains $\mathbf{0}^n$ and thus returns $\perp$. Thus, $\mathsf{id}^*=\perp$.
- $b'=P_\tau(\mathsf{id}',\mathbf{w}')=0$, i.e., $\mathbf{c}_{\mathsf{oa},0},\mathbf{c}_{\mathsf{oa},1}$ are ciphertexts of $\mathbf{d}'$. Thus the Open algorithm can uncover $\mathbf{d}'$, and find out the correct index j' such that $\mathbf{reg}[j][0]=\mathbf{d}'$ and $\mathbf{reg}[j'][1]=(\mathsf{bin}(j'),\mathsf{id}_{j'},\mathbf{G}_{j',0},\mathbf{G}_{j',1},\mathbf{d}_{j'})$. As a result, Open returns $\mathsf{id}^*=\mathsf{id}_{j'}$. This implies that the extracted identifier id' and the opened identifier $\mathsf{id}^*=\mathsf{id}_{j'}$ are the same.

The above actually implies that $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{extract}}(1^\lambda)$ outputs 1 with negligible probability. This ends the proof. □

Lemma 10 (Unforgeability). *If the* $2{-}\mathsf{RNSD}_{n,2n,c}$ *problem is hard and the VOLEitH-based NIZK argument system is extractable , then the given* GEOT *scheme is unforgeable in the random oracle model.*

Proof. First, note that our GEOT is extractable by Lemma 9. We now prove that the advantage in the experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{unforge}}(1^\lambda)$ of any PPT adversary $\mathcal{A}$ is negligible.

We remark that this proof is quite similar to that of Lemma 9 with the following major differences.

(a) $\mathsf{sk}_{\mathsf{GM}}$ is not known by $\mathcal{A}$. Therefore, $\mathcal{A}$ is given access to two oracles REG and GUp.
(b) The experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{unforge}}(1^\lambda)$ also returns 1 if $\mathsf{IsActive}(\tau,\mathsf{id}')=0$. Here $(\mathsf{id}',\mathbf{w}')\leftarrow\mathsf{Extract}(\tau_{\mathrm{ext}},\mathsf{pp},\tau,\psi)$, where (τ,ψ) is a valid message output by $\mathcal{A}$.

Due to the similarity, we proceed as in Lemma 9 until the place where we had considered two cases. To tackle the above differences, we consider the following three cases.

Case 1: $(\mathsf{id}', \mathbf{G}'_0, \mathbf{G}'_1, \mathbf{d}'', \mathbf{d}') \neq (\mathsf{id}_{j'}, \mathbf{G}_{j',0}, \mathbf{G}_{j',1}, \mathbf{d}'_{j'}, \mathbf{d}_{j'})$. Then it is guaranteed that one can find different pairs $(\mathbf{u}, \mathbf{v}), (\mathbf{u}', \mathbf{v}') \in \{0,1\}^n \times \{0,1\}^n$ such that $h_{\mathbf{B}}(\mathbf{u}, \mathbf{v}) = h_{\mathbf{B}}(\mathbf{u}', \mathbf{v}')$. In the extreme case, $h_{\mathbf{B}}(\mathbf{u}, \mathbf{v}) = h_{\mathbf{B}}(\mathbf{u}', \mathbf{v}') = \mathbf{u}_\tau$. This however finds a solution to the $2-\mathsf{RNSD}_{n,2n,c}$ problem.

Case 2: $(\mathsf{id}', \mathbf{G}'_0, \mathbf{G}'_1, \mathbf{d}'', \mathbf{d}') = (\mathsf{id}_{j'}, \mathbf{G}_{j',0}, \mathbf{G}_{j',1}, \mathbf{d}'_{j'}, \mathbf{d}_{j'})$ and $\mathsf{IsActive}(\mathsf{info}_\tau, \mathsf{id}_{j'}) = 0$. The former implies that the extracted value of the leaf $\mathsf{bin}(j')$ in the (main)-Merkle tree at τ is $\mathbf{d}_{j'} = \mathbf{d}' \neq \mathbf{0}^n$. The latter, however, implies that the leaf $\mathsf{bin}(j')$ value is indeed $\mathbf{0}^n$. Thus, we have two different paths from the leaf node $\mathsf{bin}(j')$ to the root $\mathbf{u}_\tau$. Hence, one is able to find a vector $\mathbf{z} \in 2\text{-}\mathsf{regular}(2n, c)$ such that $\mathbf{B} \cdot \mathbf{z} = \mathbf{0} \bmod 2$. This again solves the $2\text{-}\mathsf{RNSD}_{n,2n,c}$ problem.

Case 3: $(\mathsf{id}', \mathbf{G}'_0, \mathbf{G}'_1, \mathbf{d}'', \mathbf{d}') = (\mathsf{id}_{j'}, \mathbf{G}_{j',0}, \mathbf{G}_{j',1}, \mathbf{d}'_{j'}, \mathbf{d}_{j'})$ and $\mathsf{IsActive}(\mathsf{info}_\tau, \mathsf{id}_{j'}) = 1$. We consider two subcases.

- $b' = P_\tau(\mathsf{id}', \mathbf{w}') = 1$, in other words, $\mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}$ are ciphertexts of $\mathbf{0}^n$. Since the McEliece cryptosystem is correct, Step 1 of the Open algorithm obtains $\mathbf{0}^n$ and thus returns $\perp$. Thus, $\mathsf{id}^* = \perp$.
- $b' = P_\tau(\mathsf{id}', \mathbf{w}') = 0$, i.e., $\mathbf{c}_{\mathsf{oa},0}, \mathbf{c}_{\mathsf{oa},1}$ are ciphertexts of $\mathbf{d}'$. Thus the Open algorithm can uncover $\mathbf{d}'$, and find out the correct index j' such that $\mathbf{reg}[j][0] = \mathbf{d}'$ and $\mathbf{reg}[j'][1] = (\mathsf{bin}(j'), \mathsf{id}_{j'}, \mathbf{G}_{j',0}, \mathbf{G}_{j',1}, \mathbf{d}_{j'})$. As a result, Open returns $\mathsf{id}^* = \mathsf{id}_{j'}$. This implies that the extracted identifier id' and the opened identifier $\mathsf{id}^* = \mathsf{id}_{j'}$ are the same.

The above then implies that $\mathbf{Exp}^{\mathsf{unforge}}_{\mathcal{A}}(1^\lambda)$ returns 1 with negligible probability. $\square$